- Executive Summary: MQTT Version 6.0
  - What Is This?
  - The Problem
  - The Solution
    - 1. 32-bit Stream Sequence Numbers
    - 2. Stream Epoch
    - 3. Named Durable Queues (`$queue/` namespace)
    - 4. Pull-Based Flow Control (FETCH)
    - 5. Mandatory TLS 1.3 + Optional Payload Encryption (Zero Trust)
    - 6. Single-Queue Multi-Consumer (SQMC) Semantics
  - Business Impact
  - Compatibility Strategy
  - Scope and Standardization Path
  - Reference Implementation Path (HiveMQ)
  - Recommendation
- Why MQTT v6.0? Motivation, Rationale, and Intended Audience
  - 0. Important Context: What This Proposal Is Not
  - 1. Who This Is For
    - Primary Use Cases
    - Primary Audience
    - Who Should Stay on MQTT v5.0
  - 2. The Problem Statement: What MQTT v5.0 Cannot Do
    - 2.1 No Durable, Session-Independent Queues
    - 2.2 No Message Ordering Guarantee Across Sessions
    - 2.3 Push-Based Flow Control vs. Pull-Based Flow Control
    - 2.4 No Cluster-Aware Sequence Numbers
    - 2.5 Shared Subscriptions Have Loose Delivery Semantics
  - 3. Why Not Use Kafka or AMQP?
    - At the Edge (Sensors, RTUs, PLCs, Smart Meters)
    - At the Broker Tier (The Gap MQTT v6.0 Fills)
  - 4. Design Principles
    - Principle 1: The Protocol Must Survive Broker Restarts
    - Principle 2: Consumers Control Their Own Rate
    - Principle 3: Sequence Numbers Are a First-Class Protocol Citizen
    - Principle 4: Cluster Failover Must Be Transparent to Well-Behaved Clients
    - Principle 5: Backward Compatibility Is a Non-Negotiable Constraint
    - Principle 6: Layered Standardization
  - 5. Expected Benefits
    - For Industrial Operators
    - For Platform Engineers
    - For the MQTT Ecosystem
  - 6. Explicit Non-Goals

- 7. Relationship to the MQTT v5.0 Specification
  - Compatibility Boundaries
- MQTT Version 6.0
  - Industrial Stream and Advanced Queuing Extension
  - 1. Introduction
    - 1.1 Conformance
    - 1.2 Terminology
    - 1.3 Normative References
    - 1.4 Data Representation
  - 2. MQTT Control Packet Format
    - 2.1 Fixed Header
    - 2.2 Variable Header Properties
  - 3. MQTT Control Packets
    - 3.1 CONNECT – Connection Request
    - 3.2 CONNACK – Connect Acknowledgement
    - 3.3 PUBLISH – Publish Message
    - 3.4 SUBSCRIBE – Subscribe Request
    - 3.5 FETCH – Pull Request (Type 16) [NEW]
  - 4. Operational Behavior
    - 4.1 The `$queue/` Namespace
    - 4.2 Pull-Based Flow Control
    - 4.3 Cluster Consistency
    - 4.4 Single-Queue Multi-Consumer (SQMC)
  - 5. Summary of Changes: v5.0 vs. v6.0
  - 6. Compatibility Layer
    - 6.1 Negotiation
    - 6.2 Feature Mapping (Native → Compat)
    - 6.3 Broker Requirements for Compat Mode
  - 7. Security Considerations
    - 7.1 Access Control for `$queue/` and `$SYS/queues/`
    - 7.2 Epoch Integrity
    - 7.3 Sequence Number Side-Channel
    - 7.4 High-Watermark Persistence
    - 7.5 Transport Security — TLS 1.3 Requirement
    - 7.6 Optional Payload Encryption (Zero Trust)
  - Annex A: SECS/GEM Mapping
    - A.1 Critical Scenario: Process Start Command (S2F21)
  - Annex B: New Reason Codes
- Alternative Approaches to MQTT v6.0
  - 1. The Core Design Choice: Breaking vs. Non-Breaking
  - 2. Alternative A: Pure MQTT v5.0 Extension (Property-Based Shim)
    - Description
    - Implementation

        - Pros
        - Cons
        - The Fragmentation Risk
        - Verdict
    - 3. Alternative B: MQTT v5.0 with Reserved Property IDs (0x30–0x45)
        - Description
        - Implementation
        - Pros
        - Cons
        - Verdict
    - 4. Alternative C: Native v6.0 with New Packet Type (Chosen Approach)
        - Description
        - Implementation
        - Pros
        - Cons
        - The Dual-Mechanism Trade-Off
        - Verdict
    - 5. Alternative D: Replace MQTT Entirely (Kafka / AMQP 1.0 / NATS JetStream)
        - Description
        - Comparison
        - Analysis
        - Verdict
    - 6. Alternative E: Extend MQTT Shared Subscriptions (v5.0 `$share/`)
        - Description
        - Pros
        - Cons
        - Why `$share/` Is Not a Durable Queue
        - Verdict
    - 7. Design Decision Matrix
- MQTT v6.0 — Addressing Criticisms and Defending the Proposal
    - A Note on Where This Comes From
        - Why Standardize Now — Not “Ship an Extension First”
    - What This Proposal Is Not
    - “MQTT v5.0 Already Does This” — Point-by-Point
        - Section 2.1: “That’s a Shared Subscription”
        - Section 2.2: Packet Identifiers and Message Ordering
        - Section 2.3: “Receive Maximum Already Provides Flow Control”
        - Section 2.4: “Cluster-Aware Sequence Numbers” — On the Epoch
        - Section 2.5: Shared Subscription Semantics — Exclusive Consumer / Hot Standby
    - “This Should Be Handled at the Application Level”
        - What I Have Seen in Customer Deployments
        - The Historical Precedent

- "Do You Think MQTT Should Be a Message Queue?"
  - My Direct Answer
  - Why Not Kafka (The Answer That Never Changes)
- "Too Much HOW, Not Enough WHAT and WHY"
- Summary of Responses
- References
- Critical Perspectives, Rebuttals & Technical Pitfalls
  - Part 1: Standards Committee & Architecture Critique
    - Criticism 1: "MQTT's success is its simplicity. v6.0 turns it into a bloated Kafka clone."
    - Criticism 2: "The `$queue/` namespace breaks topic hierarchy semantics."
    - Criticism 3: "The FETCH packet (Type 16) is a hard break that undermines the whole 'compatible' story."
  - Part 2: Software Engineering & Performance Critique
    - Criticism 4: "The original proposal's Leader Node model contradicts HiveMQ's masterless architecture."
    - Criticism 5: "Replication lag will cause data loss in the replication gap window."
    - Criticism 6: "Persistent queues on the broker will cause storage bloat and operational nightmares."
  - Part 3: SRE & Operational Critique
    - Criticism 7: "The FETCH mechanism adds latency compared to push delivery."
    - Criticism 8: "Custom property IDs in the reserved range could conflict with future OASIS standardization."
  - Technical Pitfalls
    - Pitfall 1: Sequence Wraparound at 2^32
    - Pitfall 2: Idempotency Table Growth
    - Pitfall 3: Competing Consumer Fairness
    - Pitfall 4: Epoch Reset as a DoS Vector
    - Pitfall 5: Virtual FETCH Overhead (Compat Mode)
    - Pitfall 6: `$queue/` Namespace Collision with Existing Topics
  - Summary: Risk Matrix
- MQTT v6.0 Performance Analysis: High-Volume Use Cases
  - 1. Workload Profile
  - 2. Performance Bottlenecks in the Current Proposal
    - 2.1 CAS Counter Contention at Scale
    - 2.2 High-Watermark Inadequacy Under Burst
    - 2.3 Quorum Write Latency vs. Real-Time Telemetry
    - 2.4 FETCH Batch Size Tuning for Burst Recovery
    - 2.5 Rate Limiting Granularity
    - 2.6 Epoch Reset Thundering Herd
  - 3. Updated Property Table
  - 4. Namespace Additions
  - 5. Summary: Adaptations for DEP Scale

- Critical Analyses of MQTT v6.0
  - Part 1 — Security Analyst Perspective
    - 1.1 Executive Verdict
    - 1.2 Threat Model Gaps
    - 1.3 Missing Security Primitives
    - 1.4 Positive Security Properties
  - Part 2 — OT SRE Perspective
    - 2.1 Executive Verdict
    - 2.2 Operational Concerns
    - 2.3 Deployment Readiness Assessment
    - 2.4 What OT SREs Actually Need
  - Part 3 — Arlen Nipper Perspective
    - 3.1 Opening Statement
    - 3.2 The Core Argument: Protocol Creep Destroys Interoperability
    - 3.3 The Right Tool Exists
    - 3.4 The MQTT Gateway Pattern Is the Correct Architecture
    - 3.5 The Constrained Device Problem
    - 3.6 A Constructive Alternative
  - Summary Comparison
- SQMC vs. HiveMQ Declared Shared Subscriptions
  - What Declared Shared Subscriptions Does
  - The Gaps
  - The Critical Gap: `[MQTT-4.8.2-5]`
  - The Exclusive Consumer Gap
  - Why Declared Shared Subscriptions Is the Argument, Not the Counterargument
  - Summary
- The Application Layer Lift — Overview
  - The Core Insight
  - The Split: Protocol vs. Application Layer
    - Protocol Layer — Broker Obligations (MQTT 6.0 Spec)
    - Application Layer — Client Library Obligations (“The Lift”)
  - Why This Answers the Committee
  - Three Options
  - Recommendation
- Option A: MQTT 6.0 Protocol Primitives + MQTT Stream Application Profile
  - 1. Overview
  - 2. The Protocol Layer: What Stays in the MQTT 6.0 Specification
  - 3. The Application Profile Layer: The Lift
  - 4. How This Addresses Committee Criticisms
    - Simon’s “mux’ing” criticism
    - The “this should be application-level” argument
    - The “bloat” and Kafka-clone criticism
    - The “standardize now vs. extension first” tension

    - 5. Feature Split: Comprehensive Reference Table
    - 6. Standardization Path
    - 7. Trade-offs
        - Advantages
        - Disadvantages
    - References
- Option B: MQTT 6.0 with Normative Client Annex — Explicit Layer Boundary
    - 1. Overview
        - The Problem With the Current Spec Structure
        - The Option B Solution
    - 2. Proposed Spec Restructure
        - Sections 1–7 (Broker Normative — Unchanged Scope, Tightened Audience)
        - Annex A (Normative) — Stream Client Implementation Requirements
        - Annex B (Informative) — Implementation Examples
    - 3. Annex A Content: What Gets Lifted
        - A.1 Stream Consumer State Machine
        - A.2 Gap Detection
        - A.3 High-Watermark Management
        - A.4 Epoch Change Protocol
        - A.5 Reconnect and Resume
        - A.6 FETCH Pacing Strategy
        - A.7 Idempotency Table Lifecycle
        - A.8 Sequence Wraparound
    - 4. How This Addresses Committee Criticisms
        - Simon's "Mux'ing" Criticism
        - The "Application-Level" Argument
        - Procedural Argument
        - The "Bloat" Criticism
    - 5. Comparison: Current Spec vs. Option B Structure
    - 6. Trade-offs
        - Pros
        - Cons
    - 7. What the Main Body (Sections 1–7) Looks Like After Restructure
    - References
- Option C: Two Independent Specifications — MQTT 6.0 Transport Extension + MQTT Stream 1.0 Application Protocol
    - 1. Overview
    - 2. MQTT 6.0 Transport Extension Specification
        - 2.1 What MQTT 6.0 Defines
        - 2.2 What MQTT 6.0 Explicitly Does NOT Define
    - 3. MQTT Stream 1.0 Application Protocol Specification
        - 3.1 Stream Consumer State Machine
        - 3.2 Gap Detection Protocol

    - 3.3 Epoch Resync Protocol
    - 3.4 High-Watermark Protocol
    - 3.5 Exactly-Once Delivery Contract
    - 3.6 FETCH Pacing
    - 3.7 Reconnect Protocol
    - 3.8 Sequence Wraparound Rules
    - 3.9 SQMC Application Patterns
  - 4. How This Addresses Committee Criticisms
    - 4.1 Simon's "Mux'ing" Criticism
    - 4.2 The "This Should Be Application-Level" Argument
    - 4.3 The "Bloat / Kafka Clone" Criticism
    - 4.4 The "Standardize Now vs. Extension First" Argument
    - 4.5 Property ID Conflict Risk
  - 5. Governance Split
  - 6. Interoperability Contract
  - 7. Trade-offs
    - Advantages
    - Disadvantages
  - 8. Analogues in Other Protocol Ecosystems
  - References

Executive Summary: MQTT Version 6.0

What Is This?

MQTT v6.0 is a proposed evolutionary extension to the MQTT v5.0 protocol that adds **targeted industrial queuing primitives** to the broker tier — without changing the edge protocol that millions of devices already run.

It is **not** a general-purpose upgrade. It is **not** "MQTT trying to become Kafka." It targets a specific tier of deployment: **semiconductor manufacturing (SECS/GEM), energy grid SCADA (IEC 61850), and large-scale industrial IoT** — environments where MQTT v5.0 is already deployed at the edge but operators are forced to bolt on Kafka or AMQP at the broker tier to get durable queuing, ordered delivery, and consumer group semantics. v6.0 eliminates that bridge.

Why standardize now rather than ship vendor extensions first? Because the extension-first path leads to fragmentation — just as `$SYS/` was implemented differently by every broker and remains unstandardized 15 years later. Standardizing these patterns at the protocol level before vendors fragment gives the ecosystem a single interoperable wire representation. See Why Standardize Now for the full argument.

For a detailed response to criticisms including "v5.0 already does this" and "this should be application-level," see Addressing Criticisms.

The Problem

MQTT v5.0 is excellent for lightweight telemetry and command delivery. However, it has structural limitations that force enterprise users to embed reliability logic inside application payloads — a pattern already codified by Eclipse Sparkplug B with its application-layer sequence numbers, and replicated ad-hoc by every industrial MQTT deployment that needs message ordering guarantees:

- **No global message ordering.** The 16-bit Packet Identifier is per-session, transient, and recycled — making it impossible for a consumer to detect gaps caused by broker restarts or cluster failovers.
- **Push-only delivery.** The broker pushes all messages to consumers as fast as they arrive. A slow consumer cannot apply backpressure without complex out-of-band coordination.
- **Session-bound queues.** Messages are only held for a client while its session exists. There is no way to create a durable queue that outlives client connections.
- **Loose shared subscription semantics.** MQTT's `$share/` mechanism provides load balancing but lacks strict exclusive-consumer and message-locking semantics needed for transactional processing.
- **No cluster-aware consistency.** When a broker node fails and a client reconnects to a new node, the protocol has no mechanism to detect what was lost or resume from an exact point.

These gaps are currently "solved" by embedding sequence numbers, idempotency keys, and acknowledgment logic in message payloads — a non-standard, non-interoperable approach.

The Solution

MQTT v6.0 introduces five targeted additions to the protocol. In compatibility mode (Track B), these additions use v5.0 User Properties and control topics that are transparent to existing parsers. In native mode (Track A, Protocol Level 6), a new packet type (FETCH) and new property IDs are introduced that require updated client libraries and brokers:

1. 32-bit Stream Sequence Numbers

A new Property (`0x30`) carrying a monotonic, cluster-wide 32-bit integer is attached to every message in the `$queue/` namespace. This allows consumers to detect gaps (`received Seq 1002 but expected 1001`), perform deduplication, and implement application-level exactly-once processing without embedding logic in payloads.

2. Stream Epoch

A new Property (`0x35`) that tracks the “era” of a queue. When a broker cluster suffers a catastrophic failure and cannot guarantee sequence continuity, the Epoch is incremented. Consumers that detect an Epoch change know they must discard local idempotency state and re-synchronize — analogous to a SECS/GEM Establish Communications (S1F13) reset.

3. Named Durable Queues (`$queue/` namespace)

Topics prefixed with `$queue/` are treated as first-class queue entities: persisted to non-volatile storage, sequenced, and retained indefinitely until consumed or expired — independent of any client session. This replaces the implicit, session-bound queue model of v5.0.

4. Pull-Based Flow Control (FETCH)

A new `FETCH` Control Packet (Type 16) lets consumers explicitly request batches of messages from the broker. The broker holds messages in the persistent queue and releases them only when asked. This eliminates the “thundering herd” problem where a recovering consumer is flooded with backlogged messages. For v5.0-compat environments, a Virtual FETCH mechanism uses a `$SYS/` control topic instead.

5. Mandatory TLS 1.3 + Optional Payload Encryption (Zero Trust)

Native Mode v6.0 connections MUST use TLS 1.3, eliminating the vulnerable cipher suites and additional round-trip latency of TLS 1.2. For deployments where the broker must be treated as an untrusted intermediary (zero trust architectures), v6.0 introduces three optional key metadata properties (`0x3A` Key ID, `0x3B` Algorithm, `0x3C` Key Version) that allow end-to-end encrypted payloads to carry the information consumers need to decrypt them — without placing any key material in the protocol. Key management (distribution, rotation, revocation) is explicitly an application-layer responsibility. This feature is entirely opt-in; deployments using TLS 1.3 transport encryption alone are fully conformant.

6. Single-Queue Multi-Consumer (SQMC) Semantics

An extension to `SUBSCRIBE` that adds two new consumer modes beyond the loose `$share/` model: - **Competing:** Messages are distributed to exactly one consumer via round-robin, with strict message locking and immediate failover if the consumer disconnects before acknowledging. - **Exclusive:** One designated consumer receives all messages; others are hot standbys that take over instantly on failure, preserving strict ordering.

Business Impact

Use Case	v5.0 Risk	v6.0 Benefit
Semiconductor FAB (SECS/GEM)	Broker restart can silently lose a “Process Start” (S2F21) command, ruining a wafer	Epoch resync + sequence gaps ensure the command is detected as lost and retried
Industrial FDC Data Collection	High-frequency sensor data floods the host system, causing crashes	FETCH-based backpressure keeps the broker as the buffer; host processes at its own rate
Financial Message Bus	Duplicate processing of payment events due to lack of idempotency primitives	32-bit Sequence as an idempotency key, checked before executing business logic
Multi-node Broker Cluster	Client failover to a new node silently re-delivers or loses messages	Last Received Seq in CONNECT allows new node to resume delivery from exact position
High-availability IoT Gateways	Primary/backup consumer logic must be hand-coded in application layer	Exclusive consumer mode with automatic hot-standby failover built into the protocol

Compatibility Strategy

MQTT v6.0 is backward-compatible with MQTT v5.0 for all unchanged v5.0 use cases: existing packet types, properties, topics, and semantics continue to work as before. A v5.0 or v3.1.1 client connecting to a v6.0 broker for standard pub/sub sees no difference in behavior.

Compatibility is not wire-transparent for v6.0-only features (FETCH, `$queue/`, Stream Sequence properties, `last-seq` / `epoch`); those require either Protocol Level 6 or the specified v5.0 compatibility mode. Specifically:

- All new properties use IDs in the currently unassigned range (`0x30–0x45`). The v5.0 spec requires conformant implementations to ignore unknown property IDs, so these properties are safe in mixed environments. However, these IDs are not formally reserved for v6.0 until an OASIS submission is accepted — a pre-standardization risk that the Compatibility Layer (User Properties) mitigates.
- The `$queue/` namespace is ACL-protected; v5.0 clients that have not completed the v6.0 handshake are rejected.
- A **Compatibility Layer** maps every native v6.0 feature to v5.0 User Properties and system topics, enabling mixed-version deployments.
- A negotiation handshake (`mqtt-ext: v6.0` in CONNECT/CONNACK) allows clients and brokers to detect v6.0 support and fall back gracefully.

The key guarantee: if you do not use `$queue/`, FETCH, or Stream Sequence properties, your deployment is fully compatible with v5.0 — no behavioral changes, no additional overhead, no migration required. See Compatibility Boundaries for the full scenario matrix.

Security considerations for the new primitives — including ACL requirements for `$queue/`, Epoch integrity, sequence number side-channels, and High-Watermark persistence — are defined normatively in Section 7 of the specification.

Scope and Standardization Path

This proposal is intentionally scoped to a single problem domain: adding durable, ordered queuing primitives for industrial broker-tier use cases. It does not attempt to replace general-purpose message queues, add stream processing, or modify MQTT's core pub/sub semantics. The features are additive and opt-in.

The recommended standardization path is **layered**: submit the core primitives (Stream Sequence, Epoch, `$queue/` namespace) as a first submission, with FETCH and SQMC as follow-on extensions if the core is accepted. This allows the committee to evaluate the proposal incrementally.

Reference Implementation Path (HiveMQ)

A reference implementation can be built using the HiveMQ Extension SDK:

Feature	HiveMQ API
Sequence injection	`PublishInboundInterceptor`
Client throttling	`ClientService.blockClient()` / `unblockClient()`
Queue persistence	`ManagedPersistenceService`
Cluster epoch management	`ClusterService` + consistent hashing on queue names

A proof-of-concept can be built today using a HiveMQ Extension alongside a Python `gmqtt` client shim, without any changes to the core broker binary.

Recommendation

Adopt MQTT v6.0 as a **two-phase rollout**:

1. **Phase 1 — Compatible Extension:** Deploy the HiveMQ Extension Plugin that intercepts `v6-*` user properties. Migrate clients to the Python/Java shim. Zero broker downtime

required.
2. **Phase 2 — Native v6.0:** Once the ecosystem supports Protocol Level 6, deprecate the shim layer and use native FETCH packets with full binary efficiency.

This proposal is production-ready as a HiveMQ extension today and provides a clear path to a formal OASIS standardization submission.

Why MQTT v6.0? Motivation, Rationale, and Intended Audience

> ***Status:*** *Part of the MQTT v6.0 Draft Proposal — March 2026*

---

0. Important Context: What This Proposal Is Not

This is not "MQTT trying to become Kafka." This is not a general-purpose upgrade. Most MQTT deployments do not need v6.0 and should remain on v5.0.

This proposal targets a specific problem: **industrial operators who already use MQTT at the edge but are forced to bolt on Kafka, AMQP, or Redis at the broker tier** to get durable queuing and ordered delivery. The edge devices cannot change — they run constrained firmware with MQTT baked in. The bridge architecture (MQTT → Kafka → Consumer) adds operational cost, a second failure domain, and a second protocol stack. v6.0 eliminates the bridge by extending the MQTT broker with the specific queuing primitives these operators need, without changing the edge protocol.

Every feature in v6.0 is already being implemented at the application layer by industrial MQTT users — sequence numbers in payloads, idempotency tables in databases, consumer group coordination in custom code. Eclipse Sparkplug B is the most prominent example: it defines application-layer sequencing on top of MQTT precisely because MQTT v5.0 does not provide these primitives natively. v6.0 proposes to move these proven patterns from the payload into the protocol where brokers can optimize for them and client libraries can handle them automatically.

Why not ship these as vendor extensions first? Because the extension-first path leads to fragmentation, not standardization. If HiveMQ ships `$queue/` as a proprietary extension, EMQX, Mosquitto, and AWS IoT Core will build incompatible versions — just as every broker implemented `$SYS/` differently, creating a mess that remains unstandardized 15 years later. HiveMQ's own Declared Shared Subscriptions — a proprietary workaround for the durable

queue gap in `$share/` — is proof that extension-driven solutions do not converge on interoperability. Standardizing early, before fragmentation, is cheaper than standardizing late.

For a detailed response to specific criticisms (including “this is Kafka,” “v5.0 already does shared subscriptions,” and “this should be application-level”), see Addressing Criticisms — Point-by-Point Rebuttals.

---

1. Who This Is For

MQTT v6.0 is targeted at a specific tier of industrial and enterprise IoT deployments that have **outgrown what MQTT v5.0 was designed to provide**. It is not a general-purpose upgrade for all MQTT users.

Primary Use Cases

Semiconductor Manufacturing (SECS/GEM): Semiconductor fabs run the SEMI E30 (GEM) protocol for equipment communication. A lost `S2F21` (Remote Process Start) command can ruin a wafer worth $10K–$50K. Today, fabs using MQTT must embed sequence numbers and idempotency keys in application payloads because MQTT v5.0 has no protocol-level mechanism to detect a gap caused by a broker restart. Every equipment vendor implements this differently.

Energy and Grid SCADA (IEC 61850): Distributed energy providers operate millions of smart meters, PMUs, RTUs, and substation gateways. MQTT is increasingly used as the edge transport for IEC 61850 telemetry, but downstream data pipelines require ordered, durable queues with 90-day regulatory retention. Today this means an MQTT-to-Kafka bridge — a second system, a second failure domain, a second team.

Primary Audience

Audience	Why They Need v6.0
Large-scale industrial operators — energy utilities, grid SCADA, semiconductor fabs, process manufacturing	Need ordered, durable message queues that survive broker restarts and client disconnections; need competing consumer patterns for high-availability pipelines
IIoT platform engineers	Building middleware that must guarantee message delivery ordering across distributed broker clusters without application-layer workarounds
Data pipeline architects	Connecting MQTT edge data to downstream analytics, historians, and SCADA systems; need pull-based flow control so consumers can dictate their own ingestion rate

Audience	Why They Need v6.0
HiveMQ extension developers	Implementing advanced queuing semantics today via custom extensions; v6.0 standardises and formalises these patterns
MQTT broker implementers	Seeking a standardised wire protocol for durable queuing so customers are not locked into proprietary extension APIs

Who Should Stay on MQTT v5.0

MQTT v6.0 is designed as a strict superset of MQTT v5.0: existing publish-subscribe use cases continue to work unchanged, while deployments that require durable queues and ordered replay can opt into the new capabilities. If your deployment matches any of the following profiles, v5.0 is the right choice and v6.0 adds complexity without benefit:

- Simple telemetry from sensors to a single backend consumer
- Devices with less than 256 KB available memory
- Short message lifetimes where loss is acceptable (dashboards, live monitoring)
- No requirement for competing consumers or consumer group semantics
- Broker infrastructure managed by a single operator with no cluster failover requirements

These deployments can connect to a v6.0 broker without modification — the broker serves v5.0 and v3.1.1 clients identically to a v5.0 broker. The v6.0 extensions are only activated when a client explicitly opts in via the v6.0 handshake.

---

2. The Problem Statement: What MQTT v5.0 Cannot Do

MQTT v5.0 is a well-designed protocol for lightweight publish-subscribe messaging. It was not designed to be a message queue. Five specific limitations emerge at scale in industrial deployments:

2.1 No Durable, Session-Independent Queues

MQTT v5.0 sessions are bound to a client identifier and a broker node. When a client disconnects: - Clean session (`cleanStart=true`): all queued messages are discarded - Persistent session (`cleanStart=false`): messages are held *for that specific client*, not for a named consumer group

The result: If a downstream analytics pipeline (one of ten competing consumers) disconnects for maintenance, its share of the message backlog is either lost or held indefinitely in a single-consumer session — neither behaviour is correct for an industrial pipeline.

What operators do today: They build application-layer queuing on top of MQTT using Redis, Kafka, or proprietary HiveMQ extensions. The MQTT broker becomes a thin transport layer and

the real queuing logic lives outside the protocol. This defeats the purpose of using a standardised protocol.

2.2 No Message Ordering Guarantee Across Sessions

MQTT v5.0 includes a Packet Identifier, but it is not a message sequence number. It exists only for QoS delivery handshakes, is scoped to a single client session, is reused after acknowledgement, and resets when the session reconnects. It therefore cannot be used to determine message ordering or detect gaps across sessions.

MQTT v6.0 introduces a broker-assigned, monotonic sequence number (Property `0x30`) that identifies the message itself rather than the delivery attempt.

MQTT v5.0 uses a 16-bit Packet Identifier that resets per session. Messages across sessions cannot be ordered. If a consumer reconnects mid-stream, it cannot determine which messages it has already processed and which it has not — not without an application-layer sequence number that the publisher must attach manually.

The result: Exactly-once processing at the application layer requires every publisher and consumer pair to implement bespoke deduplication logic. In a system with 50 publishers and 20 consumers, this is 1,000 pairwise agreements — none of them standardised.

What “1,000 pairwise agreements” means in practice:

When sequence numbers live in the payload rather than the protocol, every publisher and every consumer must agree on the payload format — and that agreement must happen pair by pair. Consider a semiconductor fab with 50 tools (publishers) sending messages to 20 downstream systems (consumers) — historians, MES, FDC analyzers, dashboards:

- Tool A sends `{"seq": 123, "data": ...}` (JSON, field name `seq`)
- Tool B sends a Sparkplug protobuf with a `sequence_number` field
- Tool C uses a custom binary header with bytes 0–3 as a big-endian sequence counter

Every consumer that wants to do deduplication or gap detection on messages from Tool A needs to know Tool A’s format. And Tool B’s format. And Tool C’s format. Each publisher-consumer pair is a “pairwise agreement” about how to find the sequence number in the payload.

50 publishers × 20 consumers = 1,000 pairs that each need to agree on a schema. In practice, teams reduce this by picking a common format — but that common format is itself non-standard. It is a company-internal convention, not a protocol guarantee. Different companies, different vendors, and different integrations all pick different conventions. The result is that interoperability between organizations requires yet another translation layer.

The v6.0 fix: The broker assigns the sequence number as a protocol property (`0x30`). Every publisher’s message gets a sequence number automatically — the publisher does not even

need to be aware of it. Every consumer reads it from the same property, regardless of payload format. The 1,000 pairwise agreements collapse to zero. The protocol is the agreement.

Put simply: **when reliability metadata is in the payload, everyone has to agree on the payload format. When it is in the protocol, the protocol is the agreement.**

But Wait — Doesn't MQTT 5.0 Already Have a Sequence Number?

MQTT v5.0 does include a Packet Identifier, but it is not a message sequence number in the sense required for ordering or replay.

The Packet Identifier: - Is 16-bit and wraps frequently - Exists only for QoS handshake state, not for message identity - Is scoped to a single client session - Is reused once the QoS exchange completes - Resets on reconnect

Because of these properties, the Packet Identifier cannot be used to determine message ordering across sessions or after reconnect. If a consumer disconnects and reconnects, the next message may reuse an earlier Packet Identifier, making it impossible to determine whether the message is new, duplicated, or missing.

In other words, **the Packet Identifier tracks delivery acknowledgement state, not message position in a stream.**

This is why industrial deployments that require ordering or deduplication add their own application-layer sequence numbers in the payload. Sparkplug B, for example, defines its own `seq` field precisely because MQTT provides no protocol-level sequence for messages.

MQTT v6.0 introduces a true message sequence number: - **Broker-assigned** — publishers do not need to maintain counters - **Cluster-wide** — valid across all nodes, not scoped to a single connection - **Monotonic** — always increasing within a queue's lifetime - **Immutable once assigned** — the sequence number is permanent - **Stable across client sessions** — persists through disconnections and reconnections

This sequence number (Property `0x30`) identifies **the message itself**, not the delivery attempt.

The distinction is crucial:

	MQTT v5.0 Packet Identifier	MQTT v6.0 Stream Sequence Number
Purpose	Tracks QoS handshake state	Tracks message position in the stream
Scope	Per-session	Cluster-wide
Lifecycle	Reused and reset	Monotonic and immutable

	MQTT v5.0 Packet Identifier	MQTT v6.0 Stream Sequence Number
Ordering	Not usable for ordering	Basis for ordering, deduplication, and replay

2.3 Push-Based Flow Control vs. Pull-Based Flow Control

MQTT v5.0 provides flow control via `Receive Maximum` — a mechanism that caps the number of QoS 1/2 messages the broker can have in-flight simultaneously. This is a valid and effective mechanism for bounding the push window. Combined with deliberate ack pacing, it allows consumers to influence the delivery rate.

However, `Receive Maximum` operates within a **push model**: the broker initiates delivery as soon as an acknowledgment frees a slot. The consumer controls the *rate* of delivery but not the *timing*. There is no mechanism for a consumer to say "hold all messages until I ask for them" without disconnecting (which discards session state if `cleanStart=true`) or withholding acknowledgments (which violates the spirit of QoS contracts).

FETCH introduces a **different semantic model** — pull-based consumption — where the consumer explicitly requests batches. This is not a replacement for `Receive Maximum` ; both serve valid purposes. `Receive Maximum` is appropriate for live pub/sub where bounded push is correct. FETCH is for queue consumption where the consumer needs absolute control over *when* it receives messages, not just *how many*.

The practical difference: A slow consumer — one doing a database write per message — under the push model fills its in-flight window, processes it, and immediately receives another window. It can never pause, drain its backlog, or catch up without disconnecting. Under the pull model, it fetches when ready. No FETCH = no delivery. Messages remain safely in the durable queue.

2.4 No Cluster-Aware Sequence Numbers

MQTT v5.0 has no concept of cluster topology. If a client is connected to Node A and Node A fails, the client reconnects to Node B. From the protocol's perspective, this is a new session — there is no way to tell Node B "I had processed up to message X on Node A; please resume from X+1."

The result: After any broker failover, consumers must either: - Re-process all messages from the beginning of the session (duplicate processing) - Trust that the application layer tracked its own position (requires external state) - Accept a gap (message loss)

None of these outcomes are acceptable for regulatory data, safety-critical commands, or financial telemetry.

2.5 Shared Subscriptions Have Loose Delivery Semantics

MQTT v5.0 introduced Shared Subscriptions (`$share/`) for competing consumers. However: - There is no guaranteed round-robin; delivery order within a group is implementation-defined - There is no concept of exclusive access (hot-standby primary/backup) - If a consumer in the group disconnects mid-processing, the in-flight message may be re-delivered to any other consumer — but only if the broker's session timeout has not expired

The result: Building a reliable competing consumer pipeline on `$share/` requires extensive application-layer coordination that must be re-implemented for each new deployment.

---

3. Why Not Use Kafka or AMQP?

This is the most frequently raised objection. The answer depends on **where in the architecture** the problem exists.

At the Edge (Sensors, RTUs, PLCs, Smart Meters)

Kafka and AMQP are not viable at the edge: - Kafka's minimum client footprint requires a full JVM stack with multi-GB RAM — not compatible with microcontrollers or constrained devices - AMQP 1.0 is a complex framing protocol — correct implementation requires significant expertise and is poorly suited to constrained devices - Kafka requires TCP connections with persistent state; many OT networks have unreliable connectivity and require disconnect-tolerant protocols - MQTT v5.0 is already deployed on hundreds of millions of devices with battle-tested client libraries for C, MicroPython, Java, and Go

Conclusion: At the edge, MQTT remains the right protocol. The question is what happens to messages *after* they reach the broker cluster.

At the Broker Tier (The Gap MQTT v6.0 Fills)

Today, industrial deployments solve the queuing problem with a pattern like this:

```
[Edge device MQTT v5.0] → [MQTT Broker] → [Bridge to Kafka/AMQP] → [Consumer]
```

This architecture requires: - A bridge component that must be deployed, scaled, and maintained - Two separate protocol stacks, two separate failure domains - Message translation overhead at the bridge - Kafka or AMQP client libraries in every downstream consumer

MQTT v6.0 eliminates the bridge by extending the MQTT broker to natively support the queuing semantics that consumers need. The edge devices continue to speak MQTT v5.0 unchanged. The downstream consumers get ordered, pull-based, durable message queues — without a separate message queue system.

```
[Edge device MQTT v5.0] → [HiveMQ v6.0 Broker — native queuing] → [Consumer via FETCH]
```

This is not “MQTT trying to become Kafka.” It is MQTT extending its native durability semantics to the broker tier where the queuing problem actually lives.

---

4. Design Principles

MQTT v6.0 is built on six explicit principles, each derived from a lesson learned in large-scale industrial deployments:

Principle 1: The Protocol Must Survive Broker Restarts

A `$queue/` message that has been published MUST be retained by the broker cluster regardless of individual node failures. The protocol defines what “retained” means (quorum write), when sequence continuity can be guaranteed (same Epoch), and when it cannot (Epoch increment on counter quorum loss).

This is a departure from v5.0’s “retained message” feature, which is per-topic and not queue-semantics aware.

Principle 2: Consumers Control Their Own Rate

Pull-based FETCH (not push) is the default for `$queue/` consumption. A consumer that is slow, restarting, or catching up a backlog is never flooded. It fetches when it is ready. This is an intentional departure from v5.0’s push-all model — but it applies **only to `$queue/` topics**, not to standard pub/sub.

Push subscriptions remain available for all non-queue topics and for use cases where push semantics are correct (live telemetry monitoring, real-time dashboards). A v6.0 broker serving standard pub/sub traffic behaves identically to a v5.0 broker.

Principle 3: Sequence Numbers Are a First-Class Protocol Citizen

Every `$queue/` message has a broker-assigned, cluster-wide monotonic sequence number (Property `0x30`). Publishers do not assign sequence numbers — the broker does, atomically, with no routing constraints. Sequence numbers are: - Immutable once assigned - Cluster-wide (not per-session or per-node) - The basis for exactly-once deduplication at the consumer - The basis for gap detection and recovery

Principle 4: Cluster Failover Must Be Transparent to Well-Behaved Clients

A client that stores its `last-seq` and `epoch` values can reconnect to *any* node in the cluster after any failure and resume processing from exactly where it left off — provided the cluster

maintained quorum. If quorum was lost (an Epoch increment event), the protocol signals this explicitly so the client can decide how to recover.

No application-layer failover logic is required. No broker-specific reconnect APIs. The protocol carries all the state the client needs.

Principle 5: Backward Compatibility Is a Non-Negotiable Constraint

MQTT v6.0 is backward-compatible with MQTT v5.0 **for all unchanged v5.0 use cases**: existing packet types, properties, topics, and semantics continue to work as before. Specifically:

- A v6.0 broker can serve v5.0 and v3.1.1 clients without modification — these clients see no difference in behavior
- The migration from v5.0 to v6.0 does not require a flag-day cutover

Honest about the limits: Compatibility is not wire-transparent for v6.0-only features. The proposal defines two tracks:

- **Track A (Native v6.0, Protocol Level 6):** Uses FETCH (Type 16) and new property IDs. This is a **breaking change** — a v5.0 broker will reject Protocol Level 6 connections, and a v5.0 client receiving a Type 16 packet will close the connection. Track A requires updated brokers and client libraries.
- **Track B (Compatibility Mode, Protocol Level 5):** Uses User Properties and Virtual FETCH via `$SYS/` control topics. This works on any v5.0 broker with a v6.0-aware extension plugin. It is functionally complete but carries string-encoding overhead and is not a formal standard — it is an extension convention.

Track B is not a transparent bridge — it requires a v6.0-aware extension on the broker and v6.0-aware client code. It is a migration path, not invisible compatibility. See §7 Compatibility Boundaries for exact details.

Principle 6: Layered Standardization

The proposal is structured so that it can be submitted to OASIS incrementally:

1. **Core layer** (submit first): Stream Sequence Number (`0x30`), Stream Epoch (`0x35`), `$queue/` namespace and persistence semantics. These are the minimum viable primitives.
2. **Consumption layer** (follow-on): FETCH packet (Type 16), Virtual FETCH, batch semantics. Depends on the core layer.
3. **Consumer group layer** (follow-on): SQMC competing/exclusive modes, consumer group management. Depends on the core layer.

This layering allows the standards committee to evaluate and accept the core primitives independently, without requiring approval of the full feature set in a single submission.

---

5. Expected Benefits

For Industrial Operators

Benefit	Mechanism
Message loss eliminated for `$queue/` topics	Quorum write + HWM-based exactly-once
Consumer restart is safe — no re-processing required	`last-seq` + `epoch` in CONNECT; broker resumes from N+1
Slow consumers cannot be flooded	Pull-based FETCH; consumer controls batch size and timing
Competing consumers with correct failover	SQMC competing mode: in-flight locks re-dispatched on consumer crash
Hot-standby consumer for safety-critical commands	SQMC exclusive mode: single active consumer with automatic promotion
Broker failover transparent to clients	Epoch-based resync protocol

For Platform Engineers

Benefit	Mechanism
Eliminates MQTT-to-Kafka bridge for queue use cases	Native `$queue/` semantics in the broker
Standardised competing consumer semantics	SQMC spec replaces bespoke `$share/` orchestration
Sequence numbers enable cross-system correlation	`Seq` + `Epoch` = immutable event identifier for audit, replay, join
No external state required for exactly-once	HWM tracked by broker + client; no Redis or database required

For the MQTT Ecosystem

Benefit	Mechanism
Clear upgrade path from v5.0	Compatibility layer; no flag-day migration
Standardised semantics reduce broker lock-in	Wire protocol defines queue behaviour, not just transport

Benefit	Mechanism
Opens MQTT to new use cases	Financial telemetry, ERP integration, command-and-control audit trails

---

6. Explicit Non-Goals

MQTT v6.0 explicitly does not aim to:

- Replace Kafka or RabbitMQ for server-to-server batch processing at data centre scale
- Provide full AMQP 1.0 semantics (routing topologies, exchanges, dead-letter queues)
- Support messages larger than the current MQTT maximum (256 MB per PUBLISH)
- Implement stream processing or CEP (complex event processing)
- Define a management API or REST interface for broker administration
- Mandate a specific cluster topology or replication protocol (the spec defines *what* must be guaranteed, not *how*)

---

7. Relationship to the MQTT v5.0 Specification

MQTT v6.0 is a **strict superset of MQTT v5.0**: existing publish-subscribe use cases continue to work unchanged, while deployments that require durable queues and ordered replay can opt into the new capabilities. Every feature defined in the OASIS MQTT v5.0 specification remains valid and unchanged. The additions in v6.0 are:

1. **New packet type**: FETCH (Control Packet Type 16) — absent from v5.0
2. **New property identifiers**: `0x30` through `0x35`, `0x41` through `0x42` — reserved ranges not used by v5.0
3. **New topic namespace**: `$queue/` — conventionally reserved (similar to `$SYS/`) but now formally specified
4. **New connection negotiation**: Protocol Level `6` in CONNECT; `mqtt-ext: v6.0` User Property for compat mode
5. **New CONNECT fields**: `last-seq` and `epoch` for stateful reconnection

No existing v5.0 packet types, property identifiers, or semantics are modified or removed.

Compatibility Boundaries

MQTT v6.0 is backward-compatible with MQTT v5.0 for all unchanged v5.0 use cases. The following table defines exactly when compatibility is guaranteed and when compatibility mode is required:

Scenario	Compatible?	Notes
v5.0 client → v6.0 broker, standard pub/sub	**Yes — fully transparent**	Client sees no difference. No v6.0 features activated.
v3.1.1 client → v6.0 broker, standard pub/sub	**Yes — fully transparent**	Same as above.
v6.0 client → v6.0 broker, `$queue/` + FETCH	**Yes — native mode**	Protocol Level 6. Full binary efficiency.
v6.0 client → v5.0 broker, compatibility mode	**Yes — via compat layer**	Client uses Protocol Level 5 + `mqtt-ext: v6.0` User Property. FETCH tunneled via `$SYS/queues/*/fetch`. Seq/Epoch carried as User Properties. Requires v6.0-aware extension on the v5.0 broker.
v6.0 client → v5.0 broker, native FETCH packet	**No — connection rejected**	v5.0 broker rejects Protocol Level 6 with Reason Code `0x84`. Client MUST fall back to compatibility mode.
v5.0 client → v6.0 broker, `$queue/` topics	**No — rejected by ACL**	v6.0 broker MUST reject `$queue/` access from clients that have not completed the v6.0 handshake.
v6.0 properties on non-queue topics	**Ignored**	v5.0 clients and brokers MUST ignore unknown property IDs per the v5.0 spec. No harm, no benefit.

The key guarantee: If you do not use `$queue/`, FETCH, Stream Sequence, or `last-seq` / `epoch`, your deployment is fully compatible with v5.0 — no behavioral changes, no additional overhead, no migration required. The v6.0 extensions activate only when explicitly opted into.

MQTT Version 6.0

Industrial Stream and Advanced Queuing Extension

OASIS Standard Draft Date: March 2026 **Author:** Bill Sommers, AI Solutions Lead, HiveMQ, Inc. **Base Specification:** MQTT Version 5.0 OASIS Standard (https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html)

---

1. Introduction

MQTT is a Client-Server publish/subscribe messaging transport protocol. It is lightweight, open, simple, and designed to be easy to implement.

[MODIFIED for v6.0]: Version 6.0 extends this simplicity with **Deterministic Stream Semantics**. These features provide the reliability required for mission-critical industrial environments — including semiconductor fabrication (SECS/GEM), financial messaging, and large-scale IoT deployments — where message ordering, pull-based flow control, and cluster-wide consistency are mandatory.

MQTT v6.0 is designed to be **wire-compatible** with MQTT v5.0 when the Compatibility Layer is active. All v6.0 extensions utilize the Property and User Property fields that v5.0 mandates be ignored when unrecognized. In Native Mode, v6.0 introduces a new Control Packet type and requires Protocol Level 6.

1.1 Conformance

A Client or Server is conformant to MQTT v6.0 if it:

- Meets all normative requirements of MQTT v5.0, AND
- Correctly handles Properties `0x30` (Stream Sequence Number) and `0x35` (Stream Epoch), AND
- Implements the `$queue/` namespace persistence semantics described in Section 4.1, AND
- Implements the v6.0 negotiation handshake described in Section 3.1.

1.2 Terminology

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY**, and **OPTIONAL** in this document are to be interpreted as described in RFC 2119.

[NEW for v6.0] Additional Terms:

Term	Definition
Stream Sequence Number	A 32-bit monotonic integer uniquely identifying a message within a `$queue/` namespace, persistent across sessions

Term	Definition
Stream Epoch	A 16-bit counter representing the “era” of a queue; incremented when cluster failover may have caused sequence discontinuity
FETCH	New Control Packet (Type 16) for pull-based message retrieval from persistent queues
`$queue/`	Topic namespace prefix designating a named, durable, session-independent persistent queue
SQMC	Single-Queue Multi-Consumer — a queuing pattern where each message is delivered to exactly one of multiple active consumers
Virtual FETCH	Compatibility-mode representation of FETCH using PUBLISH to `$SYS/queues/{name}/fetch`
High-Watermark	The last sequence number successfully processed by a consumer client
Distributed Sequence Counter	A cluster-wide atomic counter, one per `$queue/` ; any node may increment it atomically (e.g., via compare-and-swap) to claim the next sequence number. The storage and replication mechanism is implementation-defined.
Competing Consumer	SQMC mode: messages distributed round-robin; each message delivered to exactly one consumer
Exclusive Consumer	SQMC mode: one designated consumer receives all messages; others are hot standbys

1.3 Normative References

- MQTT Version 5.0 OASIS Standard: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html
- RFC 2119: Key words for use in RFCs to Indicate Requirement Levels

1.4 Data Representation

All data representation follows MQTT v5.0 conventions.

[NEW for v6.0]: - **Four Byte Integer:** A 32-bit unsigned integer in big-endian order. Used for Stream Sequence Numbers. - **Two Byte Integer:** A 16-bit unsigned integer in big-endian order. Used for Stream Epochs.

---

2. MQTT Control Packet Format

2.1 Fixed Header

Every MQTT Control Packet begins with a Fixed Header. The format is unchanged from v5.0.

```
Bit:     7    6    5    4    3    2    1    0
Byte 1:  [ MQTT Control Packet Type  ] [Flags]
Byte 2+: [        Remaining Length        ]
```

[NEW for v6.0] Control Packet Type 16 — FETCH:

Type Value	Name	Direction	Description
16	FETCH	Client to Server	Pull-based request for a batch of messages from a named queue

> ***Note on Compatibility:*** *The Type 16 slot was reserved in v5.0. A strict v5.0 broker will close the connection upon receiving a Type 16 packet. In environments with mixed v5.0/v6.0 brokers, use the Virtual FETCH mechanism (Section 4.2.1) instead.*

2.2 Variable Header Properties

MQTT v6.0 introduces four new Property Identifiers in the reserved range (`0x30–0x45`):

Hex	Name	Data Type	Applicable Packets
0x30	**Stream Sequence Number**	Four Byte Integer	PUBLISH, FETCH (response)
0x35	**Stream Epoch**	Two Byte Integer	PUBLISH, CONNACK, DISCONNECT
0x41	**Throughput Limit**	Four Byte Integer	CONNACK
0x42	**Batch Size**	Four Byte Integer	FETCH, PUBLISH (Virtual FETCH)

2.2.1 Stream Sequence Number (0x30)

- **Type:** Four Byte Integer (32-bit unsigned, big-endian)
- **Packets:** PUBLISH (for `$queue/` topics)
- **Normative:** The Broker MUST assign a monotonically increasing value to this property for every message published to a `$queue/` topic. If the Publisher provides this value, the

Broker MUST validate it and MAY override it.
- **Purpose:** Provides a persistent, end-to-end message identity that survives transport-layer reconnections, session restarts, and cluster failovers. Unlike the 16-bit Packet Identifier (which is per-session and recycled), this value is globally unique within a queue's lifetime.

2.2.2 Stream Epoch (0x35)

- **Type:** Two Byte Integer (16-bit unsigned, big-endian)
- **Packets:** PUBLISH, CONNACK, DISCONNECT
- **Normative:** The Broker MUST include the current Epoch in CONNACK for resuming sessions. The Broker MUST increment the Epoch when it cannot guarantee sequence continuity (see Section 4.3.2 for the exhaustive list of triggering conditions). A Client that receives an Epoch value greater than its stored Epoch MUST discard its local idempotency table and perform a full application-layer resynchronization.
- **Purpose:** Identifies the "era" of a queue's sequence space. Allows clients to distinguish between a sequence gap (a missed message) and a cluster reset (all previous sequence context is invalid).

2.2.3 Throughput Limit (0x41)

- **Type:** Four Byte Integer (KB/second)
- **Packets:** CONNACK
- **Normative:** If present, the Client MUST NOT exceed this throughput. If the Client exceeds the limit, the Broker MAY disconnect with Reason Code `0x9B (Quota Exceeded)`.
- **Purpose:** Broker-enforced rate limiting for hardware NIC and storage protection.

2.2.4 Batch Size (0x42)

- **Type:** Four Byte Integer (message count)
- **Packets:** FETCH (native); PUBLISH to `$SYS/queues/{name}/fetch` (compat mode)
- **Normative:** The Broker MUST release at most this many messages in response to a FETCH or Virtual FETCH request. If fewer messages are available, the Broker MUST release only the available count.

---

3. MQTT Control Packets

3.1 CONNECT – Connection Request

3.1.2.2 Protocol Version

- **v5.0:** Protocol Level value is `5`
- **[MODIFIED for v6.0]:** Protocol Level value is `6` for native v6.0 mode. Clients MAY use Protocol Level `5` combined with the v6.0 User Property for Compatibility Mode operation.

3.1.2.11 CONNECT Properties

All v5.0 CONNECT properties are retained. The following property is added:

[NEW for v6.0] v6.0 Capability Negotiation: - **User Property Key:** `mqtt-ext` - **User Property Value:** `v6.0` - **Normative:** A Client wishing to use `$queue/` namespaces, FETCH semantics, or SQMC consumer modes MUST include this property. A Client that does not include it MUST be treated as a legacy v5.0 Client by the Broker.

[NEW for v6.0] Resync on Reconnect: - **User Property Key:** `v6-last-seq` - **User Property Value:** The decimal string representation of the last successfully processed Stream Sequence Number - **User Property Key:** `v6-epoch` - **User Property Value:** The decimal string representation of the Client's current known Epoch - **Normative:** Upon reconnect, a Client SHOULD include these properties to enable seamless queue resumption. The Broker MUST use these values to resume delivery from the correct position (see Section 4.3).

3.2 CONNACK – Connect Acknowledgement

3.2.2.3 CONNACK Properties

All v5.0 CONNACK properties are retained. The following properties are added:

[NEW for v6.0] v6.0 Capability Confirmation: - **User Property Key:** `mqtt-ext` - **User Property Value:** `v6.0` - **Normative:** A v6.0-aware Broker MUST echo this property. If absent, the Client MUST assume a legacy v5.0 Broker and MUST NOT use `$queue/` namespaces or FETCH semantics.

[NEW for v6.0] Throughput Limit (0x41): - See Section 2.2.3.

[NEW for v6.0] Stream Epoch (0x35): - **Normative:** REQUIRED when the Client included `v6-epoch` in CONNECT. If the Broker's current Epoch differs from the Client's reported Epoch, the Broker MUST include the new Epoch in CONNACK. The Client MUST respond per Section 2.2.2.

3.3 PUBLISH – Publish Message

3.3.2 PUBLISH Variable Header

The PUBLISH packet, when directed to a `$queue/` topic, triggers additional broker-side processing.

[MODIFIED for v6.0]: ~~The Server MUST push all matching messages to connected Clients.~~ For `$queue/` topics, the Server MUST NOT push messages unless a FETCH request is active or push mode is explicitly enabled. For standard topics, push behavior is unchanged.

3.3.2.3 PUBLISH Properties

All v5.0 PUBLISH properties are retained. The following properties are added:

[NEW for v6.0] Stream Sequence Number (0x30): - **Normative:** REQUIRED for all PUBLISH packets to `$queue/` topics. The Broker MUST assign this value before persisting the message. See Section 2.2.1.

[NEW for v6.0] Stream Epoch (0x35): - **Normative:** REQUIRED for all PUBLISH packets to `$queue/` topics. The Broker MUST include the current Epoch. See Section 2.2.2.

[NEW for v6.0] Priority: - **User Property Key:** `v6-priority` - **User Property Value:** Integer 0–255 (higher values = higher priority) - **Normative:** OPTIONAL. Higher-priority messages SHOULD be delivered to consumers before lower-priority messages in the same queue.

3.4 SUBSCRIBE – Subscribe Request

3.8.2.1 SUBSCRIBE Properties

All v5.0 SUBSCRIBE properties are retained. The following User Properties are added for SQMC semantics:

[NEW for v6.0] Consumer Semantics: - **User Property Key:** `v6-semantics` - **Values:** - `competing` — Enables Competing Consumer mode (round-robin; each message delivered to exactly one consumer; message locked until PUBACK received; immediate failover on disconnect) - `exclusive` — Enables Exclusive Consumer mode (primary consumer receives all messages; standby consumers receive nothing until primary disconnects) - **Normative:** If neither value is present, the subscription behaves as standard v5.0 Shared Subscription semantics (if `$share/` prefix is used) or normal subscription semantics otherwise.

[NEW for v6.0] Consumer Group: - **User Property Key:** `v6-group` - **User Property Value:** A UTF-8 string naming the consumer group - **Normative:** Clients in the same group sharing a `$queue/` topic with `competing` semantics will receive messages via round-robin distribution. Required when using `v6-semantics: competing`.

3.5 FETCH – Pull Request (Type 16) [NEW]

The FETCH packet is a new Control Packet introduced in v6.0. It enables pull-based message retrieval from persistent queues, providing absolute backpressure control.

3.5.1 FETCH Fixed Header

```
Bit:     7    6    5    4    3    2    1    0
Byte 1:  [ 1    0    0    0  ] [ 0    0    0    0 ]
          ^--- Type = 16 ---^   ^--- Reserved ---^
Byte 2+: [         Remaining Length              ]
```

3.5.2 FETCH Variable Header

Field	Size	Description
Packet Identifier	2 bytes	Standard acknowledgement identifier
Property Length	Variable	Length of property block
Batch Size (0x42)	4 bytes	Maximum messages to return (REQUIRED)
Wait Timeout (0x36)	2 bytes	Max ms to wait if queue is empty (long-poll; OPTIONAL)
Last Received Seq (0x30)	4 bytes	Client's high-watermark for gap detection (OPTIONAL)

3.5.3 FETCH Payload

The payload contains the **Queue Name** — a UTF-8 encoded string specifying the target `$queue/` topic (e.g., `$queue/production_line_1`).

```
+-------------------------------------------------------------+
|                   FIXED HEADER (2+ bytes)                   |
| Bits 7-4: Type (16) | Bits 3-0: Reserved (0000)          |
| Remaining Length (Variable Byte Integer)                    |
+-------------------------------------------------------------+
|                    VARIABLE HEADER                          |
| Packet Identifier       (2 bytes)                           |
| Property Length         (Variable)                          |
|   Batch Size    (0x42)  (4 bytes)                           |
|   Wait Timeout  (0x36)  (2 bytes, optional)                 |
|   Last Seq      (0x30)  (4 bytes, optional)                 |
+-------------------------------------------------------------+
|                       PAYLOAD                               |
| Topic Filter (UTF-8, e.g., "$queue/production_line_1")      |
+-------------------------------------------------------------+
```

3.5.4 FETCH Normative Behavior

- The Broker MUST respond by delivering at most `Batch Size` messages from the specified queue to the Client's session.
- If the queue contains fewer messages than `Batch Size` , the Broker MUST deliver only the available messages.
- If the queue is empty and `Wait Timeout` is specified, the Broker MAY hold the request open until a message arrives or the timeout expires (long-polling).
- If `Last Received Seq` is provided, the Broker MUST validate it against the queue's current state. If the Client's value is ahead of the Broker's known state, the Broker MUST respond with a DISCONNECT containing `Reason Code 0xA0 (Epoch Reset)` .

- The Broker MUST NOT send unsolicited messages from `$queue/` topics unless the Client has an active FETCH request.

3.5.5 FETCH Acknowledgement

The Broker responds to a FETCH by publishing messages with the standard PUBLISH flow (including PUBACK for QoS 1). The Packet Identifier from the FETCH is echoed in the response batch's PUBLISH packets to allow correlation.

---

4. Operational Behavior

4.1 The `$queue/` Namespace

Topics prefixed with `$queue/` represent **Named Queues** — durable, ordered message stores that exist independently of client sessions.

4.1.0 `$queue/` Behavior by Packet Type

The `$queue/` prefix changes packet semantics as follows:

Packet	Standard Topic Behavior	`$queue/` Topic Behavior
PUBLISH (inbound)	Delivered immediately to matching subscribers	Persisted to non-volatile storage, assigned Stream Sequence Number, acknowledged only after persistence confirmed
PUBLISH (outbound)	Pushed to subscriber as soon as available	Delivered only in response to a FETCH request (or active push subscription in compat mode); includes Stream Sequence Number and Epoch properties
SUBSCRIBE	Creates a live subscription; broker pushes matching messages	Registers the client as a consumer of the named queue; delivery mode depends on `v6-semantics` property (default: pull via FETCH)
UNSUBSCRIBE	Removes subscription; no further messages delivered	Deregisters the consumer; queue continues to exist and buffer messages independently
FETCH	N/A (not applicable to standard topics)	Requests a batch of messages from the named queue; broker responds with up to `Batch Size` messages

Packet	Standard Topic Behavior	`$queue/` Topic Behavior
CONNECT (with `v6-last-seq`)	N/A (property ignored for standard topics)	Broker uses the client's last-seq and epoch to resume delivery from the correct position

4.1.1 Persistence

- **Normative:** The Broker MUST store messages published to `$queue/` topics in non-volatile storage before acknowledging the PUBACK (QoS 1) or PUBCOMP (QoS 2).
- Messages in `$queue/` MUST NOT be discarded unless:
  - Acknowledged by all intended consumers (per SQMC consumer group logic), OR
  - The message's `Message Expiry Interval` (v5.0 Property 0x02) has elapsed, OR
  - The queue's storage capacity limit has been reached (see Section 4.1.3)

4.1.2 Ordering

The ordering invariant is: **per-queue, per-consumer (or per-consumer-group).**

- **Normative:** The Broker MUST deliver messages to each individual consumer in strict ascending order of their `Stream Sequence Number` within a given `$queue/{name}`.
- The Broker MUST NOT deliver a message with Sequence N+1 before the consumer has acknowledged Sequence N (for QoS 1 and 2 delivery).
- In **Competing Consumer** mode, ordering is guaranteed per-consumer within the group, but not globally across all consumers in the group (since messages are distributed round-robin). Global ordering across the group is achievable only with a single active consumer (Exclusive mode).
- In **Exclusive Consumer** mode, ordering is globally guaranteed because only one consumer is active at any time.

4.1.3 Storage Limits

- **User Property:** `v6-queue-max-size` in SUBSCRIBE — specifies the maximum number of messages to buffer (Four Byte Integer string)
- **Behavior on limit reached:** The Broker SHOULD apply a "drop-oldest" policy by default. An alternative "reject-new" policy returns PUBACK with Reason Code `0x8D (Quota Exceeded)`.
- **User Property:** `v6-queue-ttl` in SUBSCRIBE — specifies how long (seconds) the queue persists after the last consumer disconnects

4.1.4 Access Control

- **Normative:** Access to `$queue/` topics MUST be controlled via ACLs. The Broker SHOULD treat unauthorized publish or subscribe attempts to `$queue/` the same as unauthorized

access to `$SYS/` topics.
- The Broker MUST reject subscribe requests to `$queue/` topics from Clients that have not completed the v6.0 negotiation handshake (Section 3.1).

4.2 Pull-Based Flow Control

MQTT v6.0 replaces the “push-all” delivery model for queue topics with a **consumer-driven pull model**.

4.2.1 Native Mode (FETCH Packet)

See Section 3.5. The Broker MUST hold queue messages until a FETCH request arrives. This provides absolute backpressure: if the consumer’s application layer is slow, it simply does not issue FETCH requests.

4.2.2 Compatibility Mode (Virtual FETCH)

For deployments where clients or brokers do not support Protocol Level 6:

1. **Request:** The Client publishes a message (QoS 1) to `$SYS/queues/{queue_name}/fetch`
2. **Batch Size:** The PUBLISH MUST include `User Property: ("v6-batch", "{N}")` where N is the desired message count
3. **Response:** The Broker releases the next N messages from the persistent queue to the Client’s session
4. **Normative:** The Broker MUST NOT release more than N messages per request. If no Virtual FETCH is active, the Broker MUST NOT push queue messages to the Client.

4.2.3 Semantic Equivalence Requirement

Normative: The Virtual FETCH mechanism (Section 4.2.2) and the native FETCH packet (Section 4.2.1) MUST provide identical observable behavior to the consumer. Specifically:

- The set of messages returned for a given queue state and batch size MUST be the same regardless of which mechanism is used.
- Ordering guarantees (Section 4.1.2) MUST be preserved in both modes.
- Stream Sequence Number and Stream Epoch values MUST be present in delivered messages in both modes (as native properties in Track A; as User Properties `v6-seq` and `v6-epoch` in Track B).
- The broker MUST NOT push unsolicited `$queue/` messages in either mode.

A conformant implementation MUST pass the same test suite for both mechanisms. The only permitted differences are wire encoding (binary properties vs. string User Properties) and transport overhead.

4.2.4 Rate Limiting

- **Throughput Limit (0x41) in CONNACK:** Broker-enforced KB/s limit on client publish rate
- **Reason Code 0x9B (Quota Exceeded):** Returned when a Client exceeds the throughput limit
- The `Receive Maximum` property from v5.0 continues to apply for QoS 1/2 in-flight limits

4.3 Cluster Consistency

In a multi-node deployment, any node MAY accept a publish to a `$queue/` topic. The protocol does not mandate a specific cluster topology, consensus algorithm, or replication strategy. v6.0 defines the **client-facing invariants** that any conformant clustered implementation MUST satisfy, using a **Distributed Sequence Counter** model as the reference mechanism.

4.3.1 Distributed Sequence Counter

- **Normative:** Each `$queue/{name}` MUST maintain a single logical sequence counter whose increments are serialized across all nodes in the cluster. The storage and replication mechanism for this counter is implementation-defined.
- When any node receives a PUBLISH to a `$queue/` topic, it MUST obtain the next sequence number **atomically** (e.g., via compare-and-swap, distributed lock, or equivalent) before persisting the message.
- No routing constraint is imposed on clients or publishers — **any node may sequence any publish**. Monotonicity is guaranteed by the serialized counter, not by sticky routing or leader election.
- If the sequence counter is temporarily unavailable (e.g., due to a network partition or quorum loss), the receiving node MUST reject the PUBLISH with Reason Code `0x97 (Quota Exceeded)` and the client MUST retry after reconnection.
- The Broker MUST NOT assign a sequence number speculatively (i.e., without a confirmed serialized increment). Speculative assignment risks duplicate sequence numbers across nodes.

4.3.2 Epoch Management

4.3.2.1 Scope

The Stream Epoch is **per-queue**: each `$queue/{name}` maintains its own independent Epoch value. An Epoch increment on `$queue/A` has no effect on `$queue/B`.

4.3.2.2 Triggering Conditions (Exhaustive)

The Broker MUST increment the Stream Epoch for a given `$queue/{name}` if and only if one of the following conditions is true:

1. **Sequence counter state loss:** The serialized sequence counter for the queue was unavailable during a failure window, and the Broker cannot confirm that no sequence numbers were issued, duplicated, or lost during that window.

2. **Unrecoverable message loss:** One or more persisted messages in the queue were permanently lost (e.g., all replicas of the message left the cluster before the message was acknowledged by a consumer).
3. **Sequence wraparound:** The 32-bit sequence space has been exhausted and the counter wraps to 0 (see Annex C).

The Broker MUST NOT increment the Epoch for routine events such as node restarts with intact persistent storage, consumer disconnections, or temporary network latency.

4.3.2.3 Propagation

- The new Epoch MUST be propagated in the next CONNACK or PUBLISH to any Client subscribed to the affected queue, and in a DISCONNECT (with Reason Code `0xA0 Epoch Reset`) to Clients that had in-flight messages from the affected queue at the time of the event.
- Nodes rejoining after a partition MUST defer to the surviving cluster's Epoch value for each queue.

4.3.2.4 Mixed-Epoch In-Flight Messages

If a Client has in-flight (unacknowledged) messages from Epoch E and the Broker increments to Epoch E+1:

- The Broker MUST send a DISCONNECT with Reason Code `0xA0 (Epoch Reset)` to the Client.
- The Client MUST discard all unprocessed in-flight messages from Epoch E.
- On reconnect, the Client MUST include `v6-epoch: E` (its last known Epoch) in CONNECT. The Broker responds with the new Epoch in CONNACK per Section 4.3.3.

4.3.3 Client Failover and Resync

When a Client reconnects after a broker failure:

1. Client includes `v6-last-seq: {N}` and `v6-epoch: {E}` in CONNECT
2. Broker checks its local queue state for `$queue/{name}` :
    - If Epoch matches and Seq N is present: resume delivery from N+1
    - If Epoch matches but Seq N is absent (message was lost): begin from the earliest available message and flag the gap
    - If Epoch has incremented: respond with CONNACK containing the new Epoch value; Client MUST reset its idempotency state

4.3.4 Exactly-Once Processing (Application Layer)

Using the Stream Sequence Number as an idempotency key:

1. Consumer maintains a **High-Watermark** (last processed Seq) in an atomic store alongside its business logic
2. On message receipt: if `Seq <= High-Watermark`, acknowledge (PUBACK) but discard — do not process
3. If `Seq > High-Watermark`: execute business logic AND update High-Watermark atomically
4. On Epoch reset: clear the High-Watermark and perform full reconciliation

4.4 Single-Queue Multi-Consumer (SQMC)

4.4.1 Competing Consumer Mode

When a consumer subscribes with `v6-semantics: competing`:

- **Message Locking:** When a message is dispatched from the queue to a consumer at QoS 1 or 2, it is "locked" to that consumer.
- **Lock Duration:** The lock persists until the consumer sends PUBACK (QoS 1) or PUBCOMP (QoS 2).
- **Failover:** If the consumer disconnects before acknowledging, the Broker MUST immediately unlock the message and deliver it to the next available consumer in the group.
- **Round-Robin:** The Broker distributes messages across consumers in the group using round-robin or least-loaded assignment. The specific algorithm is implementation-defined.
- **Normative:** Each message MUST be delivered to exactly one consumer in a Competing Consumer group.
- **Normative:** When dispatching a message in Competing Consumer mode, the Broker MUST include all message properties — including payload encryption properties (`0x3A`, `0x3B`, `0x3C`) — unchanged in the PUBLISH delivered to the consumer.

4.4.2 Exclusive Consumer Mode

When a consumer subscribes with `v6-semantics: exclusive`:

- The first Client to subscribe becomes the **Primary Consumer**.
- All subsequent subscribers become **Hot Standbys** — they receive no messages while the Primary is connected.
- **Normative:** If the Primary Consumer disconnects, the Broker MUST immediately promote the first Standby to Primary and resume delivery from the last acknowledged sequence.
- **Ordering guarantee:** Because only one consumer is ever active, strict ordering is preserved across failover.
- **Normative:** When delivering a message to the Primary Consumer in Exclusive mode, the Broker MUST include all message properties — including payload encryption properties (`0x3A`, `0x3B`, `0x3C`) — unchanged in the PUBLISH packet.

---

5. Summary of Changes: v5.0 vs. v6.0

Feature	MQTT v5.0	MQTT v6.0
Protocol Level	5	**6** (or 5 with compat negotiation)
Sequencing	16-bit Packet ID (per-session, recycled)	**32-bit Stream Sequence (per-queue, persistent)**
Flow Control	Push-based (Receive Maximum bounds in-flight window)	**Push + Pull: Receive Maximum for pub/sub; FETCH for queue consumption**
Queue Type	Session-bound implicit queue	**Named durable queue (`$queue/` namespace)**
Consumer Patterns	Shared Subscriptions (loose)	**SQMC: Competing + Exclusive with message locking**
Cluster Consistency	Implementation-defined, opaque to clients	**Epoch-based failover; client-visible resync**
Ordering Guarantee	Hop-by-hop, best effort	**Global monotonic ordering within queue**
Backpressure	Receive Maximum limits in-flight count	**FETCH adds consumer-initiated pull; broker buffers until requested**
Rate Limiting	None	**Throughput Limit property in CONNACK**
New Packet Types	None (Types 1-15)	**FETCH (Type 16)**

6. Compatibility Layer

MQTT v6.0 is backward-compatible with MQTT v5.0 for all unchanged v5.0 use cases: existing packet types, properties, topics, and semantics continue to work as before. Compatibility is not wire-transparent for v6.0-only features such as FETCH, `$queue/` , Stream Sequence properties, or `last-seq` / `epoch` ; those require either Protocol Level 6 or the compatibility mode specified below.

Compatibility guarantee: A v5.0 or v3.1.1 client connecting to a v6.0 broker for standard publish-subscribe sees no difference in behavior. The v6.0 extensions activate only when a client explicitly opts in via the negotiation handshake.

6.1 Negotiation

Mode	Protocol Level	User Property
Native v6.0	`6`	`mqtt-ext: v6.0` (optional, implied by level 6)
Compat v6.0 (on v5.0 broker)	`5`	`mqtt-ext: v6.0` (required)
Legacy v5.0	`5`	(absent)

6.2 Feature Mapping (Native → Compat)

Native v6.0	v5.0 Compatibility Mapping
FETCH (Type 16)	`PUBLISH` to `$SYS/queues/{name}/fetch`
Batch Size (0x42)	`User Property: ("v6-batch", "{N}")`
Stream Sequence (0x30)	`User Property: ("v6-seq", "{N}")`
Stream Epoch (0x35)	`User Property: ("v6-epoch", "{N}")`
SQMC Mode Bits	`User Property: ("v6-semantics", "competing|exclusive")`
Wait Timeout (0x36)	`User Property: ("v6-wait-ms", "{N}")`

6.3 Broker Requirements for Compat Mode

A v5.0 broker with a v6.0 Extension Plugin MUST:

1. Intercept PUBLISH packets to `$SYS/queues/*/fetch` and trigger queue message release
2. Intercept SUBSCRIBE packets containing `v6-semantics` User Properties and apply SQMC logic
3. Inject `v6-seq` and `v6-epoch` User Properties into outgoing PUBLISH packets for `$queue/` topics
4. Maintain persistent storage for `$queue/` topics independent of session state

---

7. Security Considerations

All MQTT v5.0 security requirements (TLS, Enhanced Authentication, SASL/SCRAM) are retained and remain mandatory. The following subsections define normative security requirements introduced by v6.0.

7.1 Access Control for `$queue/` and `$SYS/queues/`

- **Normative:** Access to the `$queue/` namespace MUST be controlled by ACLs. Unauthorized publish or subscribe to `$queue/` topics MUST be rejected with Reason Code `0x87 (Not`

`Authorized)`.

- **Normative:** Access to `$SYS/queues/*/fetch` control topics MUST be restricted to authenticated clients that have completed the v6.0 handshake.
- **Normative:** The Broker MUST NOT allow a client to create, subscribe to, or publish to a `$queue/` topic without first completing the v6.0 negotiation handshake (Section 3.1). Clients that have not negotiated v6.0 MUST be rejected with Reason Code `0x87 (Not Authorized)`.

7.2 Epoch Integrity

- **Normative:** The Broker MUST only increment the Stream Epoch under the conditions listed in Section 4.3.2.2 (Triggering Conditions). Sending an Epoch increment outside these conditions is a protocol violation.
- **Normative:** A spurious Epoch Reset causes all affected clients to discard idempotency state and resynchronize simultaneously. Implementations MUST treat unauthorized or unwarranted Epoch increments as a denial-of-service vector and SHOULD implement rate limiting (no more than N Epoch increments per queue per hour, where N is implementation-defined but SHOULD default to a conservative value such as 5).
- **Normative:** Epoch changes MUST be logged with timestamps, affected queue names, and triggering conditions for forensic auditing.

7.3 Sequence Number Side-Channel

- **Informative:** Stream Sequence Numbers are monotonically increasing and visible to all consumers of a queue. An observer who can read sequence numbers can infer the publication rate of the queue. Deployments where publication rate is sensitive information SHOULD restrict queue subscriptions via ACLs and SHOULD use TLS to prevent network-level observation.

7.4 High-Watermark Persistence

- **Normative:** A Client's High-Watermark (last processed sequence number) determines the resumption point on reconnect. If a Client's High-Watermark is tampered with (set to a value higher than actually processed), messages between the true and false watermark are silently skipped. Client implementations MUST persist the High-Watermark atomically with the business-logic processing of each message (Section 4.3.4).
- **Informative:** The Broker does not validate that a Client's reported `v6-last-seq` is truthful. A malicious client could report a false High-Watermark to skip messages. Deployments requiring tamper-proof watermark tracking SHOULD implement server-side watermark validation.

7.5 Transport Security — TLS 1.3 Requirement

[NEW for v6.0]

- **Normative:** A v6.0 Broker operating in Native Mode (Protocol Level 6) MUST support TLS 1.3 as defined in RFC 8446. A v6.0 Client in Native Mode MUST NOT negotiate TLS versions prior to TLS 1.3 with a v6.0 Broker.
- **Normative:** A v6.0 Broker MUST reject connections from clients that offer only TLS 1.2 or earlier cipher suites when operating in Native Mode. Reason Code `0x87 (Not Authorized)` is returned.
- **Normative:** The following TLS 1.3 cipher suites MUST be supported: `TLS_AES_256_GCM_SHA384`, `TLS_CHACHA20_POLY1305_SHA256`. Brokers MAY support additional cipher suites.
- **Informative:** TLS 1.3 eliminates the handshake round-trip latency of TLS 1.2 and removes vulnerable cipher suites (RC4, 3DES, CBC-mode AES without HMAC). For industrial deployments where both latency and auditability matter, TLS 1.3 is the appropriate floor.
- **Compatibility Note:** In Compatibility Mode (Protocol Level 5), TLS 1.3 is RECOMMENDED but NOT required. Existing v5.0 deployments may use TLS 1.2 during migration. Operators SHOULD establish a migration timeline when upgrading to Native Mode.

7.6 Optional Payload Encryption (Zero Trust)

[NEW for v6.0 — OPTIONAL FEATURE]

> ***Note:*** *This feature is entirely optional. It is never required for protocol conformance. Deployments that rely on TLS 1.3 transport encryption alone (Section 7.5) are fully conformant. Payload encryption is provided for zero trust architectures where the broker must be treated as an untrusted intermediary.*

7.6.1 Motivation

In zero trust architectures, end-to-end confidentiality requires that the broker — even a trusted HiveMQ cluster — cannot read message payloads. TLS 1.3 secures the transport hop (client→broker, broker→subscriber) but terminates at the broker. Payload encryption extends confidentiality across the broker, making the broker a sealed transport for ciphertext it cannot inspect.

This matters specifically for `$queue/` topics carrying sensitive industrial payloads (SECS/GEM process recipes, financial transactions, regulated health or grid data) where the broker operator and the data owner are organizationally distinct.

7.6.2 New Properties for Key Metadata

Three new optional properties carry encryption metadata on PUBLISH packets. These properties describe the encryption applied to the payload. They carry no key material — key distribution is an application-layer responsibility outside this specification.

Property ID	Name	Type	Description
`0x3A`	**Payload Key ID**	UTF-8 String	Opaque identifier for the symmetric key used to encrypt the payload. Consumers use this to look up the correct decryption key from their key management system.
`0x3B`	**Payload Algorithm**	Byte	Enum identifying the encryption algorithm. See Section 7.6.3.
`0x3C`	**Payload Key Version**	Two Byte Integer	Key rotation counter. Consumers can detect key rotation events without changing the Key ID.

- **Normative:** Encryption properties (`0x3A` , `0x3B` , `0x3C`) MUST only appear on PUBLISH packets to `$queue/` topics carrying encrypted payloads. A PUBLISH to a non- `$queue/` topic carrying these properties is malformed. Brokers MAY reject such packets with Reason Code `0x8A (Invalid Argument)` .
- **Normative:** A Broker that receives a PUBLISH with encryption properties MUST forward them to subscribers unchanged. The Broker MUST NOT attempt to decrypt or re-encrypt payloads.

7.6.3 Payload Algorithm Enum (Property `0x3B`)

Value	Algorithm	Notes
`0x01`	AES-256-GCM	Recommended default; AEAD; authenticated encryption
`0x02`	ChaCha20-Poly1305	Preferred for constrained clients without AES hardware acceleration
`0x03`	AES-128-GCM	Acceptable where bandwidth or compute is constrained
`0x7F`	Implementation-Defined	For private extensions; not interoperable across broker vendors

7.6.4 Nonce and Authentication Tag

When using an AEAD cipher (all standardized algorithms above), the encrypted payload format is:

```
[12-byte nonce][ciphertext][16-byte authentication tag]
```

The nonce MUST be unique per message. Implementations SHOULD derive the nonce by combining a random base with the Stream Sequence Number (`0x30`) to prevent nonce reuse across queue messages while remaining deterministic for deduplication.

7.6.5 Key Management — Application Layer Boundary

Key distribution, key rotation procedures, key revocation, and certificate management are **application-layer responsibilities**. This specification defines only the metadata properties that identify which key and algorithm were used. How keys are exchanged, stored, and rotated is intentionally outside this specification.

> *Refer to the MQTT Stream Application Profile (Option A in the Application Layer Lift documentation) for recommended key management patterns, including integration with SPIFFE/SPIRE for workload identity and HPKE for asymmetric key encapsulation.*

7.6.6 Broker Behavior for Encrypted Payloads

- **Normative:** A Broker MUST NOT reject a PUBLISH solely because it carries payload encryption properties (`0x3A` , `0x3B` , `0x3C`).
- **Normative:** A Broker MUST forward payload encryption properties to all matching subscribers unchanged.
- **Normative:** A Broker MUST NOT log, inspect, or cache the decrypted payload content. The broker is a sealed conduit for encrypted payloads.
- **Informative:** Because the broker cannot inspect encrypted payloads, content-based routing (topic-based filtering on payload content), stream processing plugins, and payload-aware QoS features are unavailable for encrypted `$queue/` messages. Topic design must carry all routing information in the topic string rather than in the payload.

7.6.7 Interaction with Stream Sequence Numbers

Stream Sequence Numbers (`0x30`) are assigned to the encrypted payload as a whole. The sequence number does not depend on plaintext content. Gap detection, high-watermark tracking, and epoch resync operate identically whether the payload is encrypted or not — all sequence semantics are in the MQTT properties layer, not the payload.

7.6.8 Backward Compatibility with MQTT v5.0 Clients

Normative: MQTT v5.0 clients MUST NOT be exposed to encrypted payloads on `$queue/` topics. The v6.0 handshake gate (Section 3.1) is the primary enforcement mechanism: a client that has not completed the `mqtt-ext: v6.0` negotiation MUST be rejected with Reason Code `0x87 (Not Authorized)` when attempting to SUBSCRIBE to any `$queue/` topic. Since v5.0 clients do not send the `mqtt-ext: v6.0` User Property in CONNECT, they cannot pass this gate and will never receive encrypted payloads.

Informative: In the event that a conformant MQTT v5.0 client inadvertently receives a PUBLISH packet carrying encryption properties (`0x3A` , `0x3B` , `0x3C`), the MQTT v5.0 specification mandates that the client silently ignore unknown property IDs. No protocol-level error or connection closure is triggered. Any failure is at the application layer (the client receives ciphertext it cannot interpret), not at the protocol layer. This provides a defense-in-depth guarantee: the handshake gate is the primary protection; the unknown-property rule is the fallback.

Annex A: SECS/GEM Mapping

For semiconductor manufacturing environments using SECS/GEM (SEMI Equipment Communications Standard):

SECS/GEM Concept	MQTT v6.0 Mapping
Transaction ID (TID)	Stream Sequence Number (0x30)
S1F13/S1F14 (Establish Comms)	CONNECT/CONNACK with v6.0 negotiation + Epoch check
S2F21 (Remote Start)	PUBLISH to `$queue/secsgem/{device_id}/S2F21` with `v6-semantics: exclusive`
S6Fx (Event Reports / FDC)	PUBLISH to `$queue/secsgem/{device_id}/events` + Virtual FETCH for backpressure
Multi-block messages	Segmented PUBLISH with sequence continuation; reassembled via Batch Size
Link Test (Heartbeat)	FETCH with `Wait Timeout` as a “proof of life” pull
Equipment Model Reset	Epoch Reset → Client performs S1F13 re-establishment

A.1 Critical Scenario: Process Start Command (S2F21)

In SECS/GEM, S2F21 (Remote Start) is mission-critical. Loss of this command can ruin a wafer.

Without v6.0: If broker Node A crashes after accepting S2F21 but before delivery, the command may be silently lost.

With v6.0: 1. Host publishes S2F21 with `Sequence: 5001, Epoch: 1` to `$queue/secsgem/tool_001/S2F21` 2. Broker Node A persists to non-volatile storage and assigns Seq 5001 3. Node A crashes; its replica is gone but the sequence counter and message data survive in other nodes' replicated storage 4. Tool reconnects with `v6-last-seq: 5000` in CONNECT 5. Node B checks persistent queue: Seq 5001 exists and was never acknowledged 6. Node B resumes delivery: tool receives S2F21 exactly once

---

Annex B: New Reason Codes

Code	Name	Description
`0xA0`	Epoch Reset	Cluster failover caused sequence discontinuity; client must reset idempotency state
`0x9B`	Quota Exceeded	Client exceeded Throughput Limit
`0x8D`	Queue Full	Queue storage limit reached; message rejected

Alternative Approaches to MQTT v6.0

This document explores the design alternatives considered during the development of MQTT v6.0, explains the trade-offs, and justifies the chosen approach.

---

1. The Core Design Choice: Breaking vs. Non-Breaking

The central tension in extending MQTT is between **expressiveness** and **compatibility**. Every proposed feature was evaluated on a spectrum:

```
[Pure v5.0 Extension] ←──────────────────────→ [New Protocol Version]
   (User Properties)      (New Properties)      (New Packet Types)
      No breaks               Safe                  Breaking
```

---

2. Alternative A: Pure MQTT v5.0 Extension (Property-Based Shim)

Description

All v6.0 features encoded entirely as `User Properties` within existing v5.0 packets. No new property IDs, no new packet types.

Implementation

- Sequence numbers as `User Property: ("v6-seq", "12345")`
- Epoch as `User Property: ("v6-epoch", "2")`
- FETCH as `PUBLISH` to `$SYS/queues/{name}/fetch` with `User Property: ("v6-batch", "100")`
- Consumer modes as `User Property: ("v6-semantics", "competing")`

Pros

- **Zero compatibility risk.** Works on any MQTT v5.0 broker, any client library, today.
- **No broker changes required.** Logic lives entirely in a HiveMQ Extension Plugin.
- **Fastest time to production.** Can be deployed as a library/plugin without protocol changes.

Cons

- **Not standardizable.** User properties are opaque strings — no binary parser optimization is possible. This is a permanent workaround, not a standard.
- **Parsing overhead.** Sequence numbers as strings ("4294967295") cost 10 bytes; as 4-byte integers they cost 4 bytes + 1 ID byte = 5 bytes.
- **No protocol-level enforcement.** A broker cannot distinguish a v6.0 sequence-aware publish from a regular publish at the binary level without inspecting all user properties.
- **No `FETCH` semantics at the broker level.** The broker cannot natively throttle push delivery; it requires application-level hacks.

The Fragmentation Risk

Shipping these features as vendor-specific User Property conventions (without standardization) creates the exact problem standardization prevents. If HiveMQ uses `v6-seq`, EMQX uses `x-seq-number`, and AWS IoT Core uses `aws-sequence`, every customer running multiple brokers needs a translation layer. This is how `$SYS/` became fragmented across brokers — every vendor implemented it differently, and 15 years later there is still no standard. Standardizing the wire representation early prevents this.

Verdict

Viable as a transitional shim. Adopted as the Compatibility Layer (Track B). Not suitable as the long-term specification because it prevents standardization and invites vendor fragmentation.

---

3. Alternative B: MQTT v5.0 with Reserved Property IDs (0x30–0x45)

Description

Use the unused/reserved property ID space (`0x30` – `0x45`) to define new typed properties. No new packet types. Protocol Level remains 5.

Implementation

- `0x30` : Stream Sequence Number (Four Byte Integer)
- `0x35` : Stream Epoch (Two Byte Integer)
- `0x41` : Throughput Limit (Four Byte Integer)
- `0x42` : Batch Size (Four Byte Integer)
- FETCH still implemented as Virtual FETCH (control topic publish)

Pros

- **Binary-efficient.** Fixed-width integers instead of string user properties.
- **Backward-compatible in practice.** MQTT v5.0 mandates that unknown Property IDs MUST be silently ignored by non-v6 clients and brokers. However, these IDs are not formally reserved for v6.0 until an OASIS submission is accepted — there is a pre-standardization window where a future v5.x revision could theoretically assign different semantics to the same IDs.
- **Minimal parser changes.** Any existing MQTT parser that follows the spec will skip unrecognized properties safely.
- **Works without Protocol Level change.** Mixed v5.0/v6.0 deployments are seamless.

Cons

- **No native FETCH.** Without a new packet type, pull semantics must still be simulated through control topics, adding round-trip overhead.
- **Subscription option bits still blocked.** MQTT v5.0 mandates bits 6-7 of the SUBSCRIBE options byte MUST be zero; exclusive/competing modes cannot be encoded natively in the packet — only in user properties.
- **Protocol-level ambiguity.** A “Protocol Level 5 client with v6 properties” is technically a v5.0 client; there is no way to formally signal v6.0 conformance without a new level.

Verdict

Adopted as the Compatibility Extension profile. This is what Track B uses. The reserved property IDs are the right mechanism; the missing piece is the FETCH packet.

4. Alternative C: Native v6.0 with New Packet Type (Chosen Approach)

Description

Full protocol version bump to Level 6. Introduces FETCH as Control Packet Type 16 — previously reserved in v5.0. New subscription option bits for SQMC modes.

Implementation

- Protocol Level `6` in CONNECT
- `FETCH` packet (Type 16) as a first-class pull mechanism
- Properties `0x30`, `0x35`, `0x41`, `0x42` as typed binary fields
- SUBSCRIBE options bits 6-7 used for Competing/Exclusive flags
- `$queue/` namespace with mandatory persistence semantics

Pros

- **Full expressiveness.** Every v6.0 feature has a dedicated, binary-efficient representation.
- **Broker-level enforcement.** A v6.0 broker can natively identify `$queue/` publishes, FETCH requests, and SQMC subscriptions at the parser level without inspecting application data.
- **Formal standardizability.** A clean Protocol Level 6 specification can be submitted to OASIS as a distinct standard.
- **Performance.** FETCH as a dedicated packet eliminates the round-trip overhead of the "publish to a control topic" pattern.

Cons

- **Breaking change for v5.0 brokers.** A strict v5.0 broker closes the connection immediately upon receiving a Type 16 packet.
- **Breaking change for subscription bits.** A v5.0 broker rejects SUBSCRIBE packets with non-zero reserved bits.
- **Requires ecosystem update.** Client libraries (paho, gmqtt, Eclipse Mosquitto) must be updated to support Level 6.
- **Deployment risk.** Mixed v5.0/v6.0 environments require the Compatibility Layer.

The Dual-Mechanism Trade-Off

Having two mechanisms for the same operation (native FETCH and Virtual FETCH) creates a testing and conformance burden. The specification requires **semantic equivalence** (see spec Section 4.2.3): both mechanisms MUST produce identical observable behavior for any given queue state. A conformant implementation MUST pass the same test suite for both. This trade-

off is accepted because the alternative — requiring Protocol Level 6 everywhere — would prevent incremental adoption.

Verdict

Chosen as the canonical v6.0 specification. The breaking changes are manageable with the Compatibility Layer (Alternative B) and negotiation handshake. The long-term correctness of a formal protocol is worth the transition cost.

---

5. Alternative D: Replace MQTT Entirely (Kafka / AMQP 1.0 / NATS JetStream)

Description

Instead of extending MQTT, adopt a different messaging protocol that already provides the required industrial features.

Comparison

Feature	MQTT v6.0	Apache Kafka	AMQP 1.0	NATS JetStream
Lightweight clients	Yes	No	Partial	Yes
Edge/IoT footprint	Excellent	Poor	Moderate	Good
Named durable queues	Yes (v6.0)	Yes (Topics)	Yes	Yes
Pull-based consumption	Yes (FETCH)	Yes (Poll API)	Yes	Yes
32-bit+ sequencing	Yes (v6.0)	Yes (Offset)	No native	Yes
Cluster consistency	Epoch-based	ISR/Leader	Partition leader	Raft consensus
Existing MQTT ecosystem	Full compatibility	None	Partial bridge	None

Feature	MQTT v6.0	Apache Kafka	AMQP 1.0	NATS JetStream
SECS/GEM mapping	Native	Requires adapter	Requires adapter	Requires adapter
Standard body	OASIS	Apache/Confluent	AMQP WG	CNCF

Analysis

Kafka provides everything v6.0 proposes, but: - Kafka clients are heavyweight (JVM, 50MB+). Not viable for edge devices or microcontrollers. - The existing investment in MQTT infrastructure (broker, client libraries, security tooling) is significant. - SECS/GEM integration with non-MQTT protocols requires a translation adapter, introducing latency and another failure point. - NATS JetStream is compelling for cloud-native workloads but lacks the industrial IoT ecosystem MQTT has built.

Verdict

Not adopted. The v6.0 extensions preserve the edge/IoT deployment model of MQTT while adding the industrial reliability features. Replacing MQTT would abandon its core advantage in constrained environments.

---

6. Alternative E: Extend MQTT Shared Subscriptions (v5.0 `$share/`)

Description

Enhance the existing MQTT v5.0 `$share/group/topic` mechanism to add message locking, exclusive consumer semantics, and failover logic — without changing the core protocol.

Pros

- Uses an existing, widely-supported mechanism
- No new packet types or property IDs required
- Implementable today as a broker plugin

Cons

- `$share/` groups are still session-dependent — the queue vanishes when the last client disconnects. Shared subscriptions distribute messages to active subscribers; they do not buffer messages when no subscriber is connected.

- No way to add explicit sequencing or gap detection within the `$share/` spec — there is no per-message identity that survives across sessions
- No standardized way to express "exclusive" (hot-standby) mode in the subscription syntax
- Consumer failover behavior (what happens when a consumer disconnects mid-processing) is implementation-defined, not standardized
- Extending `$share/` syntax creates a non-standard variation that other broker implementations won't support
- Message ordering within a shared subscription group is not guaranteed by the spec

Why `$share/` Is Not a Durable Queue

This is the most common misconception. `$share/` and `$queue/` solve fundamentally different problems:

Property	`$share/` (v5.0)	`$queue/` (v6.0)
Survives all consumers disconnecting	No	Yes
Survives broker restart (spec-mandated)	No	Yes
Message ordering guarantee	No	Yes
Gap detection	Impossible	Built-in
Named, inspectable entity	No	Yes

A shared subscription is a **delivery optimization for active subscribers**. A named queue is a **persistent message store**. See Addressing Criticisms for full analysis.

Verdict

Partially adopted. The conceptual model of consumer groups and competing delivery from `$share/` inspired the SQMC design. But v6.0 requires named queue persistence and sequencing that `$share/` fundamentally cannot provide without a new namespace.

---

7. Design Decision Matrix

Criterion	Pure User Props	Reserved Props (0x30+)	Native v6.0	Kafka/NATS	$share/ Ext
v5.0 Wire Compat	✅ Full	✅ Full	⚠️ Breaking	× None	✅ Full

Criterion	Pure User Props	Reserved Props (0x30+)	Native v6.0	Kafka/NATS	$share/ Ext
Binary Efficiency	× Poor	✅ Good	✅ Best	✅ Best	✅ Good
Native FETCH	× No	× No	✅ Yes	✅ Yes	× No
Standardizable	× No	⚠️ Partial	✅ Yes	✅ Yes	× No
Edge/IoT Viable	✅ Yes	✅ Yes	✅ Yes	× No	✅ Yes
SQMC Enforcement	× App-layer	⚠️ Partial	✅ Protocol	✅ Protocol	⚠️ Partial
Queue Persistence	× Plugin only	⚠️ Plugin only	✅ Spec-mandated	✅ Spec-mandated	× No
Time to Deploy	✅ Immediate	✅ Immediate	⚠️ Ecosystem update	× Full migration	✅ Immediate

Recommendation: Native v6.0 for new deployments with a full MQTT v6.0 ecosystem. Compatibility Layer (Reserved Props) for transitional deployments. Pure User Props shim for proof-of-concept and testing only.

MQTT v6.0 — Addressing Criticisms and Defending the Proposal

> ***Author:*** *Bill Sommers, AI Solutions Lead, HiveMQ, Inc.* ***Context:*** *Response to internal review feedback from Georg and Simon — March 2026*

A Note on Where This Comes From

Before I get into the point-by-point responses, I want to be clear about what drives this proposal: **two and a half years as a TAM in Customer Success at HiveMQ.**

I have spent that time on calls with customers who are building exactly the systems this proposal targets — semiconductor fabs running SECS/GEM over MQTT, energy providers pushing IEC 61850 telemetry through HiveMQ clusters, automotive OEMs streaming diagnostic data from factory floors. These are not hypothetical use cases. These are customers I have worked with directly, and the pain points in this proposal are patterns I have seen repeated across multiple engagements.

The common thread: **every one of these customers has built the same set of application-layer workarounds on top of MQTT v5.0.** Sequence numbers in payloads. Idempotency tables in Redis. Consumer group coordination in custom microservices. Checkpoint reconciliation jobs that run after every broker restart. And every one of them has asked some version of the same question: *"Why doesn't the broker just do this?"*

Eclipse Sparkplug B is the most visible proof that this need is real — it defines application-layer sequencing on top of MQTT (64-bit `seq` fields in protobuf payloads, birth/death certificates, state management) precisely because MQTT v5.0 does not provide these primitives natively. Sparkplug exists because MQTT has a gap. v6.0 proposes to close that gap at the protocol level.

I am not proposing that MQTT become Kafka. I am not proposing a general-purpose upgrade. I am proposing that we acknowledge what our customers are already building on top of MQTT and give them proper protocol-level tools instead of forcing them to reinvent the same workarounds in every deployment.

Why Standardize Now — Not "Ship an Extension First"

The most reasonable pushback I have heard is: *"Ship these features as a HiveMQ extension. Let the market validate them. Come back to OASIS in five years with production data."*

This sounds prudent, but it leads to **fragmentation, not validation.** If HiveMQ ships `$queue/`, FETCH, and SQMC as a proprietary extension, here is what happens next:

- **EMQX builds their own version** — similar semantics, different User Property names, different control topic paths, different consumer group behavior
- **Mosquitto community builds a plugin** — subset of features, incompatible wire format
- **AWS IoT Core adds "durable queues"** — completely proprietary, no interop with any of the above
- **Every customer running two brokers** (common in industrial — edge broker + cloud broker) now needs a translation layer between competing proprietary implementations

This is exactly what happened with `$SYS/` — every broker implemented it differently, there is no standard, and it remains a mess 15 years later. It is what happened with shared subscription behavior before v5.0 partially standardized it — and even now the spec leaves enough

implementation-defined gaps that HiveMQ had to build Declared Shared Subscriptions as a proprietary workaround.

The extension-first path does not lead to standardization — it leads to vendor lock-in dressed up as innovation. Once three vendors have incompatible implementations in production with paying customers depending on them, the OASIS committee has to reconcile three designs instead of evaluating one proposal. That is how standards processes stall for years.

The patterns are already validated. Sparkplug proves application-layer sequencing works. HiveMQ's Declared Shared Subscriptions prove durable queue semantics are needed. Every customer I have worked with who runs an MQTT-to-Kafka bridge proves pull-based consumption is needed. What is missing is not proof of concept — it is a **standard wire representation** that lets a Paho client library work with a HiveMQ broker, an EMQX broker, or any future broker without vendor-specific payload conventions.

Standardizing early — before fragmentation — is cheaper than standardizing late.

What This Proposal Is Not

This is not "MQTT trying to become Kafka." Every time I present this to an engineer, that is the first reaction. I understand why — the feature list (durable queues, pull-based fetch, sequence numbers, consumer groups) reads like a Kafka feature sheet. But the context is completely different.

Kafka is not viable at the edge. Its minimum client footprint requires a full JVM stack with multi-GB RAM. Our customers have thousands of devices — PLCs, RTUs, smart meters, semiconductor tools — running constrained firmware with MQTT baked in. These devices will be in service for 15–25 years. They cannot be reflashed with Kafka client libraries. AMQP 1.0 is similarly impractical for constrained environments.

The architecture our customers run today looks like this:

```
[Edge Device — MQTT v5.0] → [HiveMQ Broker] → [Kafka Bridge] → [Kafka] → [Consumer]
```

That bridge is a second system, a second failure domain, a second protocol stack, and a second team. Every customer I have worked with who runs this architecture has told me they would eliminate the bridge if they could. v6.0 eliminates it:

```
[Edge Device — MQTT v5.0] → [HiveMQ v6.0 Broker — native queuing] → [Consumer via FETCH]
```

The edge devices do not change. The edge protocol does not change. The consumer gets Kafka-like semantics (ordered, durable, pull-based) without a second messaging system. If Kafka could

run on a PLC, this proposal would not exist.

What Kafka does that v6.0 explicitly does not attempt: - Multi-day log retention with consumer offset replay - Stream processing or CEP (Kafka Streams, ksqlDB) - Server-to-server batch processing at data centre scale - Schema registry or schema evolution - Topic compaction

v6.0 adds exactly five things to MQTT. No more.

"MQTT v5.0 Already Does This" — Point-by-Point

Section 2.1: "That's a Shared Subscription"

Georg's criticism: Durable, session-independent queues are not needed because MQTT v5.0 Shared Subscriptions (`$share/group/topic`) already provide this.

Simon's criticism: "The protocol does not concern itself with broker nodes — that's the implementation."

I went back to the spec to make sure I was not overstating the gap. The gap is actually larger than I originally described.

The MQTT v5.0 OASIS Standard defines shared subscriptions in Section 4.8. Here is what the spec actually says — and critically, what it does not say:

1. Shared subscriptions do not buffer messages when no subscriber is connected.

The spec defines shared subscriptions as a mechanism to distribute messages across "subscribing Sessions." When no session is subscribed, the spec is silent — there is no normative requirement to buffer messages for the group. Messages published to a topic matched by a shared subscription with zero active sessions are not guaranteed to be held. This is not a durable queue — it is a fan-out optimization for active subscribers.

HiveMQ actually recognized this gap and built a proprietary feature called Declared Shared Subscriptions — pre-configured shared subscriptions that enqueue messages even when no subscriber is connected. **The fact that HiveMQ had to build a non-standard extension to get this behavior is itself evidence that `$share/` does not provide durable queue semantics.**

2. The spec explicitly prohibits re-routing unacknowledged messages to another consumer.

This is the one that matters most for industrial use cases. Section 4.8.2, normative requirement `[MQTT-4.8.2-5]` states:

> *"If the Client's Session terminates before the Client reconnects, the Server MUST NOT send the Application Message to any other subscribed Client."*

Read that again. If a consumer in a `$share/` group receives a QoS 1 message, disconnects before sending PUBACK, and its session expires — **the message is lost.** The spec explicitly prohibits the broker from re-delivering it to another consumer in the group. The message is discarded.

In a traditional competing-consumer queue (AMQP, JMS, Kafka consumer groups), an unacknowledged message is returned to the queue and re-dispatched to another consumer. In MQTT v5.0 shared subscriptions, it is **destroyed.** This is not a competing-consumer queue. It is a load balancer with a data loss failure mode.

I have seen this exact scenario in customer deployments. A consumer crashes mid-processing during a fault storm — exactly when message integrity matters most — and the in-flight messages are silently lost because the session expired before the consumer could reconnect.

3. Message ordering is explicitly NOT guaranteed for shared subscriptions.

The spec's ordering guarantee in Section 4.6, `[MQTT-4.6.0-6]` is scoped to *"Non-shared Subscriptions"* only. For shared subscriptions, ordering across consumers in the group is implementation-defined. HiveMQ's own documentation confirms this: *"you gain horizontal scalability and high availability at the cost of per-client message ordering."*

The full comparison:

Property	`$share/` (MQTT v5.0 Spec)	`$queue/` (v6.0 Proposal)
Messages buffered when no subscriber exists	Not guaranteed — spec is silent; implementation-defined	Yes — queue persists independently of sessions
Survives broker restart (spec-mandated)	No — persistence of shared sub state is implementation-defined	Yes — persistence to non-volatile storage required before PUBACK
Unacked message re-routed on consumer failure	**No — `[MQTT-4.8.2-5]` explicitly**	Yes — message lock released immediately;

Property	`$share/` (MQTT v5.0 Spec)	`$queue/` (v6.0 Proposal)
	prohibits this. Message is discarded if session terminates.	re-dispatched to next consumer
Message ordering guarantee	No — `[MQTT-4.6.0-6]` scopes ordering to non-shared subs only	Yes — strict ascending order by Stream Sequence Number
Gap detection	Impossible — no per-message identity	Built-in — Seq gaps are detectable
Named, inspectable entity	No — `$share/` is a subscription pattern	Yes — `$queue/` is a first-class entity with TTL, max-size, storage policy

The bottom line: `$share/` is a delivery optimization for active subscribers. `$queue/` is a named, durable, persistent message store. They solve different problems. Saying " `$share/` already does this" is like saying "TCP already provides reliable delivery, so we don't need a database." The reliability primitives operate at different levels and serve different purposes.

To Simon's point about the protocol not concerning itself with broker nodes — I agree. And the `$queue/` proposal does not dictate broker architecture. It defines a **client-facing contract**: messages published to `$queue/` topics MUST be persisted before acknowledgment and MUST be delivered in order with gap detection. How the broker achieves this internally is implementation-defined, just as how a broker persists session state today is implementation-defined. The difference is that `$queue/` makes the durability guarantee **normative** rather than optional.

Section 2.2: Packet Identifiers and Message Ordering

Simon's criticism: "Packet identifiers are session scoped and not intended for ordering. They are also eligible for reuse within a session. Why would we want to increase the size of session

packets by 2 bytes for arbitrary ID space? In what scenario would a client have >65535 packets inflight?"

Simon is correct on every factual point here, and I want to be clear that the proposal does not conflict with any of them.

We are **not** proposing to enlarge the Packet Identifier. The 16-bit Packet ID remains exactly as specified in Section 2.2.1. It continues to serve its original purpose: correlating QoS 1/2 acknowledgments within a single session.

The Stream Sequence Number (`0x30`) is a **separate, additional property** — a new field with a completely different purpose. The confusion is understandable because the original proposal document jumped into wire format details before clearly explaining why a new field is needed. That is my fault — Simon's "too much HOW, not enough WHY" feedback is well taken.

Here is the WHY:

	Packet Identifier (v5.0)	Stream Sequence Number (v6.0)
What it answers	"Has this specific QoS handshake completed?"	"Where is this message in the queue's total history?"
Scope	Per-session, per-connection	Per-queue, cluster-wide, permanent
Assigned by	Client or broker	Broker only
Recycled?	Yes — reused after PUBACK/PUBCOMP	Never — monotonically increasing
Survives reconnection?	No	Yes — persisted across sessions and broker restarts
Purpose	QoS ack correlation	Ordering, gap detection, exactly-once deduplication, resume-from-position

The question "when would a client have >65535 packets inflight?" is exactly the right question for Packet Identifiers — and the answer is "never in practice, so 16 bits is fine." But that is the wrong question for Stream Sequences. A queue processing 1,000 messages/second exhausts 65,535 values in 65 seconds. The sequence number represents **total message identity across the lifetime of a queue**, not in-flight count.

The overhead: 5 bytes per `$queue/` PUBLISH (1-byte property ID + 4-byte value). Zero bytes for non-queue topics. Standard pub/sub (`temperature/office`) is completely unaffected.

Both reviewers acknowledge that 2.2 (message ordering across sessions) is a valid gap. The Stream Sequence Number is the mechanism that fills it — without touching the Packet

Identifier.

Section 2.3: "Receive Maximum Already Provides Flow Control"

Georg's criticism: MQTT v5.0's `Receive Maximum` plus the frequency of client acknowledgments already provides flow control.

Georg is right that `Receive Maximum` provides flow control — it does. It caps the number of QoS 1/2 messages the broker can have in-flight simultaneously, and combined with ack pacing, it lets consumers influence delivery rate. The proposal does not claim v5.0 "lacks flow control."

The distinction is between push-based flow control and pull-based flow control. `Receive Maximum` operates within a push model: the broker initiates delivery as soon as an ack frees a slot. The consumer controls the rate but not the timing.

Here is the distinction that matters in practice:

	`Receive Maximum` (v5.0)	**FETCH (v6.0)**
Consumer controls *when* to receive	No — the broker pushes as soon as an ack frees a slot	Yes — consumer issues FETCH when ready. No FETCH = no messages.
Consumer can pause entirely	Only by disconnecting or withholding acks (both destructive)	Yes — stop issuing FETCH. Messages stay in the queue.
Prevents flood on reconnect	No — broker immediately pushes up to `Receive Maximum` from backlog	Yes — consumer fetches at its own pace

What I have seen in the field: The "slow consumer death spiral" is one of the most common support issues I dealt with as a TAM. A SCADA historian writing to a database at 10ms per message sets `Receive Maximum = 50`. The broker pushes 50 messages. The historian processes them at 100 msg/sec. But the queue has 500,000 msg/sec incoming. The broker pushes 50 more as each ack arrives. The historian falls further and further behind. Its memory fills. It crashes. It reconnects. The broker immediately pushes another 50 from the now-massive backlog. The historian crashes again.

The customer's workaround: disconnect the historian, let messages pile up in broker session queues, reconnect during a maintenance window and drain slowly. This is operational toil masquerading as flow control.

`Receive Maximum` plus ack pacing can slow down the push rate, but the consumer cannot tell the broker "I am in a maintenance window, hold everything until I come back and ask for it." FETCH

can.

FETCH is not a replacement for `Receive Maximum`. Both serve valid purposes. `Receive Maximum` is fine for live pub/sub where you want bounded push delivery. FETCH is for queue consumption where the consumer needs absolute control over when and how much it receives.

Section 2.4: "Cluster-Aware Sequence Numbers" — On the Epoch

Georg's criticism: "This is plainly wrong."

Simon's criticism: "The protocol does not concern itself with broker nodes — that's the implementation."

I want to address these together because I think the confusion is partly my fault. The original proposal described the Epoch in terms of cluster architecture (node failover, quorum loss, partition events). That framing made it sound like the protocol is prescribing implementation details. It is not.

The Epoch is not an implementation detail. It is a piece of protocol-level information that the client needs and can act on.

Here is what the Epoch actually is, stripped of all implementation language:

> ***Stream Epoch (`0x35`):*** *A value carried in CONNACK and PUBLISH packets that tells the client whether the broker's message state is continuous with what the client last saw.*
>
> - ***Same Epoch as client's stored value:*** *The broker guarantees that all sequence numbers between the client's last-seen Seq and the current Seq are accounted for. The client can trust its local high-watermark and resume processing.*
> - ***Different Epoch from client's stored value:*** *The broker* ***cannot*** *guarantee sequence continuity. Something happened — the broker does not tell the client what — that may have caused messages to be lost, reordered, or duplicated. The client MUST discard its local idempotency state and resynchronize.*

That is the entire client-facing contract. The protocol does not say "because a cluster node failed" or "because a partition occurred." It says: **"the broker's state may have changed in a way that breaks sequence continuity, and here is how the client knows."**

This is analogous to how MQTT v5.0 handles session state today. When a client reconnects with `cleanStart=false`, the broker tells the client (via the Session Present flag in CONNACK) whether the session exists. The protocol does not dictate *how* the broker persists sessions — it just

provides a client-visible signal. The Epoch extends this pattern: it tells the client whether the **queue's sequence state** is continuous, not just whether the session exists.

Why this matters — from customer experience:

The most common post-incident question I have heard from customers after a broker restart is: *"Did we lose any messages?"* Today, the honest answer is: "We don't know. The protocol doesn't tell you. You have to reconcile your application-level records against the broker's stored sessions and hope nothing fell through the cracks."

With the Epoch, the answer becomes: "If the Epoch is the same, no. If the Epoch changed, potentially yes — and here is exactly where to start your reconciliation." That is a massive improvement for incident response.

A single-node broker benefits from this too. If a standalone HiveMQ instance restarts and loses its in-memory queue state (but retains persisted data), it can increment the Epoch to signal to reconnecting clients that some messages may have been lost during the restart window. No cluster, no nodes, no quorum — just a simple "my state changed, here's your signal." The mechanism is topology-agnostic.

What operators build today without the Epoch:

Every customer I have worked with who cares about message continuity has built some version of this in application code: - Sequence numbers embedded in message payloads - Checkpoint tables in external databases (Redis, PostgreSQL) - Post-restart reconciliation jobs that compare producer records against consumer records - Manual "state reset" procedures that operators execute after broker incidents

The Epoch standardizes this signal at the protocol level. The client library handles it. The application developer does not have to build a reconciliation framework from scratch for every deployment.

Section 2.5: Shared Subscription Semantics — Exclusive Consumer / Hot Standby

Both reviewers acknowledge this is a valid gap. MQTT v5.0 does not provide exclusive consumer (hot-standby) semantics. The SQMC exclusive mode in v6.0 addresses a real need — particularly for safety-critical command streams (SECS/GEM S2F21, grid protection relay commands) where strict ordering with automatic failover is non-negotiable.

"This Should Be Handled at the Application Level"

Simon raises this directly: *"There appears to be a mux'ing of what is expected of the application and what is expected of the protocol."*

This is the right question to ask. Every feature in v6.0 **can** be implemented at the application layer. Sequence numbers, gap detection, exactly-once dedup, pull-based flow control, consumer group management — all of it. This is how it is done today.

But having spent 2.5 years watching customers do exactly this, I can tell you it does not work well in practice.

What I Have Seen in Customer Deployments

1. Every customer reinvents the same wheel, differently.

I have worked with semiconductor, energy, and automotive customers who all independently built application-layer sequencing on top of MQTT. None of their implementations are compatible. One uses JSON `{"seq": N}`, another uses protobuf with a `sequence_number` field, another uses a custom binary header. Every integration between these systems requires a translation layer that maps one sequence scheme to another. This is the N-squared problem in practice — 50 publishers and 20 consumers means 1,000 pairwise schema agreements, none standardized.

2. Application-layer reliability has a correctness problem.

The most common failure mode I have seen is not "the sequence number system doesn't work" — it is "the developer forgot to check the sequence number." Application-layer deduplication requires every consumer to correctly implement sequence validation. In a system with 20 consumers written by 5 different teams over 3 years, at least one of them will have a bug in its dedup logic. I have seen this cause duplicate processing of safety-critical commands — exactly the scenario this proposal is designed to prevent.

When the broker assigns the sequence and the client library handles dedup, the application developer cannot forget. The reliability primitive is in the infrastructure, not in the application code.

3. The broker cannot optimize what it cannot see.

When sequence numbers are in the payload, the broker is a dumb pipe. It cannot detect gaps, enforce ordering, perform dedup, resume delivery from a position, or track consumer group progress. These are all things the broker *should* be doing — and things our customers ask us why HiveMQ doesn't do.

4. Sparkplug proves the pattern.

Eclipse Sparkplug B already defines application-layer sequencing on MQTT. It proves the need is real and the industry has converged on a common pattern. v6.0 proposes to move that pattern

from the payload into the protocol — exactly as MQTT v5.0 moved message expiry, request/response correlation, and shared subscriptions from application-layer conventions into protocol properties.

5. The extension-first path leads to fragmentation, not validation.

If we tell customers "build it in your application" or "use a vendor extension," every vendor and every customer builds their own version. We end up with a dozen incompatible implementations of the same five patterns. Standardizing at the protocol level — before fragmentation — gives the ecosystem a single wire representation that every client library and every broker can implement. This is the same argument I made above in Why Standardize Now: the `$SYS/` fragmentation and HiveMQ's proprietary Declared Shared Subscriptions are proof that the extension-first path does not converge on interoperability.

The Historical Precedent

MQTT v3.1.1 had no concept of message expiry, request/response correlation, or shared subscriptions. All were implementable at the application layer. MQTT v5.0 moved them into the protocol because the industry had converged on common patterns that benefited from standardization. The same trajectory applies here.

"Do You Think MQTT Should Be a Message Queue?"

Simon asks: *"I think you hit the nail on the head when you said: 'It was not designed to be a message queue.' My question to you would be, do you think it should be? In which case, why not Kafka?"*

My Direct Answer

No, MQTT should not become a general-purpose message queue. It should remain a lightweight pub/sub transport for the vast majority of its use cases. Temperature sensors, dashboards, device commands, live monitoring — all of this is v5.0 territory and should stay there.

But MQTT is *already being used* as a message queue. Our customers are doing it right now, with application-layer hacks. The question is not "should MQTT be a message queue?" — it is "given that our customers are already using MQTT as a message queue, should the protocol provide proper primitives, or should we keep telling them to build their own?"

I think we should provide the primitives. Scoped narrowly (`$queue/` namespace only), opt-in (zero overhead for standard pub/sub), and backward-compatible (v5.0 clients and brokers are unaffected).

Why Not Kafka (The Answer That Never Changes)

The edge devices cannot run Kafka clients. A semiconductor fab has 500 tools running embedded MQTT clients on proprietary firmware that will be in service for 15–25 years. These tools cannot be reflashed. The Kafka JVM requirement is a non-starter.

The bridge architecture (MQTT → Kafka → Consumer) works but adds operational cost, a failure domain, and a team. Every customer I have worked with who runs this architecture has asked if we can eliminate the bridge. v6.0 is my answer.

"Too Much HOW, Not Enough WHAT and WHY"

Simon is right, and I appreciate the feedback. The original proposal led with wire formats and byte layouts. That was a mistake. The audience for this proposal is not protocol implementers (yet) — it is the people who need to understand *why* these changes matter before they evaluate *how* they work.

The WHAT and WHY are simple:

What Customers Need	Why They Need It	What v5.0 Offers	What v6.0 Adds
Durable queues that outlive sessions	Consumer maintenance should not cause message loss	`$share/` — but not durable, not ordered, messages lost on session termination per `[MQTT-4.8.2-5]`	`$queue/` — named, persistent, survives all consumer disconnections
Message ordering with gap detection	Consumers need to know if a message was lost	Nothing — Packet IDs are session-scoped and recycled	Stream Sequence Number — per-queue, cluster-wide, permanent
Pull-based flow control	Slow consumers crash under push load	`Receive Maximum` — caps window, but broker still pushes	FETCH — consumer pulls when ready; no FETCH = no delivery
Strict consumer groups with failover	Consumer crashes should not lose in-flight messages	`$share/` — but `[MQTT-4.8.2-5]` discards messages on session termination	SQMC — message lock released on disconnect, re-dispatched immediately

What Customers Need	Why They Need It	What v5.0 Offers	What v6.0 Adds
Broker state continuity signal	Clients need to know if messages were lost during broker restart	Session Present flag — but only signals session existence, not message continuity	Stream Epoch — signals whether sequence state is continuous

The genesis for this thinking is 2.5 years of watching our customers solve these problems in application code and asking myself whether we can do better at the protocol level. I believe we can.

Summary of Responses

Criticism	Response
"2.1 is just a shared subscription"	No. The v5.0 spec itself contradicts this: `[MQTT-4.8.2-5]` prohibits re-routing unacked messages to other consumers (messages are lost), `[MQTT-4.6.0-6]` scopes ordering to non-shared subs only, and the spec provides no normative requirement to buffer messages when no subscriber exists. HiveMQ built Declared Shared Subscriptions as a proprietary extension precisely because `$share/` lacks durable queue semantics.
"2.2 conflates concepts"	Fair. The proposal does NOT enlarge Packet Identifiers. Stream Sequence is a separate 5-byte property for a separate purpose (ordering, gap detection, dedup), added only to `$queue/` messages. Standard pub/sub is unaffected.
"2.3 is Receive Maximum"	`Receive Maximum` provides valid push-based flow control. FETCH provides a different semantic model — pull-based consumption — where the consumer controls timing, not just rate. Both serve valid purposes; they are complementary, not redundant.

Criticism	Response
“2.4 is plainly wrong”	The Epoch is not an implementation detail — it is a protocol-level signal carried in CONNACK that tells the client whether broker message state is continuous. It does not prescribe clustering, topology, or architecture. A single-node broker can use it. It is analogous to the Session Present flag but for queue sequence state.
“Protocol doesn’t concern itself with broker nodes”	Agreed — and the Epoch doesn’t either. It defines client-facing behavior: “if sequence continuity cannot be guaranteed, tell the client.” How the broker determines this internally is implementation-defined.
“This should be application-level”	Every customer I have worked with has built these primitives in application code. None of their implementations are compatible. The broker cannot optimize what it cannot see. Sparkplug already proves the need. v6.0 standardizes the pattern.
“Why not Kafka?”	Edge devices cannot run Kafka clients. The bridge adds cost and failure domains. v6.0 eliminates the bridge without changing edge devices.
“Ship it as an extension first”	The extension-first path leads to fragmentation, not validation. If HiveMQ ships it proprietary, EMQX and AWS build incompatible versions. Standardizing early — before fragmentation — is cheaper than standardizing late. `$SYS/` and Declared Shared Subscriptions are proof.
“Too much HOW, not enough WHY”	Fair. This document provides the WHY. The genesis is 2.5 years of TAM work watching customers solve these problems in application code.

References

- MQTT v5.0 OASIS Standard — Shared Subscriptions (`[MQTT-4.8.2-5]` , Section 4.8), Ordered Topics (`[MQTT-4.6.0-6]` , Section 4.6), Receive Maximum (Section 3.2.2.3.2), Packet Identifiers (Section 2.2.1)
- HiveMQ: Declared Shared Subscriptions — HiveMQ's proprietary extension to fill the durable queue gap in `$share/`
- HiveMQ: MQTT 5 Shared Subscriptions — Shared subscription behavior, ordering limitations
- HiveMQ: Shared Subscription Documentation — Implementation details and configuration
- HiveMQ: Client Load Balancing with Shared Subscriptions — Practical challenges with slow consumers
- HiveMQ: MQTT vs AMQP for IoT — Protocol complexity comparison
- Confluent Platform System Requirements — Kafka resource footprint
- Apache Kafka Consumer API — Pull-based poll() API
- Eclipse Sparkplug Specification — Application-layer sequencing on MQTT
- Sparkplug v2.2 Specification PDF — Sequence number and state management details
- SEMI SECS/GEM Standard — Semiconductor equipment communication
- SEMI E30 (GEM) Specification — Transaction semantics for semiconductor manufacturing
- IEC 61850 — Power grid communication standard
- IEC 61850 and MQTT Integration — Edge transport for grid data
- NATS JetStream Consumers — Pull-based consumer model reference
- HiveMQ: Industrial IoT Data Streaming with MQTT — MQTT at industrial scale

Critical Perspectives, Rebuttals & Technical Pitfalls

This document provides an honest assessment of the MQTT v6.0 proposal from multiple perspectives: MQTT standards committee members, IoT architects, software engineers, and SREs. For each criticism, a rebuttal and/or mitigation is provided.

Part 1: Standards Committee & Architecture Critique

Criticism 1: "MQTT's success is its simplicity. v6.0 turns it into a bloated Kafka clone."

From: MQTT protocol purists, standards committee members

The Argument: MQTT succeeded precisely because it fits in 128KB of RAM and works on a microcontroller. By adding 32-bit sequencing, Epochs, persistent queues, and FETCH semantics, we are violating the core design principle. The protocol will lose its competitive advantage against AMQP and become yet another "enterprise messaging platform."

Rebuttal:

1. **"Context Isolation" Defense:** These features are strictly opt-in. A temperature sensor publishing to `temperature/office` uses MQTT v6.0 identically to MQTT v3.1.1 — no sequence numbers, no epochs, no FETCH. The protocol grows upward, not outward. A compliant v6.0 client that doesn't use `$queue/` is binary-indistinguishable from a v5.0 client.

2. **"Standardization of Hacks" Argument:** Every major industrial MQTT deployment has already built these features — in the payload. Sequence IDs, idempotency keys, and consumer group logic are already in production payloads at industrial scale. v6.0 moves these de-facto requirements into the protocol header where brokers can optimize for them. The complexity exists; v6.0 just standardizes it.

3. **Scope Comparison:** The jump from v5.0 to v6.0 is smaller than the jump from v3.1.1 to v5.0. The v5.0 spec added Properties, Reason Codes, Shared Subscriptions, User Properties, and Will Delay — all of which "bloated" the protocol by the same argument. The ecosystem absorbed it; industrial users demanded it.

4. **Standardize Early or Fragment Forever.** The alternative — "ship it as a vendor extension first" — sounds prudent but leads to fragmentation. If HiveMQ ships `$queue/` as a proprietary extension, EMQX, Mosquitto, and AWS IoT Core will build incompatible versions. `$SYS/` is the cautionary tale: every broker implemented it differently, and it remains unstandardized 15 years later. HiveMQ's own Declared Shared Subscriptions are proof that extension-driven solutions do not converge on interoperability. Once three vendors have paying customers on incompatible implementations, the OASIS committee must reconcile three designs instead of evaluating one proposal — and standards processes stall for years. Standardizing before fragmentation is cheaper than standardizing after.

Criticism 2: "The `$queue/` namespace breaks topic hierarchy semantics."

From: MQTT architects, broker implementers

The Argument: MQTT topics are flat strings with hierarchical separators. The `$` prefix convention is fragile — `$SYS/` is implementation-specific, `$share/` is standard but limited. Creating a new `$queue/` namespace adds confusion, may conflict with existing broker topic trees, and is a "soft break" for v5.0 brokers that might route or reject it incorrectly.

Rebuttal:

1. **Existing precedent.** The `$SYS/` and `$share/` namespaces already demonstrate that `$`-prefixed topics are an accepted mechanism for protocol-level semantics. `$queue/` follows the same pattern.

2. **Explicit negotiation mitigates misrouting.** The v6.0 handshake (Section 3.1 of the spec) requires clients to confirm broker v6.0 support before using `$queue/`. If the broker does not confirm, the client MUST NOT use `$queue/` — preventing silent misrouting.

3. **ACL isolation.** The spec mandates `$queue/` access be controlled via ACLs, just like `$SYS/`. A v5.0 broker that doesn't understand `$queue/` will simply apply its default ACL policy (typically: reject `$`-prefixed topics from unprivileged clients), preventing message loss by failing loudly.

Remaining Risk: A misconfigured v5.0 broker with open ACLs COULD treat `$queue/foo` as a literal topic and accept publishes, creating a silent reliability hole. **Mitigation:** Document that `$queue/` topics MUST be ACL-blocked on v5.0 brokers.

Criticism 3: "The FETCH packet (Type 16) is a hard break that undermines the whole 'compatible' story."

From: Standards committee, library maintainers

The Argument: Type 16 was reserved in v5.0 specifically to indicate an error. A v5.0 broker receiving a Type 16 packet MUST close the connection. In a heterogeneous environment with mixed v5.0 and v6.0 brokers, a single misrouted FETCH packet causes cascading disconnections.

Rebuttal:

1. **The Compatibility Layer exists for this reason.** The Virtual FETCH mechanism (Section 4.2.2) provides identical semantics using standard v5.0 PUBLISH packets. No environment is forced to use native FETCH.

2. **Protocol-level negotiation prevents misrouting.** A client sets Protocol Level 6 in CONNECT. A v5.0 broker will reject the connection at the CONNECT stage with `Reason Code 0x84 (Unsupported Protocol Version)` — before any FETCH packet is ever sent. The client then falls back to Compatibility Mode. There is no scenario where a FETCH packet reaches a v5.0 broker in a correctly implemented client.

3. **Clean version separation.** Having a Level 6 that is truly distinct from Level 5 is better long-term than a "v5.1" that silently extends in ways that break some brokers but not others. The break is explicit and managed.

Part 2: Software Engineering & Performance Critique

Criticism 4: "The original proposal's Leader Node model contradicts HiveMQ's masterless architecture."

From: HiveMQ engineers, distributed systems architects

The Argument: The early draft proposed assigning a "Leader Node" per `$queue/` via consistent hashing, with all publishes routing through that node. HiveMQ is explicitly masterless — all nodes are peers, no consensus protocol (Raft/Paxos) is used, and no node "owns" a topic. The Leader Node model would require HiveMQ to fundamentally change its architecture.

Resolution (accepted — the proposal was updated):

This criticism is **correct**. The Leader Node model was removed from the specification in favour of a **Distributed Sequence Counter** approach that is native to HiveMQ's actual architecture:

1. **Distributed Sequence Counter.** Each `$queue/` maintains a single cluster-wide atomic counter in HiveMQ's replication data grid. Any node can increment it atomically (compare-and-swap) to claim the next sequence number. No routing changes are required; any node handles any publish. HiveMQ's existing data grid already provides the distributed primitives needed.

2. **No leader election, no Raft/Paxos.** The atomic counter replaces the entire leader/follower model. Split-brain sequencing (two nodes issuing the same Seq) is prevented by the CAS semantics of the counter, not by funneling all traffic through one node.

3. **Epoch triggers change.** Instead of "leader failover triggers Epoch++", the trigger is "replication quorum loss" — i.e., when a network partition resolves and the counter's replicated state cannot be verified. This maps directly to HiveMQ's partition detection events, which the Extension SDK already exposes.

4. **Hot-queue scaling still works.** If a single `$queue/` becomes a throughput bottleneck, the correct mitigation is application-level sharding (`$queue/line_1_A` , `$queue/line_1_B`) — exactly as you would shard a Kafka topic into multiple partitions. The counter CAS overhead per message is negligible compared to the NVMe write latency.

Criticism 5: "Replication lag will cause data loss in the replication gap window."

From: SREs, database engineers

The Argument: Node A received a publish and incremented the sequence counter to Seq 100, but crashes before the message is replicated to other nodes. Node B has no record of Seq 100. A client that was tracking Seq 100 will detect a gap, but the message is unrecoverable.

Rebuttal:

1. **Quorum write prevents the gap.** The spec requires the PUBACK to the publisher only after the message is confirmed on `N` replicas (configurable `Replication-Factor`). With `Replication-Factor: 2`, Node A crashing never loses the message because Node B already has it before the publisher received its PUBACK.

2. **The atomic counter and quorum write are decoupled.** The sequence number is claimed first (atomic CAS on the counter), then the message is written to quorum. If Node A crashes between those two steps, the claimed Seq number is "orphaned." This creates a one-time gap. The Epoch does NOT need to increment in this case — the surviving cluster knows the gap exists and can communicate it to consumers when they request that sequence range.

3. **Epoch for partition events only.** Epoch increments only when the entire counter quorum is lost (partition event) — not for individual node crashes. This reduces spurious full-resyncs that would otherwise be triggered by routine node restarts.

4. **The v5.0 baseline is worse.** A v5.0 broker silently loses the message with no signal to the client. v6.0 makes the gap explicit, auditable, and recoverable at the application layer.

Criticism 6: "Persistent queues on the broker will cause storage bloat and operational nightmares."

From: SREs, operations teams

The Argument: If every `$queue/` is persisted to non-volatile storage forever, brokers will eventually run out of disk space. "Leaked queues" (queues no one is reading) will accumulate indefinitely. This is an operational disaster in a long-running production system.

Rebuttal:

1. **Mandatory TTL and max-size properties.** The spec requires `v6-queue-ttl` (how long to hold the queue after last consumer disconnect) and `v6-queue-max-size` (message count limit). Both default to implementation-defined values but MUST be configurable. The spec also requires a "drop-oldest" or "reject-new" policy when limits are reached — no silent unbounded growth.

2. **"Tiered Storage" implementation model.** Production brokers (analogous to HiveMQ or Kafka) should implement hot/cold storage: recent messages in RAM (or NVMe), deep backlogs tiered to object storage (S3/GCS). The broker is a transient buffer with bounded local cost.

3. **Operational observability.** 32-bit sequences make "leaked queue" detection trivial: an SRE can query `MAX(Seq) - Last_Consumer_ACK_Seq` to calculate exact backlog depth per queue in real-time. This is far better than the current situation where session-bound queues silently accumulate with no tooling to measure them.

Part 3: SRE & Operational Critique

Criticism 7: "The FETCH mechanism adds latency compared to push delivery."

From: Performance engineers, real-time systems architects

The Argument: A FETCH round-trip adds 2× the one-way latency compared to a push delivery. For real-time applications, this is unacceptable.

Rebuttal:

1. **Scope clarification.** FETCH is exclusively for `$queue/` topics — durable industrial queues. Standard pub/sub topics (real-time sensor telemetry, command delivery to online devices) continue to use push delivery unchanged. The proposal does not change push semantics for non-queue topics.
2. **Cost-benefit for industrial queues.** In semiconductor FDC or financial messaging, "missing a message" is infinitely more expensive than 5ms of added latency. The FETCH mechanism prevents the host system from crashing under data spikes — which is the #1 cause of unplanned downtime in industrial MQTT deployments.
3. **Long-polling mitigates latency for near-real-time queues.** The `Wait Timeout` property in the FETCH packet enables long-polling: the broker holds the connection open and responds immediately when a new message arrives. Effective latency is near-zero for well-paced queues.

Criticism 8: "Custom property IDs in the reserved range could conflict with future OASIS standardization."

From: Standards committee members, protocol engineers

The Argument: Properties `0x30` – `0x45` are currently unassigned in MQTT v5.0, but they could be assigned by a future OASIS revision. If v6.0 squats on these IDs and a future v5.1/v6.x assigns different semantics to the same IDs, we have a compatibility disaster.

Rebuttal:

1. **This is exactly why we need formal standardization.** The solution to "these IDs might be assigned later" is to submit v6.0 to OASIS and have the IDs formally reserved. The alternative — using User Properties forever — permanently forecloses binary optimization.
2. **The proposal is already in "draft OASIS" format.** The spec is structured to be submitted as an OASIS draft. Early submission reserves the property IDs before any conflict can arise.

3. **Interim risk is manageable.** During the pre-standardization period, the Compatibility Layer (User Properties) can be used exclusively. The reserved property IDs are only activated in native v6.0 brokers that understand the spec. A v5.0 broker will silently ignore them, not misinterpret them.

Technical Pitfalls

Pitfall 1: Sequence Wraparound at 2^32

Description: A 32-bit sequence number wraps around after 4,294,967,295 messages. A high-throughput queue generating 100,000 messages/second exhausts the sequence space in ~43,000 seconds (~12 hours).

Mitigation: - For high-throughput queues, the broker MUST handle wraparound gracefully by treating `Seq 0` after `Seq 0xFFFFFFFF` as a continuation (not a gap). - For disambiguation, wraparound can be coupled with an Epoch increment — a full 32-bit cycle increments the Epoch. This gives an effective 48-bit sequence space. - A future revision may introduce 64-bit sequences for ultra-high-throughput use cases (see Annex B of the spec).

Pitfall 2: Idempotency Table Growth

Description: Consumers maintaining a “processed messages” idempotency table (for exactly-once semantics) may grow the table unboundedly if sequence numbers are not garbage-collected.

Mitigation: - The High-Watermark model (store only the latest processed Seq) is sufficient for in-order delivery queues — O(1) storage. - Only out-of-order delivery (not expected from ordered `$queue/` topics) requires a full set-based idempotency table. - Epoch Resets provide natural garbage-collection boundaries: on Epoch change, discard the entire idempotency table.

Pitfall 3: Competing Consumer Fairness

Description: Round-robin distribution may create consumer starvation if one consumer processes messages slowly. Slow consumers receive fewer messages but still hold “locks” on in-flight messages, blocking queue throughput.

Mitigation: - The spec allows brokers to implement “least-loaded” instead of strict round-robin: distribute the next message to the consumer with the fewest unacknowledged messages. - A per-consumer `Receive Maximum` limit (inherited from v5.0) caps the number of in-flight locks per consumer. - Consumers that are consistently slow should be flagged via broker metrics and operator intervention.

Pitfall 4: Epoch Reset as a DoS Vector

Description: A compromised or misconfigured broker could send spurious Epoch Reset signals, causing all connected clients to discard their idempotency state and re-synchronize simultaneously — a thundering herd.

Mitigation: - The spec requires Epoch increments only when the broker cannot guarantee sequence continuity (Section 4.3.2). Any other Epoch increment is a protocol violation. - Clients SHOULD implement Epoch change rate limiting: more than N Epoch changes per hour is anomalous and should trigger an alert and/or connection refusal. - Epoch changes MUST be logged with timestamps and node identifiers for forensic auditing.

Pitfall 5: Virtual FETCH Overhead (Compat Mode)

Description: The Virtual FETCH mechanism publishes a message to `$SYS/queues/{name}/fetch` and waits for the broker to respond with the batch. This is two round-trips per batch plus the overhead of the control topic publish.

Mitigation: - Use the `Wait Timeout` user property with long-polling to reduce idle round-trips. - For high-throughput applications, pipeline FETCH requests (send the next FETCH before the current batch is fully acknowledged). - This overhead is the explicit trade-off of Compatibility Mode. Native v6.0 FETCH eliminates it; operators should plan to migrate to Protocol Level 6 for production high-throughput deployments.

Pitfall 6: `$queue/` Namespace Collision with Existing Topics

Description: Deployments that already use topic paths containing the string `queue` (e.g., `myapp/queue/events`) may have topic tree management issues or operator confusion.

Mitigation: - The `$queue/` prefix begins with `$` , which is already a reserved character in MQTT (standard topics never start with `$`). No collision with user-defined topic spaces is possible. - Operator documentation should clearly distinguish `$queue/` (protocol-level named queues) from `myapp/queue/` (application-level topic paths).

Summary: Risk Matrix

Criticism / Pitfall	Severity	Probability	Mitigation Quality
Type 16 breaks v5.0 brokers	High	High (mixed env)	✅ Full (compat layer)
Leader node model (removed)	High	Resolved	✅ Full (distributed counter)

Criticism / Pitfall	Severity	Probability	Mitigation Quality
Replication lag / data loss	High	Low (with quorum)	✅ Good (epoch + quorum)
Storage bloat	Medium	Medium	✅ Good (TTL + limits)
Sequence wraparound (32-bit)	Low	Low (normal scale)	✅ Good (epoch cycle)
Epoch Reset DoS	Medium	Low	⚠️ Partial (rate limiting)
SUBSCRIBE bits break v5.0	High	High (native mode)	✅ Full (user props)
Property ID conflict (OASIS)	Medium	Low	✅ Good (formalize early)
Idempotency table growth	Low	Low (ordered queue)	✅ Good (high-watermark)
Virtual FETCH overhead	Low	High (compat mode)	⚠️ Partial (pipeline)

MQTT v6.0 Performance Analysis: High-Volume Use Cases

> ***Focus:*** *Large distributed energy providers (DEPs), grid SCADA, and industrial IoT at scale — characterised by millions of sensors, sub-second telemetry, burst fault events, and mandatory regulatory audit trails.*

---

1. Workload Profile

Dimension	Typical DEP Scale
Sensors / endpoints	1M–10M (smart meters, PMUs, RTUs, substations)
Steady-state throughput	500k–3M msg/sec cluster-wide
Burst (fault/storm event)	10×–50× steady state for 30–120 seconds
Message size	64–512 bytes (telemetry); 1–50KB (commands, recipes)

Dimension	Typical DEP Scale
Latency requirement	<10ms for control commands; 100–500ms acceptable for telemetry
Retention	90 days regulatory minimum; 7 years for audit
Consumer count	10–200 competing consumers per queue (analytics pipelines)

2. Performance Bottlenecks in the Current Proposal

2.1 CAS Counter Contention at Scale

Mechanism: The Distributed Sequence Counter requires an atomic CAS increment on the cluster data grid for every `$queue/` PUBLISH. Under contention, CAS degrades from O(1) to O(retries × RTT) as threads spin on the atomic variable.

Impact at DEP scale: A single `$queue/scada/events` receiving 500k msg/sec from 50k RTUs means ~10k CAS operations per second per broker node competing for the same counter. At 1ms inter-node RTT, contention-driven retry cycles will cap throughput well below the theoretical maximum.

Proposed fix — Partitioned Sequence Spaces:

Assign each broker node a pre-allocated sequence range of size `W` (the "window size"). Nodes increment locally without cross-node CAS until they exhaust their window, then claim the next window atomically.

```
Node 1 window: Seq 1 – 1,000,000        (local counter, no CAS)
Node 2 window: Seq 1,000,001 – 2,000,000 (local counter, no CAS)
Node 3 window: Seq 2,000,001 – 3,000,000 (local counter, no CAS)
```

- Window claiming (CAS) is amortized across `W` messages — at W=1M and 500k msg/sec, each node claims a new window every ~2 seconds.
- Gaps between windows are valid: Seq 1,000,000 → 1,000,001 is a valid "gap" — consumers must tolerate intra-window ordering while the global order is maintained at window boundaries.
- Window size `W` should be a configurable queue property (`v6-seq-window-size`).

New spec normative: The Stream Sequence Number MUST be monotonically increasing within a node's assigned window. Global monotonicity across windows is RECOMMENDED but not REQUIRED — consumers MUST treat any Seq > HWM as processable regardless of gap size when the queue uses partitioned sequence windows.

2.2 High-Watermark Inadequacy Under Burst

Current spec: Consumer maintains a single `High-Watermark` integer (last processed Seq). Works for ordered, serial consumption.

Problem at DEP scale: During a grid fault event, 50 competing consumers receive messages in parallel. In-flight Seq values span a wide range simultaneously:

```
Consumer 1: processing Seq 10,001 (slow — database write)
Consumer 2: processing Seq 10,050 (fast)
Consumer 3: processing Seq 10,100 (fast)
```

If Consumer 2 and 3 finish before Consumer 1, their HWM would advance past Consumer 1's in-flight message. If Consumer 1 then crashes, the group HWM is ambiguous — "did we process 10,001?"

Proposed fix — Sliding Window HWM:

Replace the single HWM integer with a **min/max window** tracked per consumer group at the broker:

```
Group HWM = {
  committed:  10,000   (all Seq ≤ this are acknowledged by all group members)
  in_flight:  [10,001, 10,002, ..., 10,150]  (dispatched, awaiting PUBACK)
  max_dispatched: 10,150
}
```

- `committed` advances only when all Seq below the next gap are acknowledged.
- The broker tracks in-flight locks as a bitset bounded by `Receive Maximum`.
- Consumer crash recovery: any in-flight Seq belonging to the crashed consumer is re-dispatched to the next available consumer.
- Client-side HWM for exactly-once deduplication remains a single integer (`committed`), not the full window.

New spec normative: Brokers implementing SQMC competing consumer mode MUST track a per-group `committed_hwm` (the highest Seq for which all lower Seqs are acknowledged) in addition to the per-consumer in-flight lock set.

2.3 Quorum Write Latency vs. Real-Time Telemetry

Current spec: All `$queue/` PUBLISH operations require a quorum write before PUBACK. At replication factor 2 and 1ms inter-node RTT, this adds 2–5ms per message.

Impact at DEP scale: PMU (Phasor Measurement Unit) data arrives at 60 samples/sec per device. A 2–5ms PUBACK delay is acceptable. But for 50k PMUs generating 3M msg/sec total,

quorum write serialisation creates head-of-line blocking in the broker's replication pipeline — effective throughput drops by 40–60% compared to async replication.

Proposed fix — Durability Tier Namespaces:

Split the `$queue/` namespace into three tiers with different durability contracts:

Namespace	Durability	Write Path	Use Case
`$queue/`	Synchronous quorum	PUBACK after N replicas	Commands, audit, SECS/GEM S2F21
`$stream/`	Async replication	PUBACK after local write; replication in background	High-frequency telemetry, PMU data
`$log/`	Append-only, eventual	PUBACK immediately; batched replication	FDC data, bulk sensor history

- The same FETCH, Seq, and Epoch semantics apply to all three tiers.
- `$stream/` and `$log/` topics carry a `Durability` property (`0x43`) in PUBLISH indicating the tier: `0x01` = sync quorum, `0x02` = async, `0x03` = eventual.
- Consumers subscribing to `$stream/` or `$log/` MUST tolerate occasional Seq gaps (due to async write failures) and SHOULD NOT use exactly-once semantics based on Seq alone.

2.4 FETCH Batch Size Tuning for Burst Recovery

Current spec: `Batch Size` is a fixed integer per FETCH request.

Problem: During burst recovery (after a 60-second grid storm event), a consumer with a 10M-message backlog issuing `batch=100` requests requires 100k round-trips to drain the queue. At 10ms RTT, that is 16 minutes of drain time.

Proposed fix — Adaptive Batch Size:

Add two optional FETCH properties:

- `v6-batch-max` : Upper bound on batch size the consumer will accept (consumer protection).
- `v6-batch-target-ms` : Target delivery window in milliseconds. The broker calculates the batch size needed to fill the window based on queue throughput, bounded by `v6-batch-max` .

For a consumer that wants "deliver as many as I can process in 500ms", the broker calculates:

```
batch = min(v6-batch-max, queue_throughput_per_ms × 500)
```

This eliminates the need for consumers to manually tune batch sizes as queue depth fluctuates.

2.5 Rate Limiting Granularity

Current spec: Throughput Limit (Property `0x41`) is per-connection in KB/s.

Problem for DEPs: A DEP SCADA system has thousands of client connections sharing the same broker. A per-connection limit cannot prevent a single rogue data pipeline from saturating broker NIC bandwidth while legitimate control traffic is dropped.

Proposed fixes:

1. **Per-queue rate limit:** Add a `v6-queue-rate-limit` User Property to SUBSCRIBE, specifying max msgs/sec the broker will release from that queue to this consumer group — independent of connection-level limits.

2. **Priority-based admission:** High-priority messages (`v6-priority: 255`) bypass the rate limiter. This ensures S2F21 (process start) commands are never delayed by FDC data storms.

3. **Credit-based flow control:** Replace the token-bucket CONNACK approach with an explicit credit system:

    - Broker sends `CREDIT {queue_name, msgs_per_sec}` in CONNACK or a new control topic.
    - Consumer sends `CREDIT_RETURN` when it can accept more.
    - This is analogous to AMQP 1.0's link credits and prevents both flooding and starvation.

2.6 Epoch Reset Thundering Herd

Current spec: On Epoch reset, all consumers connected to the affected queue simultaneously discard their HWMs and begin full resyncs.

Problem at DEP scale: With 200 competing consumers on `$queue/grid/events` , a single partition event triggers 200 simultaneous full resyncs — each reading the queue from Seq=0. This is a thundering herd that can exhaust broker I/O for minutes.

Proposed fix — Staggered Resync:

Add a `v6-resync-jitter-ms` property to DISCONNECT (Reason Code 0xA0). Each consumer receives a different jitter value (e.g., uniformly distributed over 0–30,000ms). Consumers delay their CONNECT retry and resync by this value. The broker staggers the resyncs, preventing I/O saturation.

For the broker to generate useful jitter values:

```
jitter_for_consumer_i = (consumer_group_size × resync_window_ms / N) × i
```

where `i` is the consumer's ordinal within the group (known from SQMC group membership).

---

3. Updated Property Table

ID	Name	Type	Packets	Purpose
`0x30`	Stream Sequence	4B Int	PUBLISH	Monotonic message ID
`0x35`	Stream Epoch	2B Int	PUBLISH, CONNACK	Cluster era
`0x41`	Throughput Limit	4B Int (KB/s)	CONNACK	Per-connection rate cap
`0x42`	Batch Size	4B Int	FETCH	Max messages per batch
`0x43`	**Durability Tier**	1B Enum	PUBLISH	`0x01` sync, `0x02` async, `0x03` eventual
`0x44`	**Batch Target Ms**	2B Int	FETCH	Target delivery window for adaptive batching
`0x45`	**Resync Jitter Ms**	2B Int	DISCONNECT (0xA0)	Staggered resync delay

---

4. Namespace Additions

Namespace	Durability	Sequence	Exactly-Once
`$queue/`	Sync quorum	Strict monotonic	Supported
`$stream/`	Async replicated	Best-effort monotonic	Not guaranteed
`$log/`	Eventual / append	Monotonic within node	Not supported
`$SYS/queues/*/fetch`	N/A (control)	N/A	N/A

---

5. Summary: Adaptations for DEP Scale

Problem	Scale Trigger	Fix
CAS counter contention	>100k msg/sec per queue	Partitioned sequence windows (`v6-seq-window-size`)
Single-integer HWM inadequate	>10 competing consumers	Sliding window HWM — committed + in-flight bitset
Quorum write latency	>1M msg/sec telemetry	Durability tier namespaces (`$stream/` , `$log/`)
Slow backlog drain	Post-burst recovery	Adaptive batch size (`v6-batch-target-ms`)
Per-connection rate limit too coarse	Thousands of connections	Per-queue rate limit + priority admission bypass
Epoch reset thundering herd	>50 consumers, partition event	Staggered resync with `v6-resync-jitter-ms`

Critical Analyses of MQTT v6.0

> *Three independent perspectives critiquing the MQTT v6.0 proposal.*
>
> ***See also:*** *[Addressing Criticisms — Point-by-Point Rebuttals](#) for responses to specific feedback from HiveMQ internal reviewers, including why `$share/` is not a durable queue, why `Receive`*

> *`Maximum` is not pull-based flow control, and the case for protocol-level vs. application-level reliability primitives.*

Part 1 — Security Analyst Perspective

Authored from the perspective of an industrial cybersecurity engineer with SCADA/ICS background, focused on OT/IT convergence risks.

1.1 Executive Verdict

MQTT v6.0 introduces meaningful architectural improvements for industrial messaging but expands the attack surface in ways the proposal does not adequately address. **The proposal is not ready for security-conscious deployment without significant additions to the specification.** The gaps are not fatal, but they are non-trivial.

1.2 Threat Model Gaps

1.2.1 Sequence Number as an Oracle

The Stream Sequence Number (`0x30`) is a monotonically increasing, cluster-wide counter assigned to every `$queue/` message. Any authenticated client that can publish and read the PUBACK Seq value gains **exact knowledge of global queue throughput**, message volume, and burst patterns — without consuming a single message.

A threat actor with read-only access to `$queue/scada/events` can infer: - Production rates across a facility - Fault event frequency and duration - Whether a process halt has occurred (Seq stops incrementing)

The proposal does not classify Seq values as sensitive metadata, does not propose per-consumer Seq obfuscation, and does not restrict Seq visibility to consumers. For regulated industries (NERC CIP, IEC 62443), publishing operational telemetry rates to all authenticated clients via a side-channel is a material information disclosure risk.

Recommendation: The spec should require that PUBACK Seq values are visible only to the publishing client, not leaked to other clients via CONNACK negotiation or shared subscription acknowledgements. Brokers should support `Seq-Blind` consumer mode where Seq is replaced with a client-local opaque token.

1.2.2 Epoch Reset as a Denial-of-Service Vector

Section 4.3.2 specifies that an Epoch increment forces all consumers to clear their High-Watermark and perform a full resync. The Epoch value is carried in CONNACK and PUBLISH packets — it is visible to any client that can connect to the broker.

A threat actor who can induce a cluster partition — or who can forge a CONNACK packet on a weakly-secured network segment (common in OT brownfield) — can trigger mass resync across all consumers, effectively causing a **denial of service on the consumer tier** for the duration of the resync window.

The proposal's mitigation (`v6-resync-jitter-ms`) addresses the thundering herd from legitimate partitions but does not address adversarial Epoch injection.

Recommendation: Epoch values in CONNACK MUST be authenticated via a broker-signed MAC or HMAC-SHA256, allowing clients to detect forged Epoch increments. This requires TLS mutual authentication (mTLS) to be normative for `$queue/` operations, not advisory.

1.2.3 `$SYS/queues/*/fetch` as an Unauthenticated Control Plane

The Virtual FETCH mechanism tunnels FETCH requests through `PUBLISH $SYS/queues/{name}/fetch` . In the compat-mode deployment, the broker extension reads this topic to release message batches.

The proposal does not specify authorization rules for `$SYS/queues/` writes. In most MQTT deployments, `$SYS/` is read-only for clients by convention. However, the proposal requires clients to *publish* to this namespace, which reverses the convention and may bypass ACL rules that blanket-deny `$SYS/` writes to non-admin clients.

A malicious client that can publish to `$SYS/queues/targetqueue/fetch` with `last-seq=0` can force the broker to stream the entire queue history to any consumer group — a data exfiltration path through the broker's own FETCH mechanism.

Recommendation: `$SYS/queues/*/fetch` MUST be protected by per-client, per-queue ACL identical to the `$queue/` subscription ACL. The spec must state this explicitly rather than deferring to broker implementation.

1.2.4 HWM Manipulation via Message Replay

The exactly-once deduplication relies entirely on the client's in-memory High-Watermark. The spec states that `Seq ≤ HWM → PUBACK but discard` . HWM is not persisted to durable storage in the base spec — it is reconstructed on reconnect.

If a client reconnects after a crash and the broker is still holding in-flight Seq values below the new HWM (derived from the last acknowledged FETCH), the client may re-process messages it already handled, violating exactly-once semantics. The gap between "last Seq the client persisted to its own DB" and "last Seq the broker tracks as committed" is an integrity risk in safety-critical command pipelines.

For `$queue/safety/interlocks` carrying process start/stop commands, duplicate processing of a "process start" command is a physical safety issue, not merely a data integrity issue.

Recommendation: Exactly-once guarantees for QoS=1 MUST require broker-side, per-consumer-group committed-HWM persistence (described in performance-analysis.md section 2.2) as normative, not optional. The base spec should not permit a compliant broker to hold HWM only in memory.

1.3 Missing Security Primitives

Gap	Risk Level	Mitigation Required
No mandatory mTLS for `$queue/`	High	Normative mTLS requirement in §4
No Seq visibility access control	Medium	Per-consumer Seq masking option
No Epoch authentication	High	Broker-signed Epoch in CONNACK
`$SYS/queues/` write ACL undefined	High	Explicit ACL requirement in compat spec
HWM not persisted by default	Medium–High	Normative broker-side HWM persistence
No message integrity field	Medium	Optional HMAC User Property recommendation
No audit trail for SQMC group membership changes	Medium	Mandatory SQMC group change event to `$SYS/`

1.4 Positive Security Properties

To be balanced: the proposal does introduce features that *improve* security posture relative to v5.0:

- **Named durable queues** make message provenance auditable (queue name + Seq = immutable event ID for 90-day regulatory retention).
- **Competing consumer mode** eliminates shared subscription race conditions that can cause message loss in QoS=1 — a correctness improvement that benefits audit trails.
- **Epoch tracking** provides a tamper-evident marker: if the Epoch advances unexpectedly, consumers can detect cluster disruption that might indicate an infrastructure attack.

The framework is sound. The spec needs hardening before it is appropriate for IEC 62443 Security Level 2+ environments.

Part 2 — OT SRE Perspective

Authored from the perspective of a senior site reliability engineer responsible for operational technology infrastructure at a large industrial facility — substations, PLCs, DCS, SCADA, and the middleware connecting them.

2.1 Executive Verdict

MQTT v6.0 solves the right problems but creates new operational burdens that the proposal underestimates. **The migration cost for existing OT deployments is significantly higher than the proposal acknowledges.** For brownfield facilities running MQTT v3.1.1 at the edge, v6.0 is a multi-year programme, not a phased upgrade.

2.2 Operational Concerns

2.2.1 The “Two Tracks” Problem Creates Indefinite Parallel Operations

The proposal defines Track A (native v6.0) and Track B (compat extension), recommending “Native v6.0 with an optional Compatibility Layer for hybrid deployments.” In practice, industrial facilities do not retire protocol versions on a planned schedule — brownfield devices stay in service for 10–25 years.

What this means in the field: An OT team will run Track B (compat) indefinitely, not as a transition, but as a permanent parallel stack. The compat layer then becomes load-bearing infrastructure that must be maintained across HiveMQ major versions.

Track B is not a “transition path” — it is a *permanent architecture tier* for any facility that cannot replace embedded MQTT clients on PLCs, RTUs, or smart meters. The proposal should acknowledge this and commit to long-term compat layer support with explicit SLA guarantees.

2.2.2 Epoch Reset Operational Impact is Unquantified

Section 4.3.2 specifies that an Epoch reset requires all consumers to perform a full resync from Seq=0. For a `$queue/` with 90 days of regulatory data at 500k msg/sec:

```
90 days × 86,400 sec/day × 500,000 msg/sec = ~3.9 × 10^12 messages
```

A full resync of 3.9 trillion messages is not a recovery operation — it is a service restoration incident measured in **days to weeks**, not minutes. The proposal’s staggered resync jitter (`v6-resync-jitter-ms`) prevents the thundering herd but does not address the total volume of data that consumers must re-read.

The proposal needs: 1. A normative upper bound on how far back a “full resync” actually reads (e.g., “from the last `committed_hwm` , not from Seq=0”). 2. A mechanism for consumers to declare their own recovery horizon (e.g., “I only need messages from the last 24 hours for real-time dashboards”). 3. Documented broker-side resync budgeting — how many concurrent resyncs can a broker support without degrading live traffic?

2.2.3 `$stream/` and `$log/` Durability Tiers Require New Operational Runbooks

The performance analysis introduces `$stream/` (async) and `$log/` (eventual) namespaces. From an OT operations perspective, these three namespaces have fundamentally different failure modes:

- `$queue/` : Message is either committed or rejected. No ambiguity.
- `$stream/` : Message appears committed (PUBACK issued) but may be lost on node failure before replication. Ops team must assume messages are “probably there.”
- `$log/` : PUBACK on local write, eventual consistency, gaps are possible. Ops team must build gap-tolerant consumers for every `$log/` subscriber.

OT operators who are accustomed to a single namespace (`$queue/`) with deterministic guarantees now have three namespaces with three different reliability contracts. **Misconfiguring a safety-critical sensor stream from `$queue/` to `$stream/` is a silent reliability regression** — the system continues to operate normally until a node fails, at which point data loss occurs with no alarm.

Recommendation: The spec should mandate a mandatory `Durability-Confirmed` property in CONNACK for any client subscribing to `$stream/` or `$log/` , explicitly informing the client of the durability tier. Brokers should refuse `$stream/` subscriptions from clients that have not acknowledged the reduced durability guarantee.

2.2.4 Adaptive Batch Size Tuning Is Not Observable

The `v6-batch-target-ms` property asks the broker to calculate batch size based on "queue throughput." There is no mechanism in the proposal for a consumer to inspect what batch size the broker actually chose, nor to observe the broker's throughput estimate.

In OT environments, when message processing falls behind (a common condition during fault storms), the ops team needs to answer: "Is the broker delivering batches correctly, or is our consumer too slow?" Without visibility into the broker's batch-size calculation, this is a black box diagnostic problem.

Recommendation: Add a `v6-batch-actual` User Property to FETCH response PUBLISH packets, reporting the actual batch size used and the broker's throughput estimate at time of dispatch.

2.2.5 SQMC Group Membership Has No Health Signal

The SQMC competing consumer model tracks group membership at the broker. If a consumer silently stalls (network partition at the consumer, not the broker), the broker may continue dispatching messages to the stalled consumer's in-flight lock set indefinitely — blocking the `committed_hwm` from advancing for the entire group.

The proposal does not specify: - A per-consumer in-flight timeout (how long can a lock be held before the broker considers the consumer failed and re-dispatches?) - A SQMC health status endpoint or `$SYS/` topic for group membership monitoring - Alerting or operator notification when `committed_hwm` stalls

In a production SCADA environment, a stalled HWM is a high-severity incident. It must be observable.

2.3 Deployment Readiness Assessment

Scenario	v6.0 Readiness	Concern
Greenfield broker + new OT devices	Ready	Ideal target environment
Greenfield broker + legacy MQTT v3 devices	Partial	Track B required indefinitely
Existing HiveMQ + v5 clients, upgrade broker	Partial	Extension plugin needed; Epoch semantics change
Multi-vendor broker environment	Not ready	SQMC and Epoch semantics are HiveMQ-specific

Scenario	v6.0 Readiness	Concern
Air-gapped OT network, infrequent updates	Not ready	HWM persistence and Epoch reset recovery not defined for offline operation

2.4 What OT SREs Actually Need

The proposal solves the engineering problems well. What is missing is the **operational layer**:

1. **Health and observability APIs** — SQMC group health, HWM progress, resync status, broker-side batch stats, all exposed via `$SYS/` topics or a new `$MGMT/` namespace.
2. **Operational runbooks** for Epoch reset recovery — including expected duration at various queue depths.
3. **Durability tier migration tooling** — safe procedure for moving a topic from `$queue/` to `$stream/` without data loss.
4. **Long-term compat layer commitment** — explicit version support matrix for Track B.

Part 3 — Arlen Nipper Perspective

Authored from the perspective of Arlen Nipper, co-inventor of MQTT, arguing against the adoption of MQTT v6.0.

Note: The views expressed here are a constructed argument representing this perspective for design evaluation purposes. They do not represent statements made by Arlen Nipper.

3.1 Opening Statement

I have spent thirty years watching MQTT succeed precisely because it does one thing exceptionally well: it moves small messages reliably over unreliable networks with minimal overhead. The protocol fits in a 128KB microcontroller. It runs over satellite links at 9600 baud. It works in a refinery where the network engineer left the company in 2011 and no one knows what hardware is in the ceiling.

MQTT v6.0 proposes to solve problems that are real, but the solutions it proposes are not MQTT solutions. They are message queue solutions borrowed from AMQP and Kafka and dressed in MQTT clothing. The result is a protocol that will be too complex for the constrained devices MQTT was designed to serve, while being less capable than the systems it is trying to imitate.

3.2 The Core Argument: Protocol Creep Destroys Interoperability

MQTT's greatest strength is that every compliant broker and client is interoperable. A Mosquitto broker, a HiveMQ cluster, a tiny RTOS client on an ESP32, and an enterprise application on AWS IoT Core all speak the same language. You can replace any component without changing any other.

MQTT v6.0 breaks this in two ways.

First, it introduces a new packet type (FETCH, Type 16) that is not backward-compatible. Any broker that does not support FETCH cannot serve v6.0 consumers. Any v6.0 consumer that requires FETCH cannot fall back to a v5.0 broker. The Track B compat layer is explicitly a workaround for this fragmentation, not a solution.

Second, it introduces broker-side semantics — SQMC group membership, distributed sequence counters, sliding window HWMs — that are not part of the wire protocol but are required for correct operation. Two brokers that both claim MQTT v6.0 compliance may behave entirely differently when a consumer crashes mid-batch, because the spec leaves the in-flight timeout undefined. This is not interoperability. This is vendor lock-in with a standardised handshake.

3.3 The Right Tool Exists

Every feature in MQTT v6.0 already exists in a mature, production-proven system. The proposal is asking MQTT to become a subset of these systems:

v6.0 Feature	Existing Solution
Named durable queues	RabbitMQ AMQP queues, since 2007
Pull-based FETCH with batch control	Apache Kafka consumer poll API, since 2011
Competing consumer with exactly-once	AMQP 1.0 link credits + settlement modes
Sequence numbering + gap detection	Kafka topic partition offsets
Cluster-wide atomic sequence counter	Zookeeper / etcd distributed counter
Epoch-based failover signalling	Kafka group coordinator epoch

The industrial customers who need these features should be using Kafka, RabbitMQ, or AMQP 1.0 at the tier where these features matter — the data historian, the analytics pipeline, the command-and-control system. These are server-to-server or server-to-cloud links where running a Kafka client is entirely feasible.

At the edge — the RTU, the PLC, the smart meter — MQTT v5.0 is entirely sufficient. These devices do not need FETCH semantics. They need to publish telemetry and receive commands.

MQTT v5.0 does this well.

3.4 The MQTT Gateway Pattern Is the Correct Architecture

The proposal acknowledges that many edge devices will remain on MQTT v3.1.1 or v5.0 indefinitely. It responds by creating a compat layer. I would respond differently: **use a gateway**.

```
[Edge Device MQTT v5.0] → [Edge Broker (HiveMQ/Mosquitto)]
                                        ↓
                            [Protocol Bridge / Gateway]
                                        ↓
                      [Kafka / RabbitMQ / AMQP 1.0 — full semantics]
                                        ↓
                            [Analytics / SCADA / Cloud]
```

This architecture: - Keeps edge devices on the protocol they already run - Provides full queue semantics at the tier that needs them - Does not require edge firmware updates - Does not introduce new MQTT broker implementations - Uses battle-tested systems for the queue layer

The proposal essentially reinvents this architecture within a single protocol. That is not simplification — it is complexity concealment.

3.5 The Constrained Device Problem

Section 3.1 of the proposal states that “all three tiers (`$queue/` , `$stream/` , `$log/`) use the same FETCH, Seq, and Epoch semantics.” A compliant v6.0 client must:

1. Track a High-Watermark (or sliding window HWM) persistently across power cycles
2. Detect Epoch changes and discard HWM on Epoch mismatch
3. Issue FETCH packets (or Virtual FETCH via `$SYS/`) with correct `last-seq` values
4. Handle gap alerts (Seq N missing, resume from N+1)
5. Implement exactly-once deduplication via HWM comparison

This is not a protocol that fits in 128KB. This is not a protocol that a firmware engineer at a meter manufacturer can implement correctly in six months without dedicated protocol expertise. MQTT’s original success came from the fact that a competent embedded developer could read the spec in a day and write a correct client in a week.

The reference Python shim in this proposal is 300 lines of non-trivial async code. That is the minimum viable client. The Java extension is longer. The constrained device story is “use the compat layer and pretend it’s MQTT” — which is precisely what the gateway pattern achieves without protocol fragmentation.

3.6 A Constructive Alternative

I am not arguing that HiveMQ should not build these features. I am arguing that these features should not be called MQTT v6.0, and should not require an OASIS standards process that will take years, produce inevitable compromises, and lock in design decisions that may not survive first contact with production deployments.

What I would recommend instead:

1. **HiveMQ Extension v2 for Advanced Queuing** — implement `$queue/`, SQMC, sequence numbers, and FETCH as a first-class HiveMQ extension, not a protocol revision. Ship it. Let the market validate the semantics. Allow competitors to implement compatible extensions.

2. **An OASIS MQTT Best Current Practice (BCP) document** — define the User Property schema (`v6-seq`, `v6-epoch`, `v6-semantics`) as a standardised extension pattern. This enables interoperability without a wire protocol break.

3. **Five years of production data** — if the HiveMQ extension proves out at DEP scale, with demonstrated adoption across multiple vendors, then bring it to OASIS as a v6.0 proposal with real-world validation behind it.

MQTT became the dominant IoT protocol because it was simple, stable, and widely implemented. Those properties are worth more than any individual feature. The right way to honour that legacy is not to add features — it is to build the features where they belong, at the system layer, and let MQTT remain what it has always been: a lightweight, reliable, interoperable publish-subscribe transport.

> ***Author's response:*** *This is the most reasonable objection to the proposal, and it deserves a direct answer. The extension-first path sounds prudent, but it leads to* ***fragmentation, not validation.*** *If HiveMQ ships `$queue/` as a proprietary extension, EMQX, Mosquitto, and AWS IoT Core will each build incompatible versions — similar semantics, different wire formats, different User Property names, different consumer group behavior. This is exactly what happened with `$SYS/`: every broker implemented it differently, and it remains unstandardized 15 years later. HiveMQ's own Declared Shared Subscriptions — built because `$share/` lacks durable queue semantics — are proof that extension-driven solutions do not converge on interoperability without a standard to converge on.*
>
> *The patterns themselves are already validated. Sparkplug proves application-layer sequencing works. Declared Shared Subscriptions prove durable queue semantics are needed. Every MQTT-to-Kafka bridge in production proves pull-based consumption is needed. What is missing is not proof of concept — it is a standard wire representation. Standardizing early, before vendors fragment, is cheaper than reconciling three incompatible implementations at OASIS five years from now. See Addressing Criticisms — Why Standardize Now for the full argument.*

Summary Comparison

Dimension	Security Analyst	OT SRE	Arlen Nipper
Overall verdict	Needs hardening	Migration cost underestimated	Wrong layer for these features
Top concern	Seq as info oracle, Epoch injection	Epoch reset at scale, compat permanence	Protocol fragmentation, constrained device burden
Fatal flaw?	No — addressable	No — operational gaps	Yes — misplaced complexity
Would adopt?	Yes, with security addendum	Yes, greenfield only	No — gateway pattern preferred
Primary ask	Normative mTLS + Seq ACL + Epoch MAC	Observability APIs + resync runbooks	Ship as HiveMQ extension first

SQMC vs. HiveMQ Declared Shared Subscriptions

> ***Context:*** *A common question when reviewing the MQTT v6.0 SQMC proposal is whether HiveMQ's existing Declared Shared Subscriptions feature already provides the same capabilities. The short answer: partially, but with four significant gaps — and its existence as a proprietary extension is itself evidence that the standard has a gap.*

What Declared Shared Subscriptions Does

HiveMQ Declared Shared Subscriptions are pre-configured `$share/` groups defined in broker configuration (rather than dynamically via SUBSCRIBE). Their key addition over standard

`$share/` is that messages are **buffered even when no subscriber is connected** — the queue persists across subscriber disconnects.

This is a real and valuable improvement over the standard `$share/` behavior, where the spec is silent on what to do with messages when no subscriber is active.

The Gaps

Declared Shared Subscriptions solve the offline-buffering problem. They do not solve the remaining gaps that SQMC addresses:

Capability	`$share/` (MQTT v5.0 Spec)	HiveMQ Declared Shared Subscriptions	SQMC (`$queue/` + v6.0)
Buffer messages when no subscriber connected	Not guaranteed — spec is silent	✅ Yes — HiveMQ's primary addition	✅ Yes — `$queue/` persists independently
Re-dispatch unacked message on consumer disconnect	× No — `[MQTT-4.8.2-5]` prohibits; message discarded if session terminates	× No — `[MQTT-4.8.2-5]` still applies at spec level	✅ Yes — lock released immediately; re-dispatched to next available consumer
Exclusive consumer / hot-standby mode	× No	× No	✅ Yes — primary consumer + automatic standby promotion on disconnect
Message ordering guarantee	× No — `[MQTT-4.6.0-6]` scopes ordering to non-shared subscriptions only	× No	✅ Yes — strict ascending Stream Sequence Number per queue
Gap detection	× Impossible — no per-message identity	× Impossible	✅ Built-in — sequence gaps are detectable and auditable
Survives broker restart (spec-mandated)	× No — persistence is implementation-defined	✅ Yes (HiveMQ-specific behavior)	✅ Yes — persistence to non-volatile storage required before PUBACK
Interoperable across broker vendors	✅ Yes — standard	× No — HiveMQ-proprietary	✅ Yes — standardized wire

Capability	`$share/` (MQTT v5.0 Spec)	HiveMQ Declared Shared Subscriptions	SQMC (`$queue/` + v6.0)
			format
Named, inspectable entity	× No — subscription pattern only	× No — broker config entry only	✅ Yes — `$queue/` is a first-class entity with TTL, max-size, storage policy

The Critical Gap: `[MQTT-4.8.2-5]`

The most consequential difference is message re-dispatch on consumer failure. [MQTT v5.0 Section 4.8.2, normative requirement `[MQTT-4.8.2-5]`](#) states:

> *"If the Client's Session terminates before the Client reconnects, the Server MUST NOT send the Application Message to any other subscribed Client."*

This means: if a consumer in any `$share/` group (including a Declared Shared Subscription) receives a QoS 1 message, disconnects before sending PUBACK, and its session expires — the message is **discarded**. The spec explicitly prohibits the broker from re-delivering it to another consumer.

In traditional competing-consumer queues (AMQP, JMS, Kafka consumer groups), an unacknowledged message is returned to the queue and re-dispatched. In MQTT v5.0 shared subscriptions, it is destroyed.

SQMC breaks from this by treating `$queue/` topics as session-independent persistent stores. When a competing consumer disconnects, the message lock is released by the broker and the message is immediately re-dispatched — regardless of session state. This is the behavior industrial systems require and cannot get from any `$share/` -based mechanism, proprietary or otherwise.

The Exclusive Consumer Gap

Declared Shared Subscriptions provide no equivalent to SQMC Exclusive Consumer mode. There is no mechanism in `$share/` — declared or dynamic — to designate one consumer as primary and hold others as hot standbys that take over instantly on primary disconnect while preserving strict ordering.

This pattern is non-negotiable for safety-critical command streams: SECS/GEM S2F21 (Remote Start), grid protection relay commands, and financial settlement messages where exactly one consumer must process each message in strict order, with zero-delay failover and no message loss or duplication on consumer failure.

Why Declared Shared Subscriptions Is the Argument, Not the Counterargument

The existence of Declared Shared Subscriptions as a HiveMQ-proprietary feature is itself evidence that the MQTT standard has a gap. HiveMQ built it because `$share/` alone did not satisfy customer requirements. The same customers who use Declared Shared Subscriptions are the ones asking for re-dispatch-on-failure and exclusive consumer semantics — features that Declared Shared Subscriptions does not provide.

The extension-first path (build it proprietary, standardize later) is also how `$SYS/` became permanently fragmented: EMQX, Mosquitto, and HiveMQ each implemented it differently, and 15 years later there is still no standard. If SQMC semantics are standardized now — before EMQX, AWS IoT Core, and others build incompatible variants — the ecosystem gets a single interoperable wire representation. If they are not, every customer running two broker vendors will need a translation layer between competing proprietary implementations.

Summary

Declared Shared Subscriptions is a useful proprietary improvement to `$share/` that addresses the offline-buffering gap. It does not address:

1. **Message re-dispatch on consumer failure** — `[MQTT-4.8.2-5]` prohibits this at the spec level for all `$share/`-based mechanisms
2. **Exclusive consumer / hot-standby semantics** — not available in any `$share/` variant
3. **Message ordering guarantees** — explicitly out of scope for shared subscriptions per the v5.0 spec
4. **Interoperability** — a HiveMQ-only feature; no other broker supports it

SQMC addresses all four. Its standardization closes the gap that Declared Shared Subscriptions can only partially fill — and closes it in a way that works across every v6.0-compliant broker.

The Application Layer Lift — Overview

> ***Context:*** *This document addresses the standards committee criticism that the MQTT 6.0 specification conflates broker obligations with application-layer responsibilities. It proposes three structural approaches to drawing a clear boundary between what the* ***protocol*** *provides and what the* ***application layer*** *must implement using those primitives.*

The Core Insight

The MQTT 6.0 proposal solves two distinct problems that have historically been bundled together:

1. **Protocol Primitives** — Things the *broker* must implement: durable queue storage, sequence number assignment, epoch signaling, pull-based message delivery, and strict consumer group enforcement. These require changes to the wire format and broker behavior. They cannot be "lifted" to the application layer — if the broker doesn't do them, no amount of client-side code can compensate.

2. **Application Patterns** — Things the *client library* must implement using those primitives: gap detection, epoch resync procedures, high-watermark tracking, idempotent processing workflows, consumer state machines, and reconnect/resume logic. These sit above the protocol. They are expressible in any language, testable independently of the broker, and can evolve faster than an OASIS standard.

The current MQTT 6.0 specification interleaves both. This is the source of the committee's "mux'ing" objection (Simon, March 2026). The three options below each propose a different way to draw the boundary explicitly.

The Split: Protocol vs. Application Layer

Regardless of which structural option is chosen, the feature split is the same:

Protocol Layer — Broker Obligations (MQTT 6.0 Spec)

Feature	What the Broker Does	Property / Mechanism
Durable Queue Storage	Persist messages before PUBACK; retain independent of sessions	`$queue/` namespace
Stream Sequence Assignment	Assign monotonically increasing 32-bit integer per	Property `0x30`

Feature	What the Broker Does	Property / Mechanism
	queue message	
Epoch Signaling	Set Epoch in CONNACK when sequence continuity cannot be guaranteed	Property `0x35`
Pull-Based Delivery	Hold messages until FETCH received; deliver requested batch	FETCH / Virtual FETCH
SQMC Enforcement	Enforce competing/exclusive consumer modes; re-dispatch on disconnect	`v6-semantics` User Property
Throughput Limiting	Enforce KB/s limits set in CONNACK	Property `0x41`
v6.0 Handshake	Echo `mqtt-ext: v6.0` in CONNACK to confirm capability	User Property

Application Layer — Client Library Obligations ("The Lift")

Responsibility	What the Client Library Does
Gap Detection	Track `expected_seq` per queue; detect when `received_seq ≠ expected_seq`
High-Watermark Tracking	Persist last successfully processed Seq per queue (O(1) state)
Epoch Change Handler	On Epoch mismatch in CONNACK: discard idempotency state, re-establish watermark
Consumer State Machine	Manage states: CONNECTING → HANDSHAKING → FETCHING → PROCESSING → ACKNOWLEDGING → RESYNCING
Exactly-Once Workflow	Check seq against watermark → process → advance watermark → PUBACK
Reconnect / Resume	Send `last-seq` and `current-epoch` as User Properties in CONNECT
FETCH Pacing	Manage batch sizing, pipeline depth, and back-pressure signaling

Responsibility	What the Client Library Does
Idempotency Table Lifecycle	When to use watermark vs. set-based dedup; garbage collection on Epoch reset
Sequence Wraparound	Treat Seq 0 after Seq 0xFFFFFFFF as continuation; coordinate with Epoch at rollover
SQMC Application Patterns	Map competing/exclusive modes to use-case requirements; handle failover scenarios

---

Why This Answers the Committee

Simon's criticism — *"There appears to be a mux'ing of what is expected of the application and what is expected of the protocol"* — is valid and can be resolved structurally. The three options below each offer a different level of separation:

Option	Separation Level	Documents	OASIS Submission
A	Hard split — two independent documents	MQTT 6.0 spec + MQTT Stream Application Profile	Protocol spec only
B	Soft split — one document, explicit boundary	MQTT 6.0 spec with Normative Annex A	Full spec as single submission
C	Maximum split — two independent specifications	MQTT 6.0 transport extension + MQTT Stream 1.0 app protocol	Two separate tracks

The key concession in all three options: **Simon is correct**. Gap detection, epoch resync, consumer state machines, and exactly-once processing workflows are application-layer responsibilities. The MQTT 6.0 specification does not need to define them — it only needs to provide the primitives that make them possible to implement correctly and interoperably.

Three Options

- **Option A: Slim Protocol + MQTT Stream Application Profile** Reduces the MQTT 6.0 spec to broker-only primitives. Publishes a separate companion "MQTT Stream Application Profile" defining client library behavior. Analogous to MQTT + Sparkplug B, but officially blessed by the MQTT 6.0 ecosystem. Recommended starting point.

- **Option B: Protocol Spec + Normative Annex A** Keeps a single MQTT 6.0 document but restructures it: Sections 1–7 contain only broker obligations; Normative Annex A defines client library requirements. Same total scope, explicit structural boundary. Easiest single OASIS submission.

- **Option C: Two Independent Specifications** Maximum separation. MQTT 6.0 is a minimal wire-format extension (broker-only). A fully separate "MQTT Stream 1.0" specification defines the application protocol that uses MQTT 6.0 as transport. Analogous to CoAP + LwM2M or HTTP + REST. Strongest answer to protocol purists; requires two standardization tracks.

Recommendation

For the current committee review stage, **Option A** offers the best balance: it visibly slims the protocol spec (addressing the "bloat" criticism), explicitly concedes the application-layer boundary (addressing Simon's mux'ing criticism), and provides a practical path where the MQTT Stream Application Profile can be adopted by Eclipse Foundation while the protocol primitives proceed through OASIS.

If the committee's primary concern is procedural scope (what OASIS is asked to ratify), **Option C** is the strongest answer. If a single-document submission is operationally required, **Option B** is the pragmatic choice.

All three options share the same feature split — only the document structure and governance path differ.

Option A: MQTT 6.0 Protocol Primitives + MQTT Stream Application Profile

> ***Author:*** *Bill Sommers, AI Solutions Lead, HiveMQ, Inc.* ***Status:*** *Discussion draft for standards committee review — March 2026* ***Addresses:*** *Simon's "mux'ing of application and protocol expectations" criticism*

---

1. Overview

The central concern raised by Simon in the committee review is that the current MQTT 6.0 specification conflates two distinct layers of obligation: what the **broker must do** and what the

client library must do. His exact phrasing — *"there appears to be a mux'ing of what is expected of the application and what is expected of the protocol"* — identifies a real structural problem in the draft, and this document addresses it directly.

Option A resolves the mux'ing by splitting the proposal into two independent documents with a clean interface between them:

1. **MQTT 6.0 Protocol Specification** — defines six broker-level primitives. Normative obligations using MUST/MUST NOT. Submitted to OASIS. Protocol parsers, broker implementers, and wire-format tooling implement this layer.
2. **MQTT Stream Application Profile** — defines client library behavior built on top of those primitives. Normative obligations using REQUIRES/RECOMMENDS. Submitted to Eclipse Foundation (alongside Sparkplug B). Client library authors and application framework developers implement this layer.

The architectural relationship is identical to MQTT itself versus Eclipse Sparkplug B. MQTT defines the transport primitives (PUBLISH, SUBSCRIBE, QoS levels). Sparkplug B defines application behavior on top of those primitives (64-bit sequence fields in protobuf payloads, birth/death certificate state management, group IDs). Two governing bodies, two documents, one coherent system.

This document does not propose removing any feature from the v6.0 ecosystem. It proposes assigning each feature to the layer where it belongs.

---

2. The Protocol Layer: What Stays in the MQTT 6.0 Specification

The MQTT 6.0 Protocol Specification defines the obligations of the **broker** — specifically, what the broker MUST do when it encounters v6.0 protocol elements. Client behavior noted below is the minimum required to invoke these broker obligations; everything else is the Application Profile's domain.

Feature	Protocol Specification Obligation	Normative Level
`$queue/` durable persistence	The broker MUST persist a message published to a `$queue/` topic to non-volatile storage before sending PUBACK. The queue MUST continue to exist and retain messages independently of any client session.	MUST
Stream Sequence Number (`0x30`)	The broker MUST assign a monotonically increasing 32-bit Sequence Number to each message accepted into a `$queue/`. Sequence Numbers MUST be cluster-wide and MUST NOT be reused within an Epoch.	MUST
Stream Epoch (`0x35`)	The broker MUST include the current Epoch value in CONNACK. The broker MUST increment the Epoch when it cannot guarantee sequence continuity for one or more queues (e.g., after a partition event or unrecoverable state loss). The broker MUST NOT increment the Epoch for routine operations that preserve continuity.	MUST
FETCH / Virtual FETCH	The broker MUST NOT push `$queue/` messages to a subscriber unless a FETCH request is pending or in force. On receiving a FETCH request, the broker MUST deliver up to the requested `batch-size` messages in ascending Sequence Number order.	MUST

Feature	Protocol Specification Obligation	Normative Level
SQMC subscription semantics	When a subscriber includes `v6-semantics: competing` in SUBSCRIBE User Properties, the broker MUST deliver each `$queue/` message to exactly one subscriber in the group. When `v6-semantics: exclusive` is specified, the broker MUST deliver to the primary subscriber only and MUST redirect to the standby subscriber if the primary disconnects.	MUST
Throughput Limit (`0x41`)	The broker MAY include a Throughput Limit property in CONNACK specifying the maximum KB/s the client is permitted to publish. If included, the broker MUST enforce this limit by refusing or delaying publishes that exceed it.	MAY / MUST

These six obligations define the complete broker contract. The MQTT 6.0 Protocol Specification is thinner than the current draft precisely because everything below this line belongs in the Application Profile.

---

3. The Application Profile Layer: The Lift

The **MQTT Stream Application Profile** defines client library behavior using the six protocol primitives above as building blocks. This is the layer that implements the reliability guarantees that industrial users actually need. The profile does not introduce new wire format; it specifies how client libraries MUST behave when processing protocol-layer signals.

This is Simon’s “mux’ing” resolved: these behaviors are explicitly and formally designated as client library obligations, not protocol obligations.

Behavior	Application Profile Specification	Normative Level
Gap detection algorithm	The client library REQUIRES a check `received_seq == expected_seq` for each message delivered from a `$queue/` . On gap detection (received > expected + 1), the library MUST invoke the registered gap handler before delivering the out-of-sequence message to the application.	REQUIRES
High-watermark tracking	The client library REQUIRES storing the last successfully processed Sequence Number per queue (O(1) state). The high-watermark is the reference point for gap detection, resume-from-position, and exactly-once deduplication for in-order delivery.	REQUIRES
Epoch change handler	On CONNACK where the broker-supplied Epoch differs from the client's stored Epoch for any queue, the client library MUST discard all stored idempotency state for that queue, update the stored Epoch, and trigger a resync before resuming consumption.	MUST
Consumer state machine	The client library REQUIRES implementing a state machine with the following states: `CONNECTING` , `HANDSHAKING` (v6.0 capability exchange), `FETCHING` , `PROCESSING` , `ACKNOWLEDGING` , `RESYNCING` . The profile	REQUIRES

Behavior	Application Profile Specification	Normative Level
	defines valid transitions and the conditions that trigger each.	
Exactly-once processing workflow	The profile defines the sequence: (1) receive message, (2) check Seq against high-watermark, (3) if duplicate, send PUBACK and skip application delivery, (4) if new, deliver to application, (5) on application success, advance high-watermark, (6) send PUBACK. Step 5 MUST occur before Step 6.	REQUIRES
Reconnect and resume	On reconnect to a v6.0 broker, the client library REQUIRES including `last-seq` and `current-epoch` as CONNECT User Properties for each queue the client was consuming. The broker uses these values to resume delivery from the correct position.	REQUIRES
Sequence wraparound	The client library MUST treat `Seq 0` following `Seq 0xFFFFFFFF` as a continuation, not a gap. If the broker increments the Epoch at wraparound, the library MUST handle this as an Epoch change event (see Epoch change handler above).	MUST
FETCH batch sizing strategy	The profile RECOMMENDS an adaptive batch sizing strategy: start with `batch-size = 1`, double on successful processing, halve on slow processing (processing time	RECOMMENDS

Behavior	Application Profile Specification	Normative Level
	exceeds a configurable threshold). This prevents the reconnect flood while allowing throughput to scale under stable conditions.	
Idempotency table management	For in-order delivery (standard `$queue/` topics), the profile REQUIRES the high-watermark model (O(1) state). Set-based idempotency tables are RECOMMENDED only when the application requires out-of-order delivery guarantees. On Epoch reset, the profile REQUIRES discarding the entire idempotency table as a garbage-collection boundary.	REQUIRES / RECOMMENDS
Consumer group coordination	The profile defines how to use SQMC `competing` mode for work-queue fan-out and `exclusive` mode for primary/standby failover. The profile also defines when application-level coordination is needed instead (e.g., cross-broker consumer groups spanning multiple `$queue/` instances require application-level partition assignment beyond what the protocol provides).	REQUIRES / defers

4. How This Addresses Committee Criticisms

Simon's "mux'ing" criticism

"There appears to be a mux'ing of what is expected of the application and what is expected of the protocol."

Option A resolves this directly. Section 2 of this document is a complete enumeration of broker obligations — the MQTT 6.0 Protocol Specification contains exactly and only those six items. Section 3 is a complete enumeration of client library obligations — every item that belongs at the application layer is explicitly placed in the Application Profile. The line is drawn and labeled. Simon is right that the current draft mixes these layers; Option A unmixes them.

The "this should be application-level" argument

Simon's implicit position — that gap detection, idempotency management, and consumer state machines belong in the application rather than the protocol — is **correct**, and Option A agrees with him. The Application Profile is precisely the document that specifies what IS at the application level. We are not pushing these behaviors into the protocol; we are standardizing them in their appropriate layer, using the Eclipse Foundation as the governing body (the same body that governs Sparkplug B, which already standardizes application-level sequencing on MQTT). The protocol spec stays narrow. The application behavior gets standardized where it belongs.

The "bloat" and Kafka-clone criticism

The MQTT 6.0 Protocol Specification under Option A contains six broker primitives. The protocol document is noticeably thinner than the current combined draft. Implementers building a compliant v6.0 broker need only implement Section 2 of this document. They do not need to implement gap detection algorithms, consumer state machines, or idempotency table strategies — those are client library concerns that live in the Application Profile. A minimal v6.0 broker footprint is achievable; the protocol is not attempting to replicate Kafka's consumer group coordinator.

The "standardize now vs. extension first" tension

Option A enables parallel standardization tracks that match the institutional capabilities of the appropriate governing bodies:

- **MQTT 6.0 Protocol Specification** — OASIS. Six broker primitives, wire format, property IDs `0x30`, `0x35`, `0x41`. This is protocol engineering; OASIS is the right home.
- **MQTT Stream Application Profile** — Eclipse Foundation, alongside Sparkplug B. Client library behavior, reference implementations, conformance test suites. This is application engineering; Eclipse Foundation already governs Sparkplug B and has the processes for this kind of document.
- **Reference implementation** — HiveMQ Extension SDK for the broker side; an open-source client library (Python/Java/Go) implementing the Application Profile for the client side.

Two tracks can advance simultaneously. If the OASIS process moves slowly, the Application Profile can be published and adopted while the protocol primitives are in draft. If the Application Profile needs revision (e.g., new batch sizing strategies emerge from production experience), it can be updated without reopening the OASIS protocol spec.

---

5. Feature Split: Comprehensive Reference Table

Feature	In MQTT 6.0 Protocol Spec?	In MQTT Stream Application Profile?	Who Implements
`$queue/` durable persistence before PUBACK	Yes	No	Broker
Queue TTL and max-size enforcement	Yes	No	Broker
Stream Sequence Number assignment (`0x30`)	Yes	No	Broker
Stream Epoch signaling (`0x35`) in CONNACK	Yes	No	Broker
FETCH request handling / Virtual FETCH dispatch	Yes	No	Broker
SQMC competing-consumer dispatch (round-robin)	Yes	No	Broker
SQMC exclusive-consumer	Yes	No	Broker

Feature	In MQTT 6.0 Protocol Spec?	In MQTT Stream Application Profile?	Who Implements
primary/standby failover			
Throughput Limit enforcement (`0x41`)	Yes	No	Broker
v6.0 handshake (`mqtt-ext:v6.0` in CONNECT/CONNACK)	Yes (detection only)	Yes (fallback logic)	Broker detects; client library handles fallback
Gap detection algorithm	No	Yes	Client library
High-watermark state tracking	No	Yes	Client library
Epoch change handler (discard idempotency state)	No	Yes	Client library
Consumer state machine	No	Yes	Client library
Exactly-once processing workflow	No	Yes	Client library
Reconnect/resume with `last-seq` + `current-epoch`	No (wire format only)	Yes (behavior)	Client library
Sequence wraparound handling	No	Yes	Client library
Adaptive FETCH batch sizing	No	Yes	Client library

Feature	In MQTT 6.0 Protocol Spec?	In MQTT Stream Application Profile?	Who Implements
Idempotency table strategy selection	No	Yes	Client library
Consumer group application-level coordination	No	Yes (scope limits defined)	Application / client library

---

6. Standardization Path

```
OASIS MQTT Technical Committee
  └── MQTT 6.0 Protocol Specification (broker primitives)
        Property 0x30 (Stream Sequence Number)
        Property 0x35 (Stream Epoch)
        Property 0x41 (Throughput Limit)
        $queue/ persistence semantics
        FETCH / Virtual FETCH broker behavior
        SQMC broker dispatch semantics

Eclipse Foundation (IoT Working Group)
  └── MQTT Stream Application Profile v1.0
        Gap detection algorithm
        Consumer state machine
        Epoch change handler
        Exactly-once processing workflow
        FETCH batch sizing strategy
        Reconnect/resume behavior
        Idempotency table management
  └── Sparkplug B (existing — application-layer sequencing on MQTT v5.0)

HiveMQ
  └── Reference implementation — broker (HiveMQ Extension SDK)
  └── Reference implementation — client (open-source, multi-language)
  └── Conformance test suite (donated to Eclipse Foundation)
```

The OASIS submission can proceed with only the six protocol primitives, making the committee's evaluation scope manageable. The Eclipse Foundation Application Profile can begin community review immediately, given the Eclipse Foundation's lighter governance process relative to OASIS. The two documents reference each other normatively but are independently versioned.

---

7. Trade-offs

Advantages

- **Protocol spec is genuinely slimmer.** The MQTT 6.0 OASIS submission covers six broker primitives, not a full consumer lifecycle specification. Committee members evaluating "is this appropriate for a protocol spec?" are evaluating a narrower document with a cleaner answer.
- **Simon's criticism is directly and structurally resolved.** The mux'ing is eliminated by design. Every feature has a designated layer and a designated governing body.
- **Application Profile can evolve faster.** If production experience reveals that the adaptive FETCH batch sizing strategy needs revision, the Eclipse Foundation can publish Application Profile v1.1 without reopening the OASIS protocol spec. Protocol stability and application-layer agility are decoupled.
- **Aligns with established precedent.** Sparkplug B proves that the MQTT ecosystem already accepts this two-layer model. The Application Profile is not a novel concept; it is an explicit formalization of what Sparkplug B does informally.
- **Appropriate governing bodies.** OASIS for wire format and broker obligations. Eclipse Foundation for client library behavior and reference implementations. Each body handles what it handles well.

Disadvantages

- **Two documents to maintain.** An implementer building a complete v6.0 consumer must read and conform to both the protocol spec and the Application Profile. Cross-document consistency must be actively managed.
- **Risk of Application Profile fragmentation.** If the Application Profile is not adopted widely, client libraries may implement the protocol primitives but diverge on the consumer state machine, gap detection, and idempotency strategies. This is the same fragmentation risk that motivated the proposal in the first place, now shifted one layer up. Mitigation: the reference implementation and conformance test suite must be published simultaneously with the Application Profile.
- **Compliance testing is split.** A "fully compliant v6.0 system" requires broker conformance to the protocol spec AND client library conformance to the Application Profile. Test harnesses must cover both layers. This is more complex than a single combined conformance suite.

References

- Simon's committee review feedback — quoted directly in `docs/rebuttals.md` , Section: "This Should Be Handled at the Application Level"
- `docs/rebuttals.md` — full point-by-point responses to Georg and Simon

- `docs/critiques-and-rebuttals.md` — technical pitfalls and mitigations (idempotency table growth, sequence wraparound, Epoch DoS)
- `docs/spec/mqtt-v6.0-spec.md` — current combined specification (the document that Option A proposes splitting)
- Eclipse Sparkplug B Specification — the governing precedent for this two-layer model
- MQTT v5.0 OASIS Standard — protocol layer reference

Option B: MQTT 6.0 with Normative Client Annex — Explicit Layer Boundary

Author: Bill Sommers, AI Solutions Lead, HiveMQ, Inc. **Date:** March 2026 **Status:** Proposal — For Committee Review

---

1. Overview

The Problem With the Current Spec Structure

The current `mqtt-v6.0-spec.md` interleaves two distinct categories of normative obligation throughout its sections. Section 3.1 specifies what a client MUST include in CONNECT; Section 4.3.3 specifies what a client MUST do on reconnect; Section 4.3.4 specifies how a client MUST manage its idempotency table. These are client library behaviors. Section 4.1.1 specifies what a broker MUST do with storage before issuing PUBACK; Section 4.3.1 specifies what a broker MUST do with a distributed sequence counter; Section 4.4.1 specifies how a broker MUST handle message locking. These are broker behaviors. Both categories appear in the same numbered sections, often in adjacent subsections, with no structural signal to the reader that they represent different contractual parties.

This is the source of Simon's "mux'ing" criticism: *"There appears to be a mux'ing of what is expected of the application and what is expected of the protocol."* The criticism is accurate. The spec is not wrong in content, but it is ambiguous in structure.

The Option B Solution

Option B retains the single-document structure — one OASIS submission — but reorganizes the content to make the architectural boundary explicit. The main body (Sections 1–7) becomes broker normative requirements only. A new **Annex A (normative)** collects all client library MUST/SHOULD behaviors. The existing code examples and informative implementation guidance move to **Annex B (informative)**.

The analogy is instructive: MQTT v5.0 itself has a main body defining the protocol and informative annexes providing implementation guidance. Option B promotes client behavior from informative to normative, and separates it from broker behavior by document structure rather than by convention.

This is not a reduction in scope. All five v6.0 features remain. The total set of normative requirements is unchanged. What changes is their organization: a reader evaluating broker conformance reads Sections 1–7. A reader implementing a client library reads Annex A. The mux'ing is resolved by separation within the document.

---

2. Proposed Spec Restructure

The reorganization maps the current spec content to three structural zones:

Zone	Scope	Normative?	Audience
Sections 1–7 (main body)	Broker obligations only	Yes	Broker implementers, OASIS committee
Annex A	Stream client library obligations	Yes	Client library authors
Annex B	Example implementations, code patterns	No (informative)	Application developers

Current content moves as follows:

Sections 1–7 (Broker Normative — Unchanged Scope, Tightened Audience)

- Section 1: Introduction, conformance, terminology — retained as-is
- Section 2: Wire format, property IDs, packet types — retained; client-side property interpretation moves to Annex A
- Section 3: CONNECT/CONNACK/PUBLISH/SUBSCRIBE/FETCH packet specifications — broker-side processing only; client-side obligations (what to send, when, with what values) move to Annex A
- Section 4: Operational behavior — `$queue/` persistence, ordering, FETCH handling, Epoch management, SQMC mechanics — broker obligations only
- Section 5: v5.0 vs. v6.0 comparison table — retained
- Section 6: Compatibility Layer — broker-side compat mode requirements
- Section 7: Security Considerations — retained as-is

Annex A (Normative) — Stream Client Implementation Requirements

New section. Contains all client library MUST/SHOULD behaviors currently scattered through Sections 3 and 4. See Section 3 of this document for the complete content specification.

Annex B (Informative) — Implementation Examples

Existing Annex A (SECS/GEM Mapping) moves here. Python shim examples, Java HiveMQ extension patterns, and the Protobuf schemas currently in `src/` are referenced here as non-normative implementation guidance.

---

3. Annex A Content: What Gets Lifted

Annex A is titled **"Stream Client Implementation Requirements"** and opens with the following normative statement:

> *A conformant Stream Client implementation MUST implement all MUST-level requirements in this Annex. Failure to implement these requirements results in undefined behavior with respect to exactly-once processing, gap detection, and epoch-safe resumption.*

A.1 Stream Consumer State Machine

The client library MUST implement the following states and transitions for each `$queue/` subscription:

```
States: {CONNECTING, HANDSHAKING, FETCHING, PROCESSING, ACKNOWLEDGING, RESYNCING}

CONNECTING     -> HANDSHAKING   : CONNACK received with mqtt-ext: v6.0 confirmed
HANDSHAKING    -> FETCHING      : Epoch check complete (see A.4); FETCH issued
HANDSHAKING    -> RESYNCING     : Epoch in CONNACK != stored Epoch
FETCHING       -> PROCESSING    : PUBLISH batch received from broker
PROCESSING     -> ACKNOWLEDGING : Business logic complete for message N
ACKNOWLEDGING  -> FETCHING      : PUBACK sent; batch exhausted or next FETCH issued
ACKNOWLEDGING  -> PROCESSING    : Next message in batch available (batch not yet
exhausted)
RESYNCING      -> FETCHING      : Application confirms new high-watermark; Epoch stored
Any state      -> CONNECTING    : TCP disconnect or DISCONNECT packet received
```

The client MUST NOT issue a FETCH request from any state other than FETCHING. The client MUST NOT process messages from any state other than PROCESSING.

A.2 Gap Detection

The client MUST maintain a variable `expected_seq` per subscribed `$queue/{name}`, initialized to `last_stored_hwm + 1` on connect (or `1` on first connect).

On receipt of each PUBLISH from a `$queue/` topic:

1. If `received_seq == expected_seq`: proceed to processing; set `expected_seq = received_seq + 1`
2. If `received_seq > expected_seq`: a gap exists between `expected_seq` and `received_seq - 1`. The client MUST invoke the application's registered gap handler before processing `received_seq`. The gap handler MAY choose to halt, reconcile, or log-and-continue — this is application policy. The client library MUST NOT silently skip the gap.
3. If `received_seq < expected_seq`: the message is a duplicate (already processed). The client MUST send PUBACK but MUST NOT re-execute business logic. The client MUST NOT update `expected_seq`.

A.3 High-Watermark Management

The client MUST persist the last successfully processed Stream Sequence Number per `$queue/{name}` as the **High-Watermark** (`hwm`). Persistence MUST be durable (survives process restart). The atomicity requirement: the High-Watermark update and the business-logic side effect MUST succeed or fail together. If the High-Watermark cannot be persisted, the client MUST NOT send PUBACK.

The client SHOULD use the High-Watermark as the `v6-last-seq` User Property value in CONNECT on reconnect. This enables the broker to resume delivery from `hwm + 1` without re-delivering already-processed messages.

For in-order `$queue/` delivery (the normal case), the High-Watermark model is O(1) in storage. A set-based idempotency table is only required when the application enables delivery modes that can produce out-of-order receipt (not the default).

A.4 Epoch Change Protocol

On receiving CONNACK, the client MUST compare the Epoch value in CONNACK against its stored Epoch for each subscribed `$queue/{name}`.

If `connack_epoch != stored_epoch`:

1. The client MUST discard its local idempotency state for the affected queue (High-Watermark, `expected_seq`, and any set-based dedup table)
2. The client MUST log the Epoch change with: timestamp, queue name, previous Epoch value, new Epoch value
3. The client MUST invoke the application's registered epoch-change handler and MUST wait for it to return a new high-watermark before transitioning to FETCHING
4. The client MUST store the new Epoch before issuing any FETCH request

5. The client MUST NOT attempt to use sequence numbers from the previous Epoch for deduplication

If `connack_epoch == stored_epoch`, the client MAY proceed directly to the FETCHING state using its stored High-Watermark.

A.5 Reconnect and Resume

The client SHOULD include the following User Properties in CONNECT after any disconnection from a queue it was consuming:

- `v6-last-seq` : the decimal string of the stored High-Watermark for the most recently consumed queue
- `v6-epoch` : the decimal string of the stored Epoch for that queue

If multiple queues are active, the client SHOULD send one `v6-last-seq` / `v6-epoch` pair per queue using repeated User Properties keyed by queue name (e.g., `v6-last-seq:$queue/line_1`).

The client MUST NOT send a `v6-last-seq` value that has not been durably persisted. Reporting a false high-watermark (higher than the true last processed sequence) causes silent message loss.

A.6 FETCH Pacing Strategy

The client MUST NOT issue overlapping FETCH requests for the same `$queue/{name}`. A FETCH is outstanding from the moment it is sent until the full batch has been acknowledged (all PUBACKs sent for the batch).

The RECOMMENDED initial batch size is 1. The RECOMMENDED pacing algorithm is additive doubling on success: if a batch of N is fully processed without error, the next FETCH MAY use batch size 2N, up to a configurable maximum. On processing error, the client SHOULD reduce batch size to 1 and re-FETCH from the last acknowledged sequence.

The client MAY issue the next FETCH before the current batch is fully acknowledged, provided the outstanding FETCH is for the immediately following sequence range (pipelining). The client MUST track which sequences are outstanding across pipelined requests.

A.7 Idempotency Table Lifecycle

The High-Watermark (Section A.3) is sufficient for exactly-once processing under in-order delivery, which is the invariant provided by `$queue/` ordered delivery. The client SHOULD use the High-Watermark model and SHOULD NOT maintain a set-based idempotency table unless the application explicitly requires processing of out-of-order delivery.

If a set-based table is used, the client MUST garbage-collect entries with sequence numbers at or below the current High-Watermark. Epoch changes (Section A.4) are natural garbage-collection boundaries: the client MUST discard the entire set on Epoch change.

The client MUST NOT allow the idempotency table to grow without bound. If set size exceeds a configurable threshold, the client SHOULD log a warning and SHOULD trigger a manual reconciliation request.

A.8 Sequence Wraparound

The 32-bit Stream Sequence Number space is exhausted at `0xFFFFFFFF`. The client MUST treat `Seq 0` received after `Seq 0xFFFFFFFF` as a valid continuation, not as a gap. The broker MUST couple wraparound with an Epoch increment (per Section 4.3.2.2 of the main body); the client's Epoch change protocol (Section A.4) handles the state reset automatically.

The client MUST NOT treat a sequence gap of magnitude `0xFFFFFFFF` or greater as evidence of missing messages; it MUST instead treat it as evidence of a wraparound event and proceed through the Epoch change protocol.

---

4. How This Addresses Committee Criticisms

Simon's "Mux'ing" Criticism

> *"There appears to be a mux'ing of what is expected of the application and what is expected of the protocol."*

Option B response: The Annex A boundary makes the separation explicit and auditable within the document. Sections 1–7 (main body) = broker contract. Annex A = client contract. The mux'ing is resolved by structural separation within the spec. A committee member evaluating broker conformance reads through Section 7 and stops. A committee member evaluating client library conformance reads Annex A. The two audiences no longer read the same section headers as addressing them both.

The "Application-Level" Argument

> *The objection that gap detection, epoch handling, and consumer state management are application-level concerns and should not be in a protocol spec.*

Option B response: Annex A acknowledges this directly. Gap detection, Epoch handling, and consumer state management ARE application-layer responsibilities — they are normative requirements on the **client library**, not on the broker, and they are placed in an Annex explicitly to signal this. The main body does not require applications to manage sequence state; it requires a conformant client library to do so. This mirrors how MQTT v5.0 itself specifies client behavior (QoS retry, session resumption) separately from broker behavior, without those client behaviors being "mixed into" broker sections.

Procedural Argument

> *The concern that two separate documents (a hypothetical Option A split) create submission and versioning complexity at OASIS.*

Option B response: A single document is easier to submit to OASIS as one coherent standard. The committee evaluates broker and client behavior together in a single review cycle. There is no synchronization risk between two documents, no question of which version of a client spec corresponds to which version of a broker spec, and no procedural ambiguity about whether Annex A is binding.

The "Bloat" Criticism

> *"MQTT's success is its simplicity. v6.0 turns it into a bloated Kafka clone." (Criticism 1 in `critiques-and-rebuttals.md`)*

Option B response: The main body, after restructuring, contains only broker obligations. Sections 1–7 are trimmed of all client-side MUST/SHOULD language, which moves to Annex A. The perceived bloat was the interleaving — a reader scanning the main body encountered client library state machine requirements alongside broker wire-format requirements. After restructuring, the main body is visibly shorter and more focused. The total normative content is unchanged, but each section addresses a single audience.

---

5. Comparison: Current Spec vs. Option B Structure

Spec Content	Current Location	Option B Location	Type
Property IDs 0x30, 0x35, 0x41, 0x42 wire format	Section 2.2	Section 2.2 (unchanged)	Normative
FETCH packet structure	Section 3.5	Section 3.5 (unchanged)	Normative
Broker: assign Seq before PUBACK	Section 4.1.1	Section 4.1.1 (unchanged)	Normative
Broker: Epoch increment conditions	Section 4.3.2.2	Section 4.3.2.2 (unchanged)	Normative
Broker: SQMC message locking	Section 4.4.1	Section 4.4.1 (unchanged)	Normative
Client: send v6-last-seq in CONNECT	Section 3.1 (mixed)	**Annex A.5**	Normative
Client: discard idempotency on Epoch change	Section 2.2.2 (mixed)	**Annex A.4**	Normative
Client: High-Watermark persistence	Section 4.3.4 (mixed)	**Annex A.3**	Normative
Client: gap detection algorithm	Section 4.3.4 (implicit)	**Annex A.2**	Normative
Client: FETCH pacing	Not specified	**Annex A.6**	Normative
Client: state machine	Not specified	**Annex A.1**	Normative
Client: sequence wraparound handling	`critiques-and-rebuttals.md` Pitfall 1 (informative)	**Annex A.8**	Normative
SECS/GEM mapping	Annex A (current)	Annex B (informative)	Informative
Python/Java code examples	`src/` directory	Annex B (informative)	Informative
Protobuf schemas	`src/` directory	Annex B (informative)	Informative

6. Trade-offs

Pros

- **Single OASIS submission.** One document, one submission, one review cycle. No fragmentation risk between a broker spec and a client spec.
- **Full system evaluated together.** The committee sees both broker and client obligations in one pass. The interaction between broker Epoch propagation (Section 4.3.2.3) and client Epoch change protocol (Annex A.4) is visible in context.
- **No new scope.** All five v6.0 features are preserved. Annex A collects and formalizes client obligations that are already implied by the current spec; it does not add new ones.
- **Addresses Simon's criticism directly.** The mux'ing is resolved by the Annex boundary without splitting the document.
- **Lighter main body.** Sections 1–7 become visibly more focused after client-side MUST language is lifted to Annex A.

Cons

- **Annex A is still in the spec.** Committee members who object to the total scope (gap detection, state machine, epoch handling in a protocol spec) will still see it. Option B separates it structurally; it does not remove it. If the objection is to scope rather than to presentation, Option B does not fully resolve it.
- **Client library improvements require OASIS revision.** Because Annex A is normative and inside the spec, any update to client library MUST/SHOULD requirements (e.g., adding a new state to the state machine, or revising the FETCH pacing algorithm) requires a formal OASIS revision cycle. This is slower than maintaining a separate client implementation guide.
- **Less clean than a full split.** Option A (a separate client implementation specification document) would provide a more complete architectural separation. Option B is a compromise — it draws the boundary explicitly, but both sides of the boundary remain in the same document.

---

7. What the Main Body (Sections 1–7) Looks Like After Restructure

After lifting all client-side MUST/SHOULD language to Annex A, the main body sections become:

Section 1 — Introduction and Conformance: Unchanged. Conformance criteria updated to reference Annex A for client library conformance.

Section 2 — Wire Format: Broker-only: property ID assignments, data types, packet type definitions. Client-side property interpretation (e.g., "a client that receives an Epoch value greater than its stored Epoch MUST…") moves to Annex A.4.

Section 3 — Control Packets: Broker-only processing rules for CONNECT, CONNACK, PUBLISH, SUBSCRIBE, FETCH. What the broker MUST validate, reject, or respond to. Client-side sending obligations (what values to populate, when) move to Annex A.5.

Section 4 — Operational Behavior: Broker-only: `$queue/` persistence contract, Epoch increment conditions and propagation, SQMC locking semantics. The "Exactly-Once Processing" subsection (Section 4.3.4) moves entirely to Annex A, as it describes client library behavior with no broker obligation.

Section 5 — Summary of Changes: Unchanged comparison table.

Section 6 — Compatibility Layer: Broker-side compat mode requirements. Client-side compat mode behavior (which User Property to send instead of which native property) moves to Annex A.

Section 7 — Security Considerations: Unchanged. Sections 7.3 and 7.4 (Sequence Number Side-Channel and High-Watermark Persistence) move to Annex A, as both describe client implementation obligations.

The result is a main body of roughly 60–70% of the current length, with all remaining content directly addressing broker implementers. The substantive content removed from the main body does not disappear — it becomes the formally bounded client contract in Annex A.

References

- MQTT v5.0 OASIS Standard: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html
- Current v6.0 specification: `docs/spec/mqtt-v6.0-spec.md`
- Standards committee criticisms: `docs/rebuttals.md` , `docs/critiques-and-rebuttals.md`

Option C: Two Independent Specifications — MQTT 6.0 Transport Extension + MQTT Stream 1.0 Application Protocol

> ***Author:*** *Bill Sommers, AI Solutions Lead, HiveMQ, Inc.* ***Date:*** *March 2026* ***Status:*** *Proposal — for standards committee review*

1. Overview

Option C is the maximum-separation answer to the committee's structural criticisms. It dissolves the layering ambiguity by making the boundary absolute and architectural: two distinct specifications, governed independently, versioned independently, with a clean contractual interface between them.

MQTT 6.0 becomes a minimal wire-format extension specification. It defines new property IDs, the `$queue/` namespace semantics, and broker-side behavioral requirements. It contains no client behavioral requirements beyond what is necessary to negotiate the extension. It does not define state machines, gap-detection algorithms, epoch-handling procedures, or application patterns. It is a transport extension, not an application protocol.

MQTT Stream 1.0 is a separate specification that defines the industrial reliability application protocol built on top of MQTT 6.0. It defines the consumer state machine, the gap-detection algorithm, the epoch-resync procedure, the exactly-once delivery contract, and FETCH pacing strategies. It references MQTT 6.0 as its required transport layer. It is to MQTT 6.0 what Sparkplug B is to MQTT 3.1.1 — an application-layer profile that leverages the transport primitives without being conflated with them.

This is not a novel pattern. The protocol ecosystem has used it for decades:

- TCP/IP defines reliable byte-stream delivery. HTTP defines an application protocol for document transfer over TCP.
- MQTT 3.1.1 defines a lightweight pub/sub transport. Sparkplug B defines an application protocol for industrial telemetry over MQTT.
- CoAP defines a constrained application protocol over UDP. LwM2M defines a device management application protocol over CoAP.

Option C places MQTT 6.0 and MQTT Stream 1.0 in that same well-understood relationship. The transport layer provides primitives. The application layer defines what to do with them.

---

2. MQTT 6.0 Transport Extension Specification

The MQTT 6.0 spec is a **broker-only behavioral specification**. It defines what a conformant broker MUST do. It imposes no normative requirements on client application logic.

2.1 What MQTT 6.0 Defines

Feature	Specification Content
Property 0x30: Stream Sequence Number	Four Byte Integer. A conformant broker MUST assign a monotonically increasing, per- `$queue/` , cluster-wide sequence number

Feature	Specification Content
	to every message it persists. The broker MUST include Property 0x30 in the PUBLISH delivered to consumers. Clients MUST NOT set this property; if present in a client PUBLISH, the broker MUST overwrite it.
Property 0x35: Stream Epoch	Two Byte Integer. A conformant broker MUST include this property in CONNACK when the client presents a `last-seq` in CONNECT. If the broker cannot guarantee sequence continuity from `last-seq` to the current queue head, it MUST set a new Epoch value. What constitutes "cannot guarantee" is implementation-defined; the signal is normative, the trigger is not.
Property 0x41: Throughput Limit	Four Byte Integer (KB/s). A conformant broker MAY include this property in CONNACK to enforce a per-connection bandwidth ceiling. Client behavior upon receiving this property is advisory only from the MQTT 6.0 perspective; MQTT Stream 1.0 defines normative client handling.
`$queue/` Namespace	A conformant broker MUST persist a message published to a `$queue/` topic to non-volatile storage before sending PUBACK to the publisher. A `$queue/` entity MUST exist independently of any client session. A conformant broker MUST NOT discard a `$queue/` message because all consumers are disconnected.
Virtual FETCH	A conformant broker MUST support pull-based delivery via PUBLISH to `$SYS/queues/{name}/fetch` with `batch-size` User Property. The broker MUST hold messages in the queue until a FETCH is received; it MUST NOT push messages to a `$queue/` consumer except in response to a FETCH.
SQMC Semantics	A conformant broker MUST honor the `v6-semantics` User Property in SUBSCRIBE.

Feature	Specification Content
	`competing` mode: broker distributes each message to exactly one consumer in the group. `exclusive` mode: broker delivers all messages to a single designated consumer; on that consumer's disconnect, the broker MUST transfer delivery to the next-eligible subscriber without message loss.
v6.0 Handshake	A client signals v6.0 support by including `User Property: ("mqtt-ext", "v6.0")` in CONNECT. A conformant broker MUST echo this property in CONNACK. A client that does not receive the echo MUST NOT use `$queue/` topics or rely on Property 0x30/0x35 semantics.

2.2 What MQTT 6.0 Explicitly Does NOT Define

The following are out of scope for the MQTT 6.0 transport extension spec:

- How clients detect sequence gaps
- How clients respond to an Epoch change
- The client-side consumer state machine
- Exactly-once delivery guarantees at the application layer
- The high-watermark persistence protocol
- FETCH batch-size selection algorithms or pipelining strategies
- The reconnect protocol (what `last-seq` and `current-epoch` values clients SHOULD present in CONNECT)
- Sequence wraparound handling at the client side
- Application patterns for competing vs. exclusive consumer modes

These belong to MQTT Stream 1.0.

---

3. MQTT Stream 1.0 Application Protocol Specification

MQTT Stream 1.0 is an application-layer protocol specification that references MQTT 6.0 as its required transport extension. A conformant MQTT Stream 1.0 client library implements all of the following normatively.

3.1 Stream Consumer State Machine

A conformant client MUST implement the following FSM:

```
CONNECTING → HANDSHAKING → FETCHING → PROCESSING → ACKNOWLEDGING
                ↑                                          │
                └──────────── RESYNCING ←──────────────────┘
                                  ↑
                        (Epoch change detected)
```

State	Entry Condition	Exit Condition
CONNECTING	Client initiates TCP/TLS connection	CONNACK received
HANDSHAKING	CONNACK received	Broker echoes `mqtt-ext: v6.0`; client loads persisted high-watermark
FETCHING	HANDSHAKING complete	FETCH sent; batch received
PROCESSING	Batch received	All messages in batch processed by application
ACKNOWLEDGING	Processing complete	PUBACK sent for all batch messages; high-watermark advanced
RESYNCING	Epoch in CONNACK differs from client's stored epoch	Client discards idempotency state, resets high-watermark, returns to HANDSHAKING

3.2 Gap Detection Protocol

A conformant client MUST implement the following algorithm on each received message:

1. Let `expected_seq = high_watermark + 1`
2. Let `received_seq` = Property 0x30 of the received PUBLISH
3. If `received_seq == expected_seq`: advance `high_watermark`, process message, go to step 1
4. If `received_seq > expected_seq`: a gap exists. The client MUST NOT process messages beyond the gap. The client MUST send a gap notification to the application layer with `(expected_seq, received_seq - 1, queue_name)`. The client MAY request gap-fill (implementation-defined recovery) or enter RESYNCING
5. If `received_seq < expected_seq`: duplicate. The client MUST silently discard the message without advancing `high_watermark` or notifying the application

3.3 Epoch Resync Protocol

On receiving a CONNACK where Property 0x35 differs from the client's stored epoch:

1. Log the epoch transition with stored epoch, new epoch, queue name, and timestamp

2. Discard the entire in-memory idempotency state for the affected queue(s)
3. Reset `high_watermark` to the value indicated by the broker's CONNACK (or 0 if not provided)
4. Notify the application layer that sequence continuity cannot be guaranteed for the window between the last-acknowledged sequence and the new queue head
5. Persist the new epoch value
6. Return to HANDSHAKING state

3.4 High-Watermark Protocol

- The client MUST persist `high_watermark` (last successfully processed and acknowledged sequence number) to durable local storage after each successful ACKNOWLEDGING state transition
- On reconnect, the client MUST present the persisted `high_watermark` as `last-seq` in the CONNECT User Property
- The high-watermark MUST NOT advance until the application layer confirms processing is complete and PUBACK has been sent
- For ordered `$queue/` delivery, a single integer watermark is sufficient (O(1) storage). Set-based idempotency tables are not required

3.5 Exactly-Once Delivery Contract

The full client-side exactly-once workflow:

1. Receive PUBLISH with Property 0x30 = `seq`
2. If `seq <= high_watermark` : discard (duplicate), do not deliver to application
3. If `seq > high_watermark + 1` : gap detected; enter gap-handling procedure (Section 3.2)
4. If `seq == high_watermark + 1` : deliver to application
5. Application confirms processing complete
6. Send PUBACK to broker
7. Persist `high_watermark = seq`

Steps 5–7 MUST be atomic from the application's perspective: the watermark MUST NOT be advanced if PUBACK fails, and PUBACK MUST NOT be sent if application processing fails.

3.6 FETCH Pacing

- **Baseline algorithm:** Request `batch-size = min(processing_capacity, max_batch)` where `processing_capacity` is the number of messages the consumer can process before the next FETCH
- **Pipelining:** A client MAY issue the next FETCH before the current batch is fully acknowledged, provided the pipeline depth does not exceed `Receive Maximum`
- **Back-pressure signaling:** If application processing time exceeds a configurable threshold, the client MUST reduce `batch-size` by 50% on the next FETCH. Recovery to full batch size

MUST be gradual (linear increase per successful batch)
- **Idle behavior:** If the broker returns an empty batch, the client SHOULD use long-polling (`Wait-Timeout` User Property) on the next FETCH rather than tight-loop polling

3.7 Reconnect Protocol

On reconnect, the client MUST include in CONNECT:

- `User Property: ("last-seq", "<high_watermark>")` — the last durably processed sequence number
- `User Property: ("current-epoch", "<stored_epoch>")` — the epoch value the client last saw

The broker uses these values to determine whether to increment the Epoch in CONNACK and to identify the resume point in the queue. A broker that receives `last-seq` beyond the current queue head MUST signal an error condition.

3.8 Sequence Wraparound Rules

- A 32-bit sequence wraps at 2^32 - 1. The client MUST treat Seq 0 following Seq 0xFFFFFFFF as a continuation, not a gap
- The broker MUST increment Property 0x35 (Epoch) coincident with sequence wraparound to provide an unambiguous continuity marker
- Clients MUST NOT assume that a lower sequence number following a higher one indicates a gap if the Epoch has also changed

3.9 SQMC Application Patterns

Competing mode (`v6-semantics: competing`): - Use for stateless consumers that can independently process any message in the queue - Deploy N consumer instances, each maintaining its own high-watermark - On consumer failure, surviving consumers continue processing; the failed consumer’s in-flight messages are requeued by the broker and dispatched to the next available consumer

Exclusive mode (`v6-semantics: exclusive`): - Use for stateful consumers where ordering is safety-critical (SECS/GEM command streams, protection relay sequences) - Deploy one active consumer and one standby consumer subscribed to the same queue - On active consumer failure, the broker promotes the standby automatically within one broker-side timeout interval - The standby MUST present its high-watermark in CONNECT so the broker can resume from the correct position

---

4. How This Addresses Committee Criticisms

4.1 Simon's "Mux'ing" Criticism

> *"There appears to be a mux'ing of what is expected of the application and what is expected of the protocol."*

Option C is the most complete answer to this criticism. MQTT 6.0 contains zero application-layer concerns: no state machines, no gap-detection algorithms, no consumer patterns. MQTT Stream 1.0 contains zero broker-internal concerns: no persistence requirements, no property ID definitions, no packet formats. The line is absolute. A reader of the MQTT 6.0 spec cannot find application-layer logic in it. A reader of the MQTT Stream 1.0 spec cannot find wire-format definitions in it. The mux'ing is eliminated by structural separation, not by argument.

4.2 The "This Should Be Application-Level" Argument

We agree. MQTT Stream 1.0 IS the application-level standard. It lives in a separate document, governed separately, versioned separately, with its own conformance test suite. The committee members who raised this concern were correct: gap detection, epoch handling, and consumer state machines belong at the application layer. Option C puts them there.

4.3 The "Bloat / Kafka Clone" Criticism

The MQTT 6.0 spec is a small transport extension. It does not define application patterns. It specifies five new broker behaviors and three new property IDs. The spec document is short. It is not trying to be Kafka. MQTT Stream 1.0 provides Kafka-like application semantics as a separate, opt-in specification. A broker vendor who implements MQTT 6.0 is not required to implement MQTT Stream 1.0. A client developer who targets MQTT Stream 1.0 gets Kafka-like semantics over MQTT without the Kafka footprint. These are separable decisions.

4.4 The "Standardize Now vs. Extension First" Argument

MQTT 6.0 goes to OASIS as a minimal transport extension — a narrow, well-scoped document that broker vendors can implement incrementally. MQTT Stream 1.0 can be submitted to the Eclipse Foundation (as a Sparkplug-family specification) or as an OASIS TC note — a separate track with faster iteration cycles. The application protocol can evolve without reopening the OASIS MQTT 6.0 standard. This decouples the governance cadence from the innovation cadence.

4.5 Property ID Conflict Risk

A minimal MQTT 6.0 spec using only Property 0x30 (Stream Sequence Number) and Property 0x35 (Stream Epoch) — and optionally 0x41 (Throughput Limit) — is a far smaller OASIS ask

than a comprehensive spec. Smaller footprint means fewer property ID claims, fewer normative requirements, and less surface area for OASIS technical review to find objections. The minimal-scope approach maximizes the probability of OASIS approval for the transport layer while leaving the application layer free to evolve.

---

5. Governance Split

Specification	Governing Body	Version Cadence	Who Contributes	Role
MQTT 6.0	OASIS MQTT TC	Stable / slow (years per revision)	Broker vendors, protocol architects, OASIS members	Foundational transport primitives — changes here affect the entire ecosystem
MQTT Stream 1.0	Eclipse Foundation (or OASIS TC Note)	Faster iteration (months per revision)	Application developers, industrial users, client library authors	Application-layer profile — changes here affect Stream-conformant clients only

The governance split mirrors the Sparkplug B precedent: MQTT 3.1.1 is an OASIS standard governed by the MQTT TC; Sparkplug B is an Eclipse specification governed by the Sparkplug Working Group. They have different maintainers, different release schedules, and different audiences. MQTT Stream 1.0 would occupy the same structural position relative to MQTT 6.0 that Sparkplug B occupies relative to MQTT 3.1.1 — except that MQTT Stream 1.0 would be an officially recognized companion specification rather than a third-party profile.

---

6. Interoperability Contract

An MQTT Stream 1.0 conformant client library + an MQTT 6.0 conformant broker = a fully interoperable industrial messaging system.

The contract is bidirectional:

- **MQTT 6.0 without MQTT Stream 1.0:** A broker that implements MQTT 6.0 provides the transport primitives. Any client that can issue a FETCH control publish and read Property 0x30 can consume from a `$queue/` — but the exactly-once guarantees and gap-recovery procedures are application-defined. This is sufficient for broker vendors who want to offer the primitives and let client libraries define their own application profiles.

- **MQTT Stream 1.0 without MQTT 6.0:** Not possible. MQTT Stream 1.0 is normatively dependent on MQTT 6.0. A client library that claims MQTT Stream 1.0 conformance MUST run on an MQTT 6.0 broker; it MUST NOT claim conformance when connected to an MQTT 5.0 broker.

- **Both together:** Full industrial reliability stack. Gap detection, epoch resync, high-watermark persistence, exactly-once delivery, and FETCH pacing are all specified normatively. A semiconductor fab can run any MQTT Stream 1.0 client library against any MQTT 6.0 broker and get identical behavior — no vendor-specific payload conventions, no custom middleware, no Kafka bridge.

Neither specification is complete without the other for the industrial use case. But each is complete for its own layer, and each can be implemented, tested, and certified independently.

---

7. Trade-offs

Advantages

- **Maximum committee acceptance for MQTT 6.0.** A narrow transport extension spec is the easiest possible OASIS ask. Reviewers who object to application-layer content in a protocol spec will find nothing to object to.
- **MQTT Stream 1.0 evolves faster.** Eclipse Foundation governance moves faster than OASIS. The application protocol can be updated as industrial requirements evolve without reopening the MQTT 6.0 OASIS standard.
- **Clear governance split.** Broker vendors own MQTT 6.0 conformance. Client library authors own MQTT Stream 1.0 conformance. Responsibilities are unambiguous.
- **Clean precedent.** The Sparkplug B model is understood by the standards community. Option C fits a known pattern.

Disadvantages

- **Two separate standardization processes.** The working group must maintain two documents, two conformance test suites, and two submission tracks. This is more work than a single integrated specification.
- **Interoperability requires conformance to both specs.** Implementers who want full industrial reliability must implement both layers. The answer to “which spec do I implement?” is “both, depending on your role.” This adds explaining overhead, particularly for new adopters.
- **Risk of MQTT Stream 1.0 underadoption.** If MQTT 6.0 ships and MQTT Stream 1.0 fails to gain traction, the transport primitives (Property 0x30, Property 0x35, `$queue/`) may be underutilized or reimplemented inconsistently by client library authors — recreating the fragmentation problem at the application layer instead of the protocol layer.

- **Conformance surface area.** A client library can claim “MQTT 6.0 compatible” without implementing the Stream state machine. This may create a false sense of industrial readiness in the ecosystem.

---

8. Analogues in Other Protocol Ecosystems

This separation pattern is well-established and understood by standards bodies. The table below shows the direct parallels.

Transport Spec	Application Profile	Relationship	Parallel to Option C
MQTT 3.1.1 (OASIS)	**Sparkplug B** (Eclipse)	Sparkplug defines payload encoding, sequence numbers, and state management on top of MQTT pub/sub	Most direct analogue: MQTT Stream 1.0 would be the official Sparkplug B for MQTT 6.0
HTTP/1.1 (IETF RFC 2616)	**REST** (Fielding, 2000)	REST is an architectural style for distributed hypermedia systems over HTTP; it is not an HTTP extension	Architectural style defined separately from the transport protocol
CoAP (IETF RFC 7252)	**LwM2M** (OMA SpecWorks)	LwM2M defines a device management application protocol over CoAP; CoAP defines the constrained transport	IoT transport + IoT application management profile, separate governance bodies
AMQP 1.0 (OASIS)	**JMS 2.0** (Jakarta EE)	JMS defines a Java messaging API that can be backed by AMQP 1.0 as a wire transport; the API spec and the wire spec are independent	API/application spec built on top of a wire transport standard

The CoAP + LwM2M parallel is particularly apt: both are IoT protocols, CoAP is IETF-governed and LwM2M is OMA-governed, and the separation is clean enough that LwM2M implementations can run over CoAP or HTTP depending on the deployment. MQTT Stream 1.0 over MQTT 6.0 follows exactly this model.

The Sparkplug B parallel is the most persuasive for this committee: Sparkplug already defines application-layer sequencing (`bdSeq` , `seq` fields in protobuf payloads), birth/death certificate state management, and reconnect procedures on top of MQTT 3.1.1 — and it works. MQTT Stream 1.0 would provide those same guarantees at the wire level, under official MQTT ecosystem governance, for any MQTT 6.0 broker. Option C is not a novel idea. It is the formalization of a pattern the MQTT community already uses in production.

References

- MQTT v5.0 OASIS Standard
- Eclipse Sparkplug B Specification
- OMA LwM2M Specification
- RFC 7252: CoAP
- AMQP 1.0 OASIS Standard
- MQTT v6.0 Proposal Spec: `docs/spec/mqtt-v6.0-spec.md`
- Committee Rebuttals: `docs/rebuttals.md`
- Critiques and Rebuttals: `docs/critiques-and-rebuttals.md`
- Alternative Approaches: `docs/alternatives.md`